

Operating System

ICT1308

M.M.A.Nisansala

CONTENT

- Introduction
- Process Management
- Synchronization
- Memory management
- Deadlock
- File Management
- Disc Management

CHAPTER I-

Introduction

Computer System

- Computer system is a collection of hardware and software components designed to provide an effective tool for computation.
- Hardware refers to electrical, mechanical, and electronic parts that make up the computer.
- Software refers to the set of programs written to provide services to the system. It gives life and meaning to hardware and brings it to the operational level.
- Software's are basically two types
 1. Application software
 2. System Software

Computer System

1. Application software

- Set of programs written for a specific area of application. Examples are word processors, spreadsheets, and database management systems.

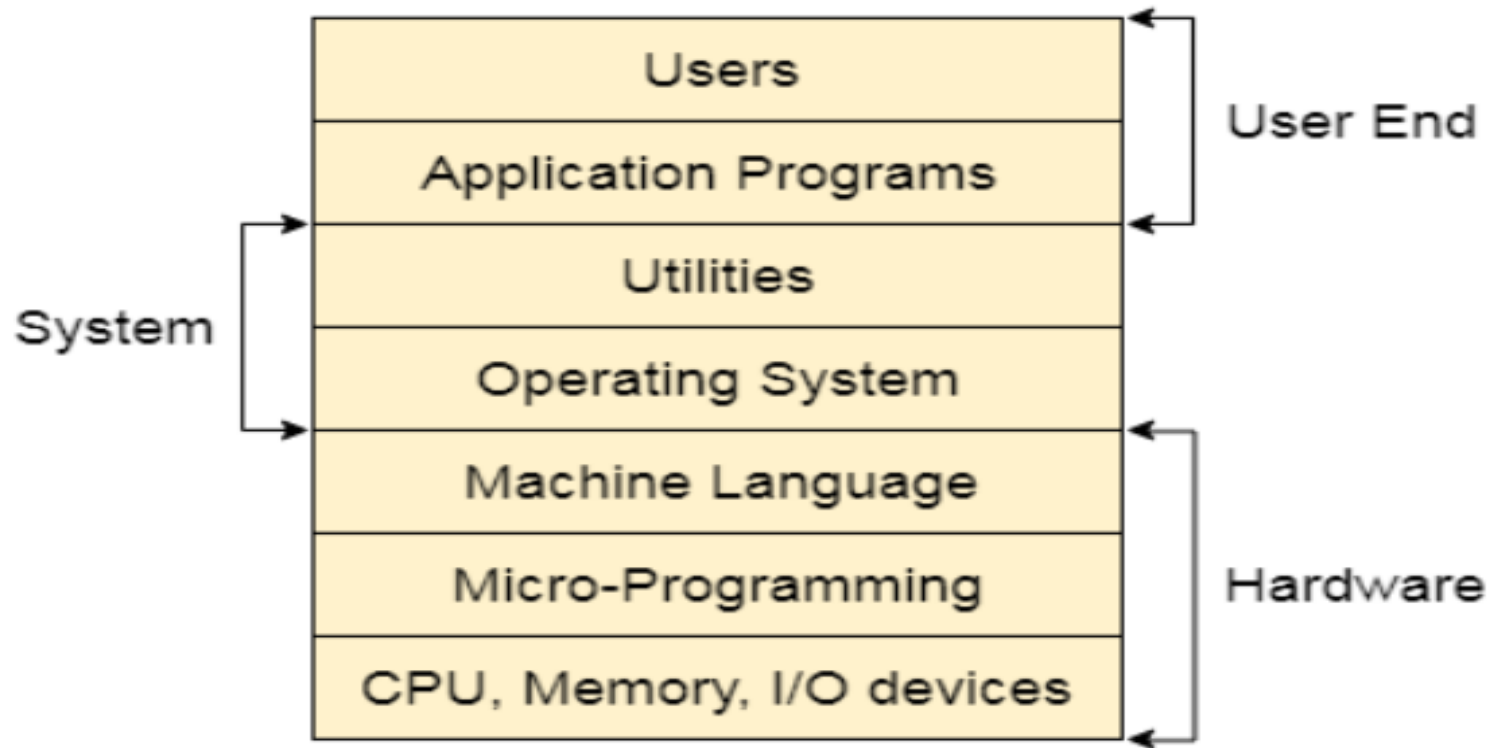
2. System software

- Set of programs written for running computer hardware and applications and managing its resources such as memory, processors, and resources.
- Provide a platform for running application software.

Structure of a computer system

- Computer system consists of:
 - **Users** -People who use the computer
 - **Application programs**-Compilers,Databases, Games, Video players, browsers, etc.
 - **System programs**-Shells, Editors etc.
 - **Operating system** -Special program which acts as a interface between the user and hardware
 - **Hardware**-CPU, Disk, memory, etc.

Structure of a computer system

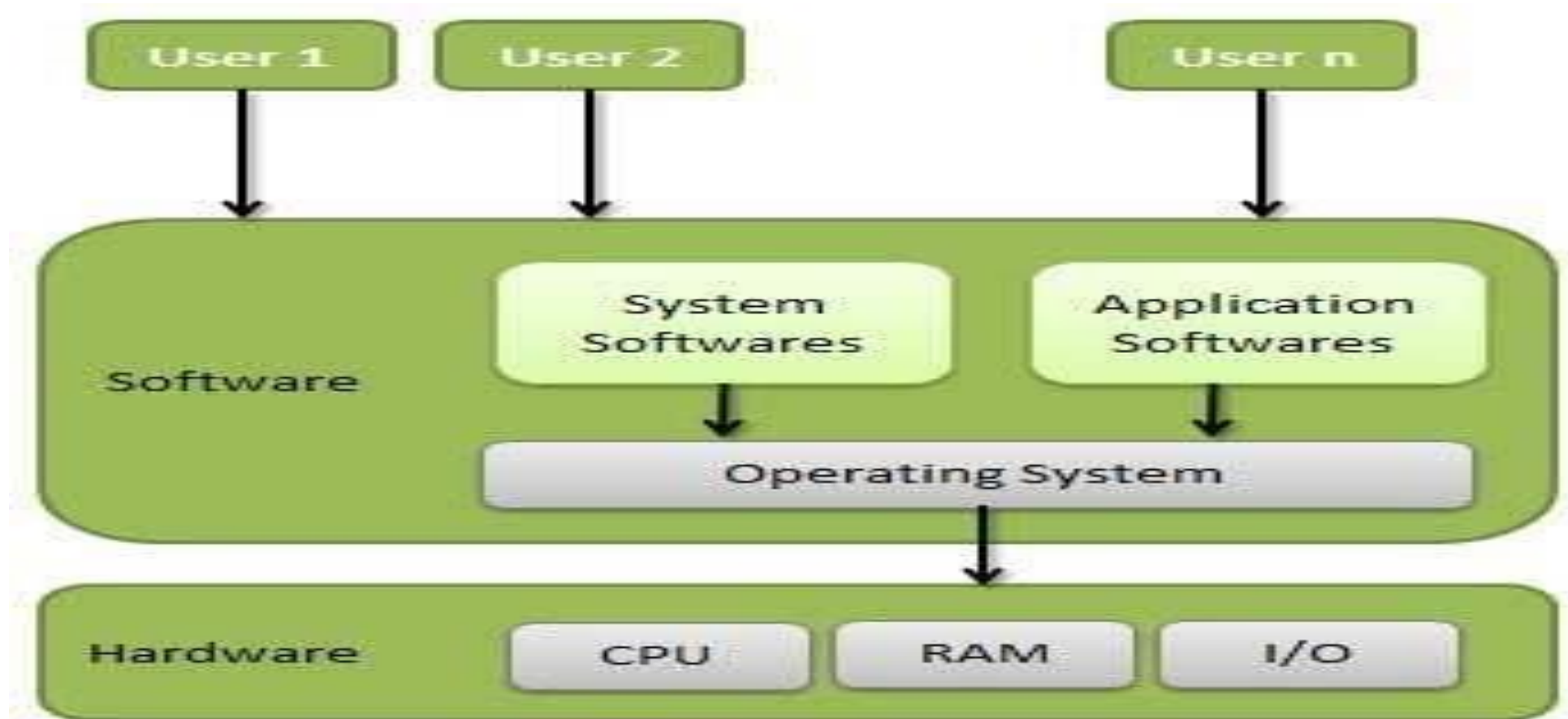


What is an Operating System?

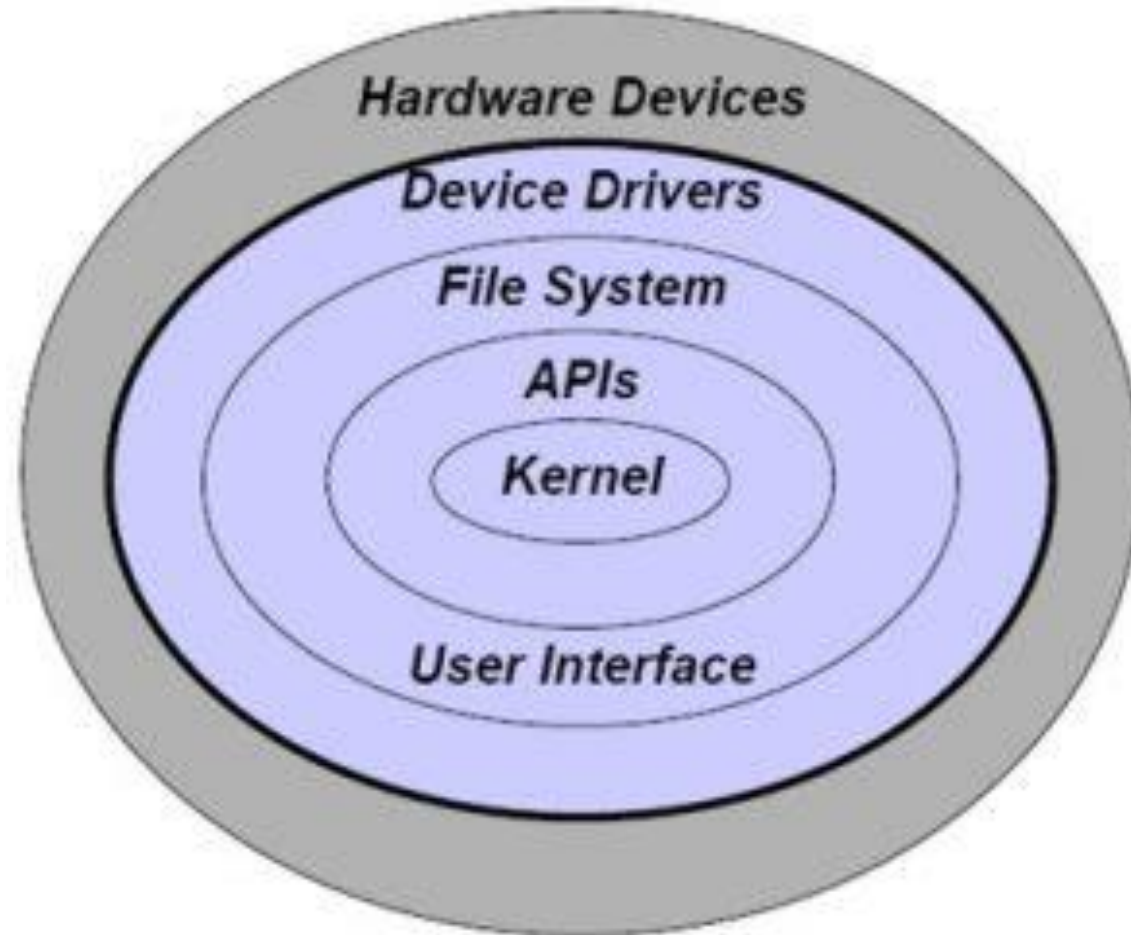
- OS is a program that acts as an intermediary between a user of a computer and the computer hardware.
- An OS is a collection of system programs that together control the operation of a system.

Examples-UNIX, MS-DOS, MS-Windows, Windows/NT, MacOS

Structure of Operating System

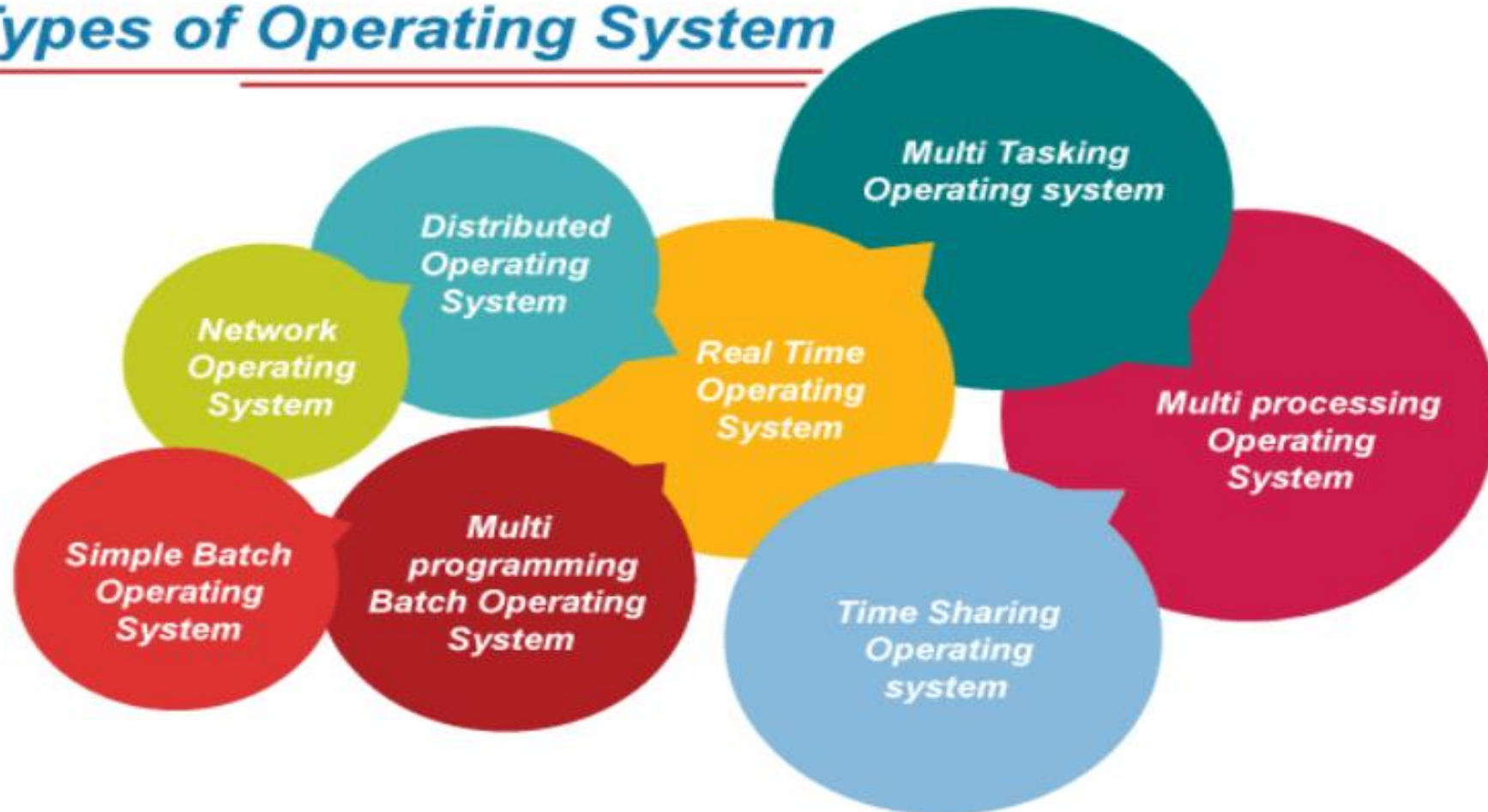


Components of OS



Types of Operating Systems

Types of Operating System



Types of Operating Systems

• Batch Operating System

- Access is given to more than one person; they submit their respective jobs to the system for execution.
- The system put all of the jobs in a queue on the basis of first come first serve and executes jobs one by one.
- Users get their respective outputs when all the jobs get executed.
- Purpose of this OS was to transfer the control from one job to another as soon as the job was completed.

Types of Operating Systems

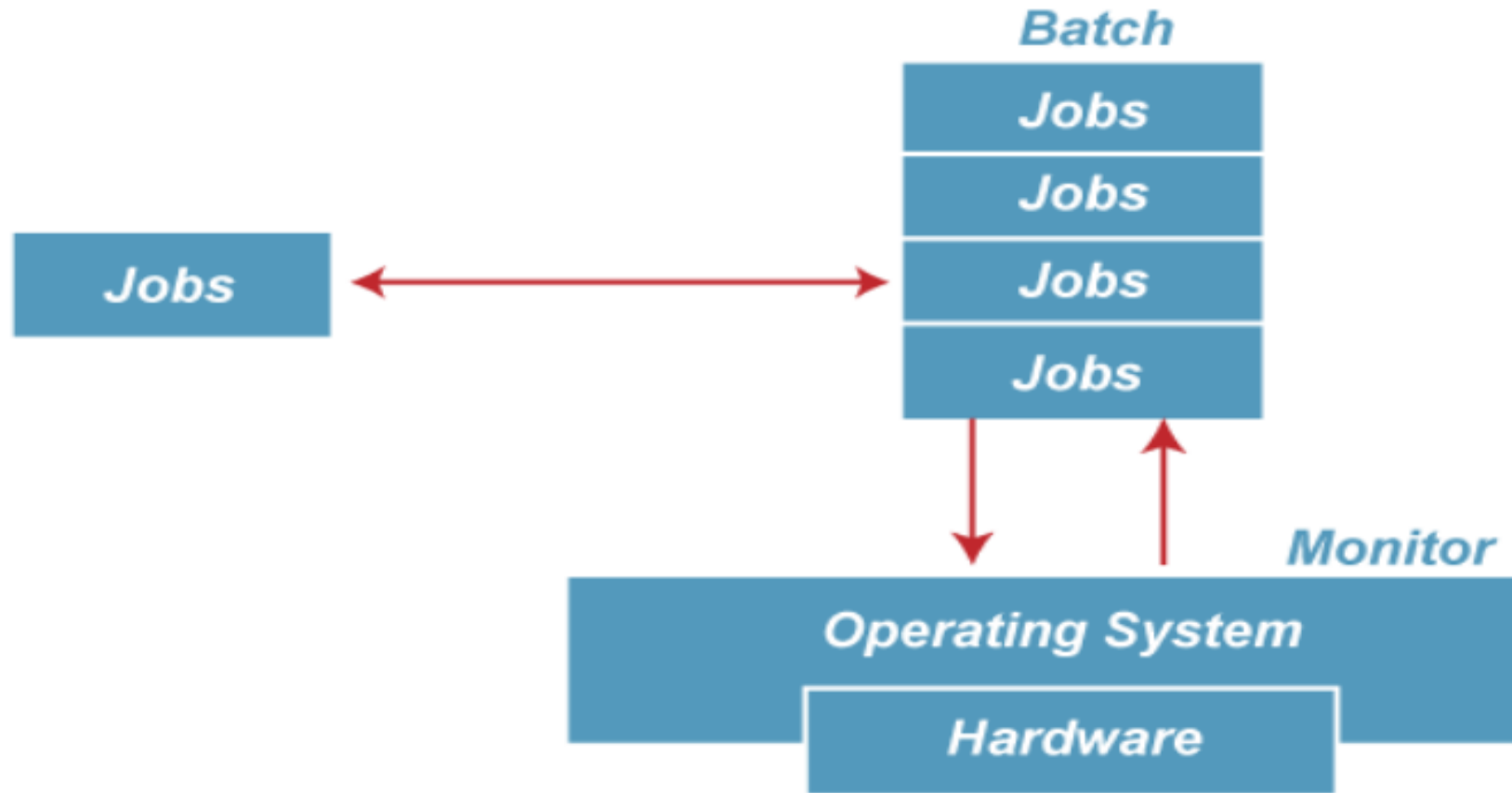
Advantages

- The use of a resident monitor improves computer efficiency as it eliminates CPU time between two jobs.

Disadvantages

- Starvation
- Not interactive

Types of Operating Systems



Types of Operating Systems

• Multiprogramming OS

- Extension to batch processing where the CPU is always kept busy.
- Each process needs two types of system time: CPU time and IO time.
- In a multiprogramming environment, when a process does its I/O, the CPU can start the execution of other processes.

Advantages

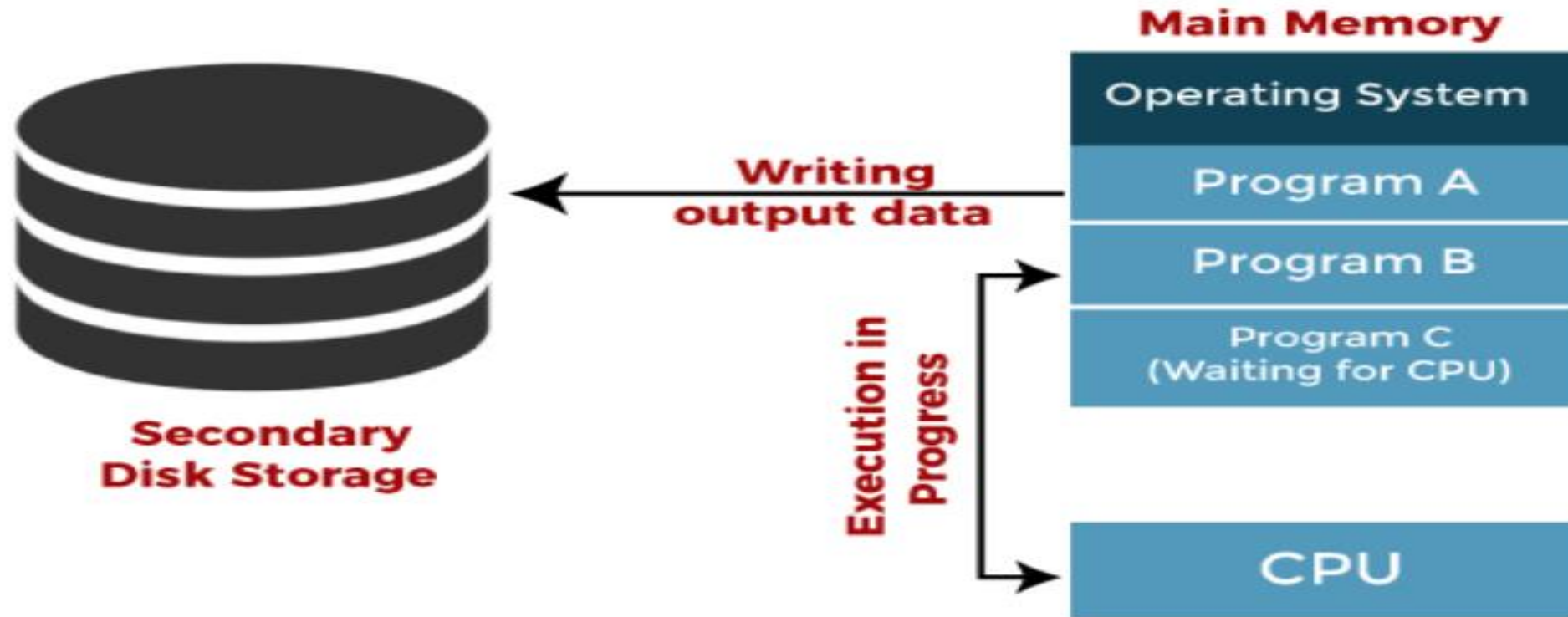
- Throughput of the system is increased as the CPU always had one program to execute.
- Response time can also be reduced.

Types of Operating Systems

Disadvantages

- Various systems resources are used efficiently, but they do not provide any user interaction with the computer system.

Types of Operating Systems



Types of Operating Systems

- **Multiprocessing OS**

- Parallel computing is achieved by multiprocessing.
- There is more than one processor present in the system which can execute more than one process at a time.
- This increases the throughput of the system.

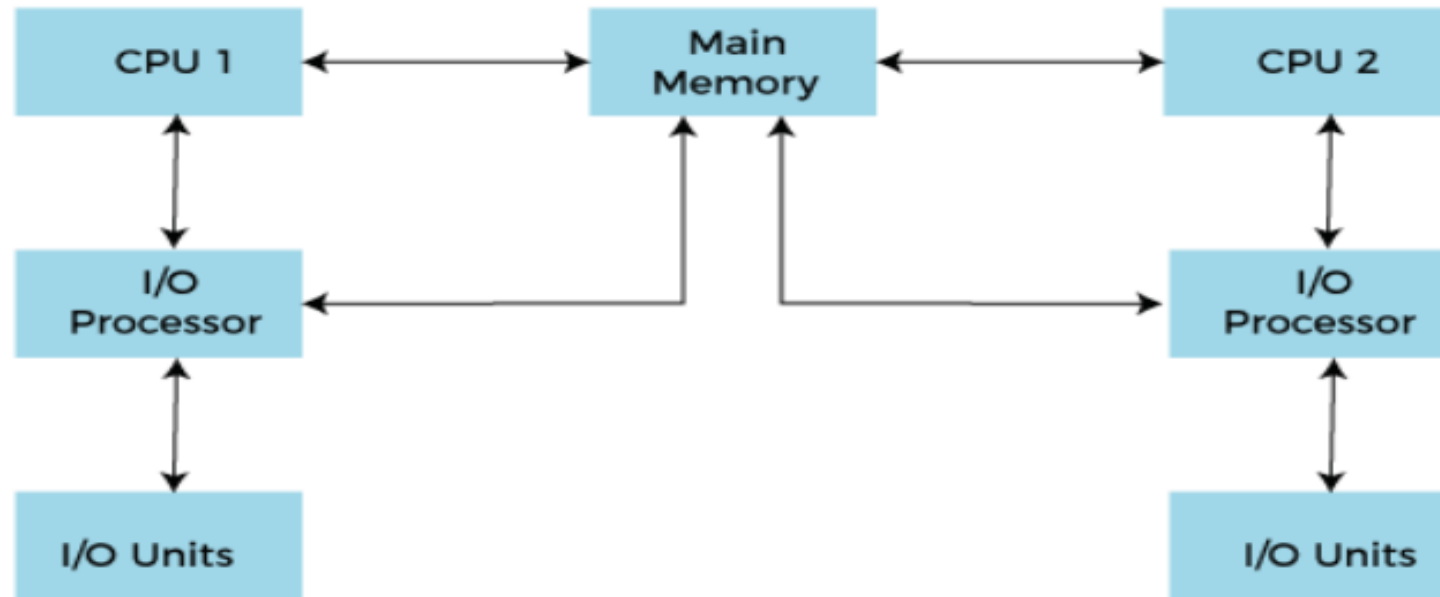
Advantages

- Increased reliability
- Increased throughput

Types of Operating Systems

Disadvantages

- Complex and sophisticated as it takes care of multiple CPUs simultaneously.



Types of Operating Systems

- **Multitasking OS**

- Logical extension of multiprogramming system that enables multiple programs simultaneously.
- It allows a user to perform more than one computer task at the same time.

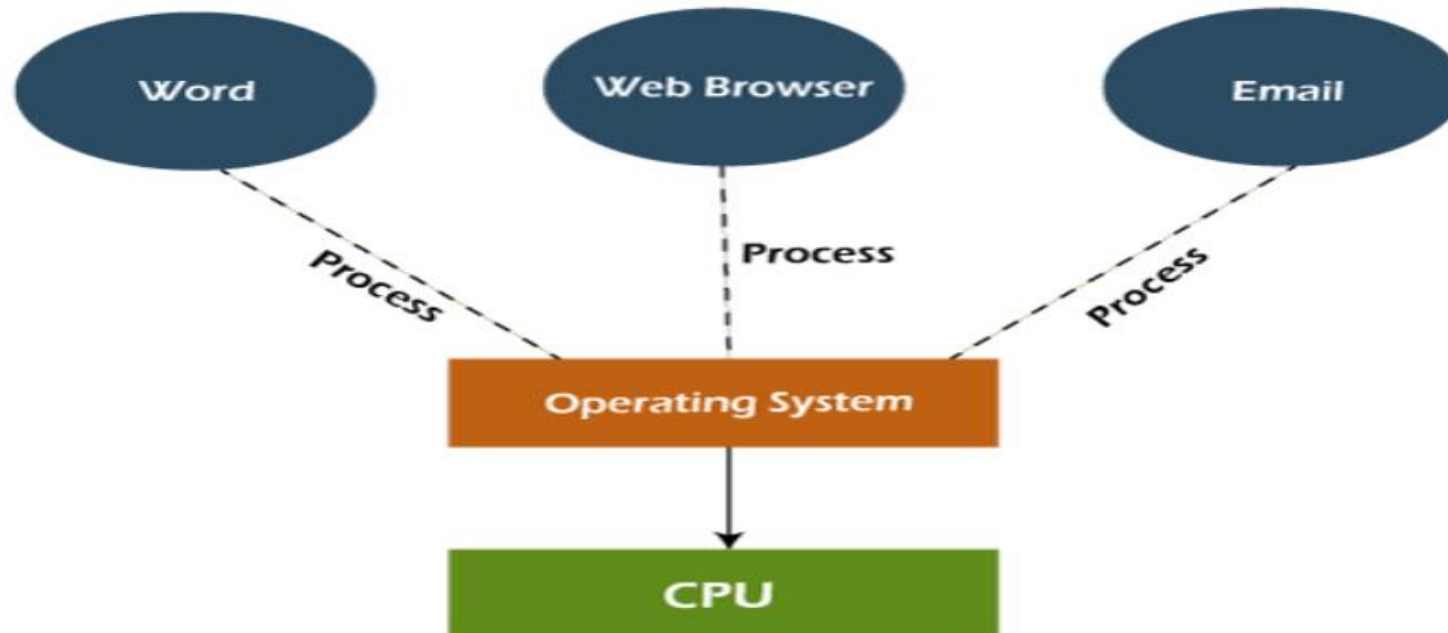
Advantages

- More suited to support multiple users simultaneously.
- Have well defined memory management.

Types of Operating Systems

Disadvantage

- Busier at the same time to complete any task in a multitasking environment, so the CPU generates more heat.



Types of Operating Systems

- **Network Operating System**

- Include software and associated protocols to communicate with other computers via the network conveniently and cost-effectively.

- Types of network OS:

- 1. Peer to peer

- 2. Client-server system

Advantages

- network traffic reduces due to the division between clients and the server.
- This type of system is less expensive to set up and maintain.

Types of Operating Systems

Disadvantages

- Failure of any node in the system affects the whole system.
- Trained network administrators are required.



Types of Operating Systems

- **Real Time Operating System**

-Each job carries a certain deadline within which the job is supposed to be completed, otherwise huge loss will be there or even if the result is produced it will be completely useless.

-The Application of a Real-Time system exists in the case of military applications, if you want to drop a missile, then the missile is supposed to be dropped with a certain precision.

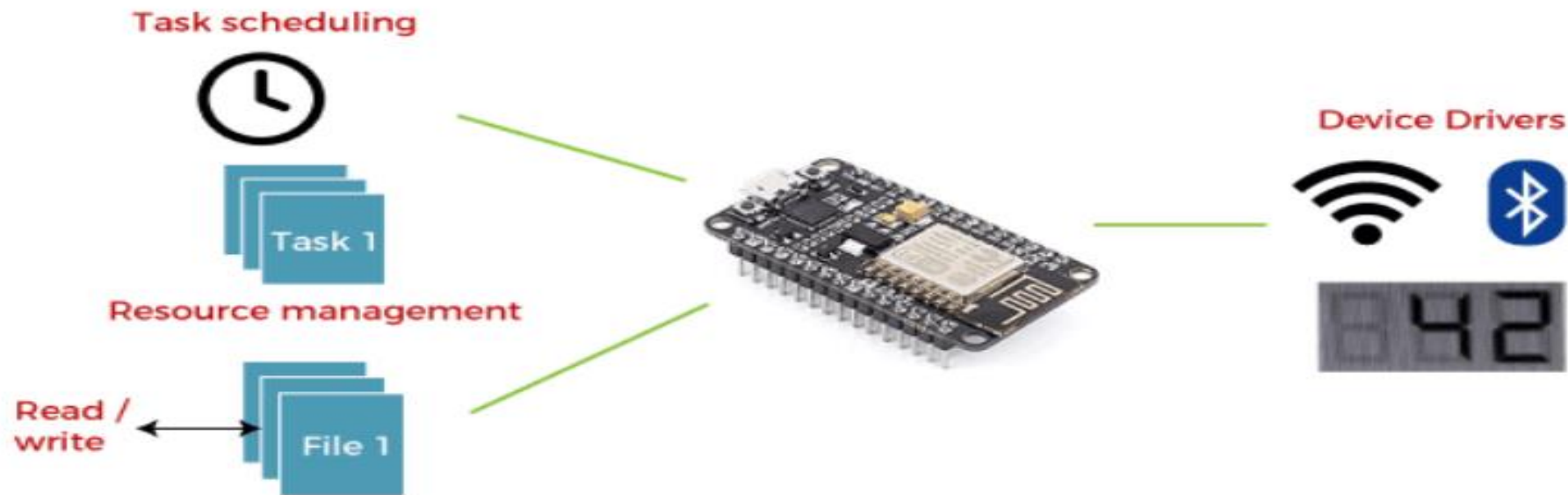
Advantages

- Easy to layout, develop, and execute real-time applications under real-time OS
- Maximum utilization of devices and systems.

Types of Operating Systems

Disadvantages

- Very costly and develop
- Very complex and consume critical CPU cycles.



Types of Operating Systems

- **Time-sharing OS**

- Computer resources are allocated in a time-dependent fashion to several programs simultaneously.
- Thus it helps to provide a large number of user's direct access to the main computer
- It is a logical extension of multiprogramming.
- In time-sharing, the CPU is switched among multiple programs given by different users on a scheduled basis.

Types of Operating Systems

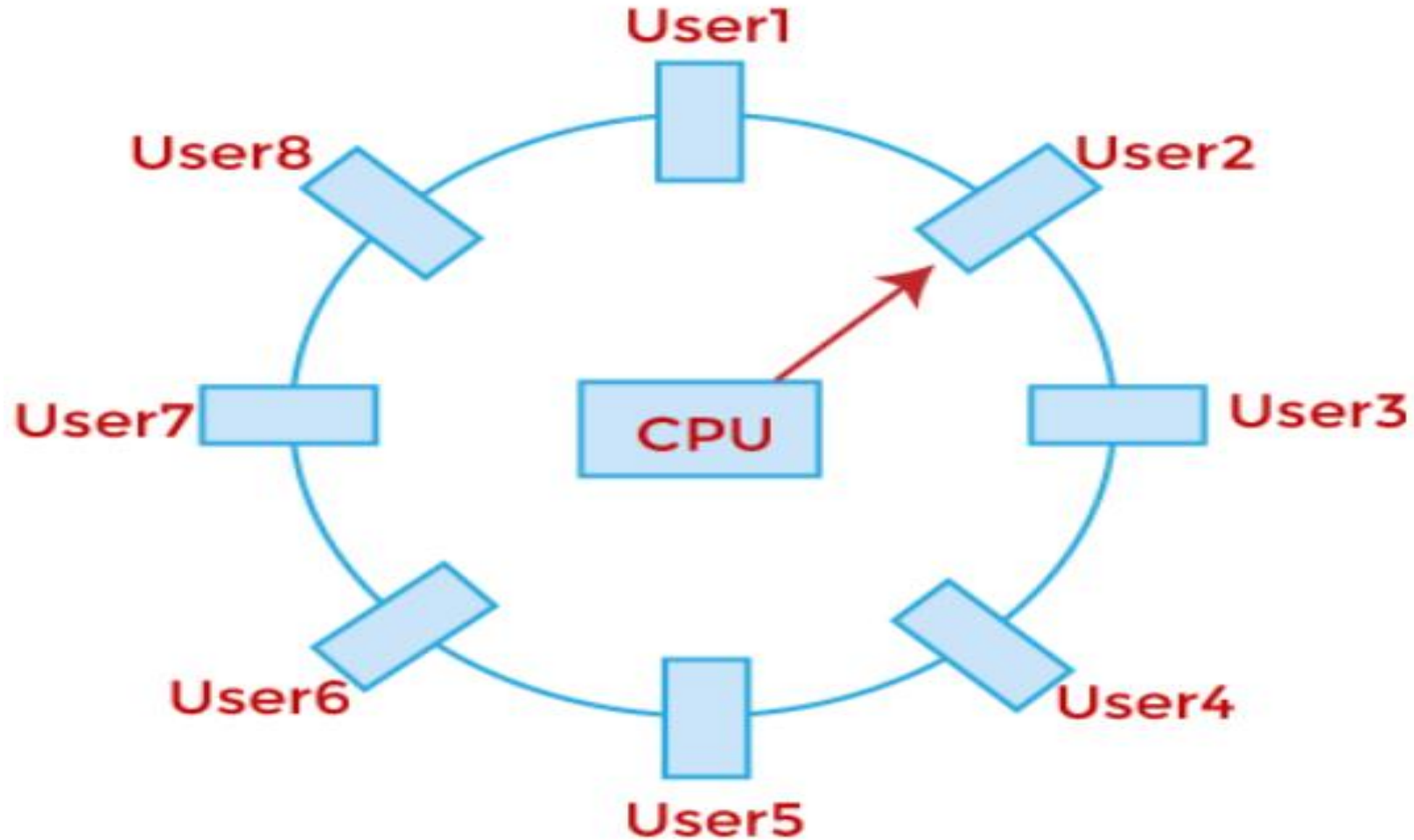
Advantages

- Provide effective utilization and sharing of resources
- System reduces CPU idle and response time

Disadvantages

- Data transmission rates are very high compared to other methods.
- Security and integrity of user programs loaded in memory and data need to be maintained as many users access the system at the same time.

Types of Operating Systems



Types of Operating Systems

• Distributed OS

- Not installed in a single machine, distributed into parts and these parts are loaded on different machines.
- A part of the distributed Operating system is installed on each machine to make their communication possible.
- More complex, large, and sophisticated than Network operating systems because they also have to take care of varying networking protocols.

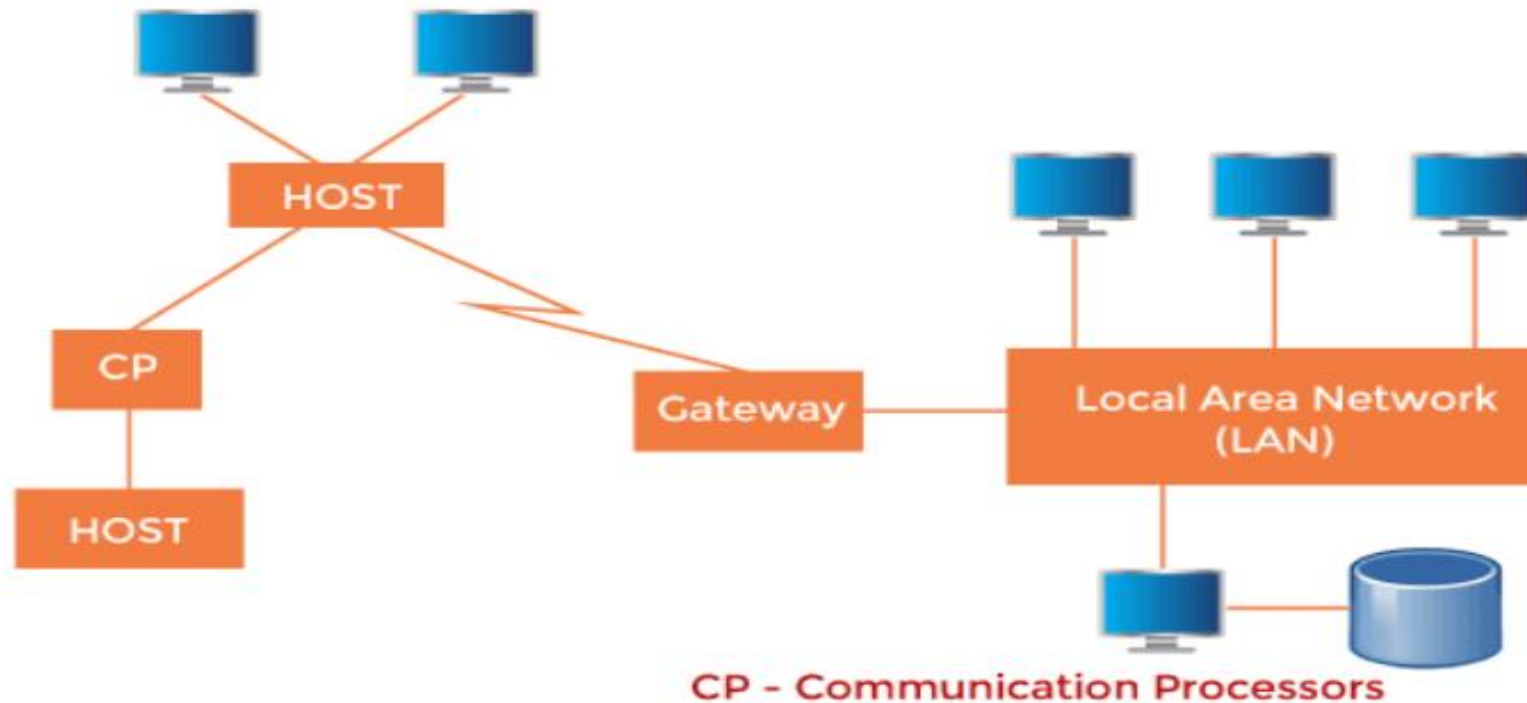
Advantages

- Provide sharing resources
- Fault tolerate

Types of Operating Systems

Disadvantages

- Protocol overhead can dominate computation cost.



Functions of Operating Systems

1. Process Management

- A process is a program in execution.
- Process needs specific resources, including CPU time, files, and I/O devices to accomplish its task.
- OS is responsible for following activities in connection with process management.
 - Create, load, execute, suspend, resume, and terminate Processes.
 - Switch system among multiple processes in main memory.
 - Provide communication mechanisms so that processes can communicate with each others.

Functions of Operating Systems

- Provide synchronization mechanisms to control concurrent access to shared data to keep shared data consistent.
- Allocate/de-allocate resources properly to prevent or avoid deadlock situations.

2. File management

- File is a collection of related information defined by its creator.
- Most files represent data, source, and object forms and programs.
- Data files can be of any type like alphabetic, numeric, and alphanumeric.
- Also files are normally organized into directories to ease their use and these directories can contain files and other directories.

Functions of Operating Systems

- OS is responsible for following activities in connection with file management.
 - File creation and deletion.
 - Directory creation and deletion
 - The support of primitives for manipulating files and directories.
 - Mapping files onto secondary storage.

Functions of Operating Systems

3. I/O Device management

- Hide the peculiarities of specific hardware devices from the user.
- I/O Device Management provides an abstract level of H/W devices and keeps the details from applications to ensure proper use of devices, prevent errors, and provide users with a convenient and efficient programming environment.
- Tasks of I/O device management components are,
 - Hide the details of Hardware devices.
 - Manage main memory for the devices using cache, buffer, and spooling.
 - Maintain and provide custom drivers for each device,

Functions of Operating Systems

4. Network management

- Network management is the process of keeping your network healthy for efficient communication between different computers.
- Computer network is a collection of various types of computers connected with each other.
- Network management comprises fault analysis, maintaining the quality of service, provisioning of networks, and performance management.
- Featured of network management,
 - Network administration
 - Network maintenance

Functions of Operating Systems

- Network operation
- Network provisioning
- Network security

5. Main memory management

- Main memory is a repository of quickly accessible data shared by the CPU and I/O devices.
- Main memory is a volatile storage device.
- The main motivation behind Memory Management is to maximize memory utilization on the computer system.

Functions of Operating Systems

- OS is responsible for following activities in connection with memory management.
 - Keep tracks of which parts of memory are currently being used and by whom.
 - Decide which processes to load when memory space becomes available.
 - Allocate and deallocate memory space as needed.

6. Secondary Storage Management

- Programs and the data they access must be in the main memory during execution to execute programs.
- Since the main memory is too small to permanently accommodate all data and programs, the computer system should provide secondary storage to back up the main memory.

Functions of Operating Systems

- Most modern computer systems use disks as the principal online storage medium for both programs and data.
- Most programs, like compilers, assemblers, sort routines, editors, formatters, and so on, are stored on the disk until loaded into memory and then use the disk as both the source and destination of their processing.
- OS is responsible for following activities,
 - Free Space management
 - Storage allocation

Functions of Operating Systems

7. Security Management

- Security Management refers to a mechanism for controlling the access of programs, processes, or users to the resources defined by computer controls to be imposed, together with some means of enforcement.
- For example memory addressing hardware ensures that the process can only execute within its own address space.
- Timer ensures that no process can gain control of the CPU without turning over it.

Functions of Operating Systems

8. Command interpreter system

- Command interpreter is the primary system between the user and the rest of the system.
- Command interpreter system executes a user command by calling one or more underlying system programs or system calls.
- Many commands are given to the operating system by control statements. A program that reads and interprets control statements is automatically executed.
- This program is called the shell and a few examples are Windows DOS command window, Bash of Unix/Linux, or C-Shell of Unix/Linux.

Goals of OS

- Convenience-An OS makes a computer more convenient for the user for using.
- Efficiency-An OS allows the computer system resources to be used in an efficient manner, to ensure good resource utilization efficiency and provide appropriate corrective actions when it becomes low
- Ability to evolve-An OS should construct in such a way as to permit the effective development, testing, and introduction of new system functions without interfering with service.

Functions of Operating Systems

Thank You!

Process Management

ICT1308

Process

- The program in execution is called a process. In order to accomplish its task process needs the computer resources.
- Differences between process and Program

Process	Program
Process is a dynamic object	Program is a static object
Process is a sequence of instruction execution	Program is a sequence of instructions
Process loaded into main memory	Program loaded into secondary storage devices
Time span of process is limited	Time span of program is unlimited

Attributed of a process

- Process ID - When a process is created a unique ID is assigned to the process for unique identification of the process in the system.
- Program counter-stores address of the last instruction of the process on which the process was suspended.
- Process state- process from its creation to its completion goes through various states: new, ready, running, and waiting.
- Priority- The process with the highest priority among the processes gets the CPU first. This is also stored on the process control block.
- General purpose registers- Every process has its own set of registers which are used to hold the data which is generated during the execution of the process.

Attributed of a process

- List of open files- During the execution every process uses some files which need to be present in the main memory. OS also maintains a list of open files in the Process Control Block(PCB).
- List of Open devices- Os also maintains a list of open devices used during the execution process.

Process States

- The process from its creation to completion passes through various states. Minimum number of states is five.

1. New

A program which is going to be picked up by the OS into the main memory is called a new process

2. Ready

Whenever a process is created it directly enters to ready state, in which it waits for the CPU to be assigned. There can be many processes present in the ready state.

Process States

3. Running

One of the processes will be chosen from the ready state by OS depending on the scheduling algorithm.

If we have only one CPU in our system, the number of running processes for a particular time will always be one. If we have n processors in the system then we can have n processes running simultaneously.

4. Block or wait

From running state process can make transitions to block or wait state.

When a process waits for a certain resource to be assigned or for input from the user then the OS moves this process to the block or wait state and assigns the CPU to the other processes.

Process States

5. Completion or termination

When the process finishes its execution it comes in the termination state. All the context of the process will be deleted when the process is terminated by the OS.

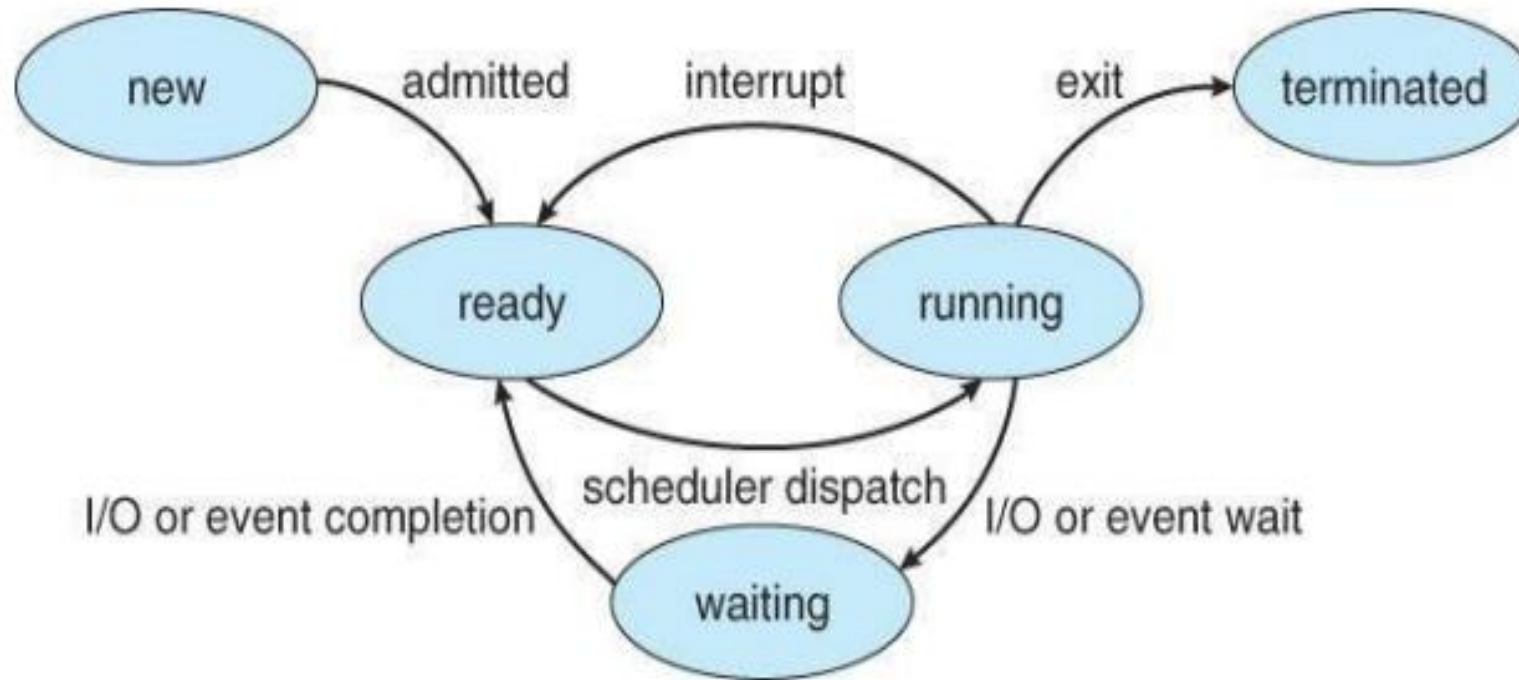
6. Suspend ready

A process in the ready state, which is moved to secondary memory from the main memory due to lack of resources (mainly primary memory) is called in the suspend ready state.

7. Suspend wait

Instead of removing the process from the ready queue, it's better to remove the blocked process which is waiting for some resources in the main memory.

Process States



Process States

- New->Ready: OS creates a process and prepares the process to be executed, then OS moves the process into the ready queue.
- Ready->Running :: OS selects one of the Jobs from ready Queue and move them from ready to Running.
- Running->Terminate: When the Execution of a process has been Completed, OS terminates that process from running state
- Running->Ready : When the time slot of the processor expired (or) If the processor received any interrupt signal, the OS shifted Running -> ReadyState

Process States

- Running -> Waiting: A process is put into the waiting state if the process needs an event occurs (or) an I/O Device require,
- Waiting->Ready: A process in the waiting state is moved to the ready state when the event for which it has been Completed.

Threads

- A process is divided into number of lightweight process, each lightweight process is said to be a Thread.
- The Thread has a program counter (Keeps track of which instruction to execute next), registers (holds its current working variables), and stack (execution History).
- Thread States are born State, ready State, running, Sleep , and dead.

Ex-Word processor

Typing, Formatting, spell checking, savings are threads

Difference between process and Thread

Process	Thread
Process takes more time to create.	Thread takes less time to create.
it takes more time to complete execution & terminate.	Less time to terminate.
Execution is very slow.	Execution is very fast.
It takes more time to switch b/w two processes.	It takes less time to switch b/w two threads.
Communication b/w two processes is difficult .	Communication b/w two threads is easy.
Process can't share the same memory area.	Threads can share same memory area.
System calls are requested to communicate each other.	System calls are not required.
Process is loosely coupled.	Threads are tightly coupled.
It requires more resources to execute.	Requires few resources to execute.

Operations on the process

- Creation
- Scheduling
- Execution
- Deletion/Killing

Process scheduling in OS

- Long term scheduler
- Short term scheduler
- Medium term scheduler

Long term scheduler

- Also known as job scheduler.
- It chooses the processes from the secondary memory and keeps them in the ready queue maintained in the primary memory.
- The purpose of the long-term scheduler is to choose a perfect mix of IO-bound and CPU-bound processes among the jobs present in the pool.
- If the job scheduler chooses more IO bound processes then all of the jobs may reside in the blocked state all the time and the CPU will remain idle most of the time.
- This will reduce the degree of Multiprogramming.
- Therefore, the Job of long term scheduler is very critical and may affect the system for a very long time.

Short term scheduler

- Also known as CPU scheduler.
- It selects one of the Jobs from the ready queue and dispatch to the CPU for the execution.
- A scheduling algorithm is used to select which job is going to be dispatched for the execution.
- The Job of the short term scheduler can be very critical in the sense that if it selects job whose CPU burst time is very high then all the jobs after that, will have to wait in the ready queue for a very long time.
- This problem is called starvation which may arise if the short term scheduler makes some mistakes while selecting the job.

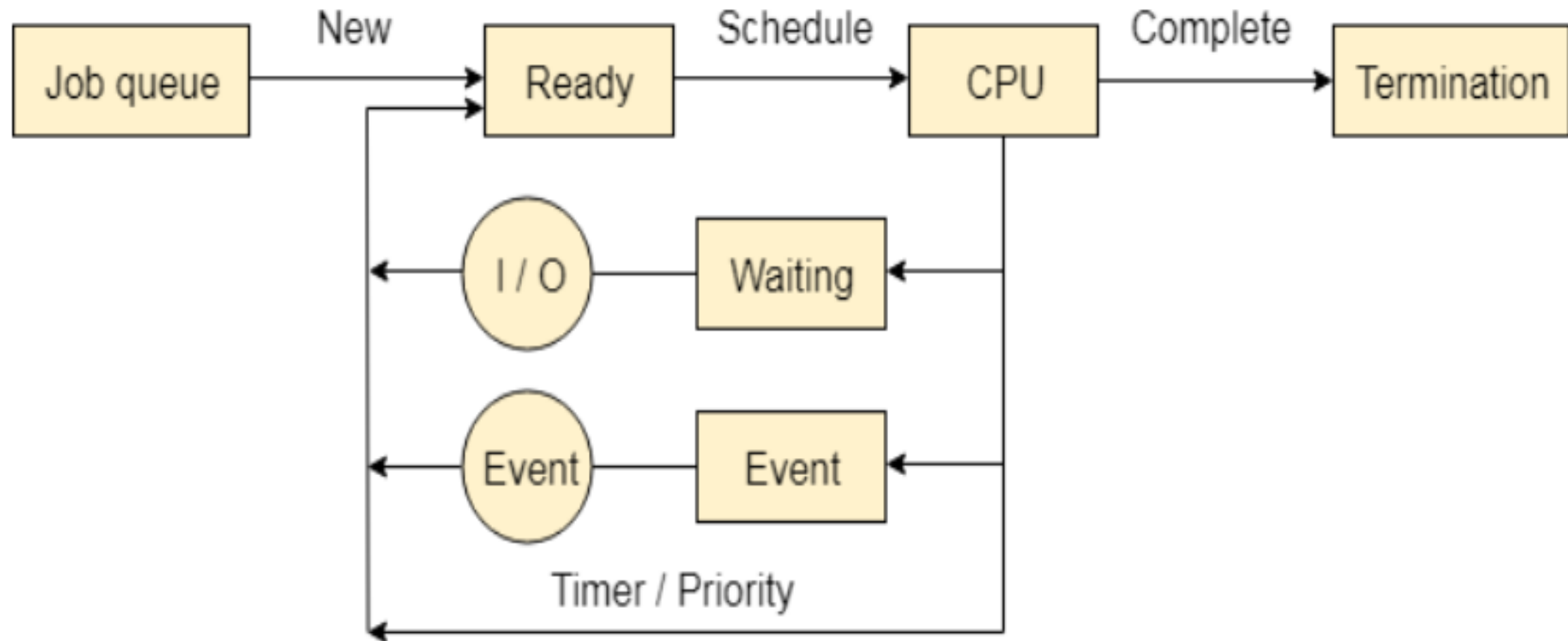
Medium-term scheduler

- Take care of swapped-out processes.
- If running stat process needs IO time then the state should change from running to waiting.
- It removes the process from the running state to make room for the other processes.
- Such processes are the swapped-out processes and this procedure is called swapping.
- The medium-term scheduler is responsible for suspending and resuming the processes.
- It reduces the degree of multiprogramming. The swapping is necessary to have a perfect mix of processes in the ready queue.

Process Queues

- OS manages various types of queues for each of the process state.
- The PCB related to the process is also stored in the queue of the same state.
- If the Process is moved from one state to another state then its PCB is also unlinked from the corresponding queue and added to the other state queue in which the transition is made.

Process Queues



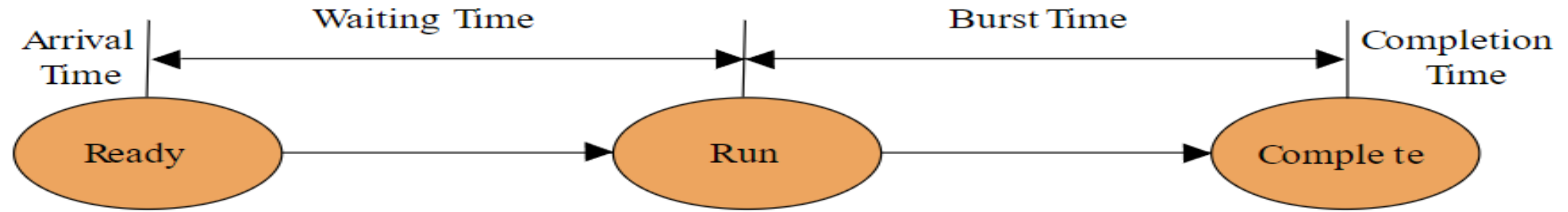
Process Queues

1. Job Queue-In starting, all the processes get stored in the job queue. It is maintained in the secondary memory. The long term scheduler (Job scheduler) picks some of the jobs and put them in the primary memory.
2. Ready Queue-Ready queue is maintained in primary memory. The short term scheduler picks the job from the ready queue and dispatch to the CPU for the execution.
3. Waiting Queue-When the process needs some IO operation in order to complete its execution, OS changes the state of the process from running to waiting. The context (PCB) associated with the process gets stored on the waiting queue which will be used by the Processor when the process finishes the IO.

Time related to process

- **Arrival time**- Time at which process enters into ready queue is called arrival time.
- **Burst time**-The total amount of time required by the CPU to execute the whole process is called the Burst Time. This does not include the waiting time
- **Completion time**-The Time at which the process completes its execution, is called completion time.
- **Turnaround time**-The total amount of time spent by the process from its arrival to its completion.
- **Waiting time**-The Total amount of time for which the process waits for the CPU to be assigned is called waiting time.
- **Response time**-The difference between the arrival time and the time at which the process first gets the CPU is called Response Time.

Time related to process



$$CT - AT = WT + BT$$

$$TAT = CT - AT$$

$$\text{Waiting Time} = TAT - BT$$

TAT → Turn around time

BT → Burst time

AT → Arrival time

CPU scheduling

- **In Multiprogramming systems**, the Operating system schedules the processes on the CPU to have the maximum utilization of it and this procedure is called **CPU scheduling**.
- The Operating System uses various scheduling algorithm to schedule the processes.
- This is a task of the short term scheduler to schedule the CPU for the number of processes present in the Job Pool.
- Whenever the running process requests some IO operation then the short term scheduler saves the current context of the process (also called PCB) and changes its state from running to waiting.
- .

CPU scheduling

- During the time, process is in waiting state; the Short term scheduler picks another process from the ready queue and assigns the CPU to this process. This procedure is called **context switching**.
- The Operating system maintains a process control block during the lifetime of the process.
- The Process control block is deleted when the process is terminated or killed.

Scheduling algorithms in OS

- Purpose of scheduling algorithms:
 - Maximum CPU utilization
 - Fair allocation of CPU
 - Maximum throughput
 - Minimum turnaround time
 - Minimum waiting time
 - Minimum response time

Types of Scheduling algorithms in OS

- First Come First Serve-The process with the minimal arrival time will get the CPU first. The lesser the arrival time, the sooner the process gets the CPU. It is the non-preemptive type of scheduling.
- Round Robin –OS define time quantum(slice).All processes will get executed incyclic way. Each of the process will get the CPU for a small amount of time (called time quantum) and then get back to the ready queue to wait for its next turn. It is a preemptive type of scheduling.
- Shortest Job First-The job with the shortest burst time will get the CPU first.
- Shortest remaining time first-he OS schedules the Job according to the remaining time of the execution.

Types of Scheduling algorithms in OS

- Priority based scheduling-The priority will be assigned to each of the processes. The higher the priority, the sooner will the process get the CPU. If the priority of the two processes is same then they will be scheduled according to their arrival time.
- Highest Response Ratio Next-The process with highest response ratio will be scheduled next. This reduces the starvation in the system.

Preemptive vs Non preemptive approach

- **Preemptive approach**-OS allocates the resources to a Process for a predetermined period of time. The process transitions from running state to ready state or from waiting state to ready state during resource allocation. This switching happens because the CPU may assign other processes precedence and substitute the currently active process for the higher priority process.
- **Non-preemptive approach**-The resource cannot be withdrawn from a process before the process has finished running. If a process or job is started execution, then the operating system must complete its process or job. Until, the process or job is over the new or next process or job does not start its execution.

First Come First Serve

- Whichever process enters the ready queue first is executed first. This shows that First Come First Serve Algorithm follows First In First Out (FIFO) principle.
- Advantages
 1. In order to allocate processes, it uses the First In First Out queue.
 2. The FCFS CPU Scheduling Process is straight forward and easy to implement.
 3. In the FCFS situation preemptive scheduling, there is no chance of process starving.
 4. As there is no consideration of process priority, it is an equitable algorithm.

First Come First Serve

- Disadvantages

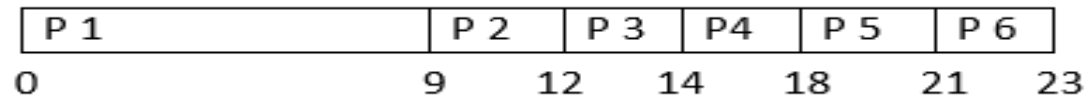
1. FCFS CPU Scheduling Algorithm has Long Waiting Time
2. FCFS CPU Scheduling favors CPU over Input or Output operations
3. In FCFS there is a chance of occurrence of Convoy Effect
4. Because FCFS is so straight forward, it often isn't very effective. Extended waiting periods go hand in hand with this. All other orders are left idle if the CPU is busy processing one time-consuming order.

First Come First Serve

S No	Process ID	Process Name	Arrival Time	Burst Time
1	P1	A	0	9
2	P2	B	1	3
3	P3	C	1	2
4	P4	D	1	4
5	P5	E	2	3
6	P6	F	3	2

First Come First Serve-Non preemptive approach

- Gantt chart for above example



Turn Around Time=Completion Time-Arrival time

Waiting time=Turn around time- Burst Time

First Come First Serve

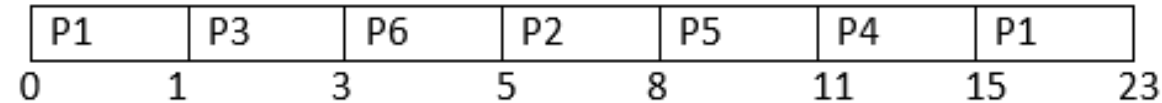
S No	Process ID	Process Name	Arrival Time	Burst Time	Completion Time	Turn around Time	Waiting Time
1	P1	A	0	9	9	9	0
2	P2	B	1	3	12	11	8
3	P3	C	1	2	14	13	11
4	P4	D	1	4	18	17	13
5	P5	E	2	3	21	19	16
6	P6	F	3	2	23	20	18

First Come First Serve

- Average CT= $(9+12+14+18+21+23)/6$
 $=16.6667$
- Average WT= $(0+8+11+13+16+18)/6$
 $=66/6$
 $=11$
- Average TAT= $(9+11+13+17+19+20)/6$
 $=89/6$
 $=14.83$

First come first serve-Preemptive approach

- Gantt chart



S No	Process ID	Process Name	Arrival Time	Burst Time	Completion Time	Turn around Time	Waiting Time
1	P1	A	0	9	23	23	14
2	P2	B	1	3	8	7	4
3	P3	C	1	2	3	2	0
4	P4	D	1	4	15	14	10
5	P5	E	2	3	11	9	7
6	P6	F	3	2	5	2	0

Shortest Job First Scheduling

- SJF scheduling algorithm, schedules the processes according to their burst time.
- In SJF scheduling, the process with the lowest burst time, among the list of available processes in the ready queue, is going to be scheduled next.

Advantages

- Maximum throughput
- Minimum average waiting

Shortest Job First Scheduling

Disadvantages

1. May suffer with the problem of starvation
2. It is not implementable because the exact Burst time for a process can't be known in advance.

Question

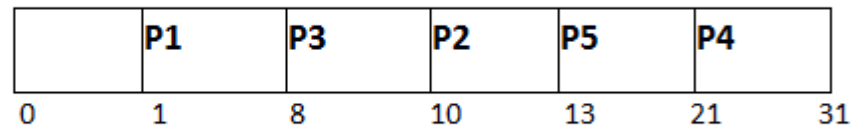
PID	Arrival Time	Burst Time
1	1	7
2	3	3
3	6	2
4	7	10
5	9	8

Shortest Job First Scheduling

PID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
1	1	7	8	7	0
2	3	3	13	10	7
3	6	2	10	4	2
4	7	10	31	24	14
5	9	8	21	12	4

Shortest Job First Scheduling

- Since, No Process arrives at time 0 hence; there will be an empty slot in the **Gantt chart** from time 0 to 1 (the time at which the first process arrives).
- Till now, we have only one process in the ready queue hence the scheduler will schedule this to the processor no matter what is its burst time.



$$\text{Avg waiting time} = (7 + 2 + 14 + 4) / 5 = 27 / 5$$

Shortest Remaining Time First(SRTF) scheduling

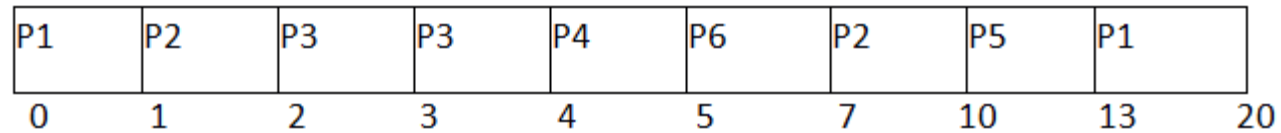
- This Algorithm is the **preemptive version** of **SJF scheduling**.
- In SRTF, the execution of the process can be stopped after certain amount of time. At the arrival of every process, the short term scheduler schedules the process with the least remaining burst time among the list of available processes and the running process.
- Once all the processes are available in the **ready queue**, No preemption will be done and the algorithm will work as **SJF scheduling**.
- The context of the process is saved in the **Process Control Block** when the process is removed from the execution and the next process is scheduled.
- This PCB is accessed on the **next execution** of this process.

Shortest Remaining Time First(SRTF) scheduling

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time	Response Time
1	0	8	20	20	12	0
2	1	4	10	9	5	1
3	2	2	4	2	0	2
4	3	1	5	2	1	4
5	4	3	13	9	6	10
6	5	2	7	2	0	5

Shortest Remaining Time First(SRTF) scheduling

- Avg waiting time=24/6



- Once all the processes arrive, No preemption is done and the algorithm will work as SJF.

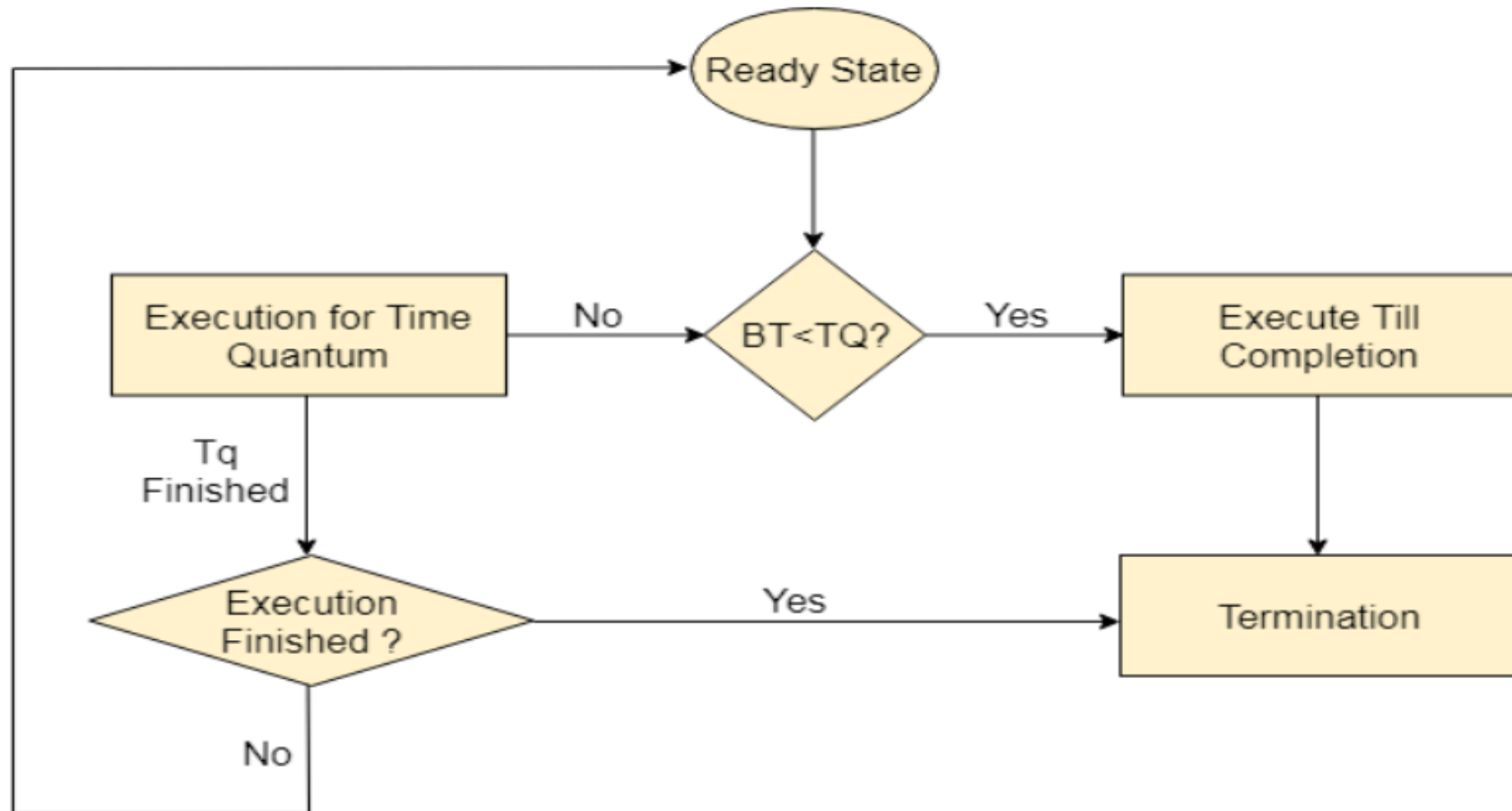
Round Robin Scheduling

- Round Robin CPU Scheduling uses Time Quantum (TQ).
- The Time Quantum is something which is removed from the Burst Time and lets the chunk of process to be completed.
- Time Sharing is the main emphasis of the algorithm. Each step of this algorithm is carried out cyclically. The system defines a specific time slice, known as a time quantum.
- First, the processes which are eligible to enter the ready queue enter the ready queue. After entering the first process in Ready Queue is executed for a Time Quantum chunk of time.
-

Round Robin Scheduling

- After execution is complete, the process is removed from the ready queue. Even now the process requires some time to complete its execution, then the process is added to Ready Queue.
- The Ready Queue does not hold processes which already present in the Ready Queue. The Ready Queue is designed in such a manner that it does not hold non unique processes.
- By holding same processes Redundancy of the processes increases.

Round Robin Scheduling



Round Robin Scheduling

S NO	Process ID	Arrival time	Burst Time
1	P1	0	7
2	P2	1	4
3	P3	2	15
4	P4	3	11
5	P5	4	20
6	P6	4	9

Assume time quantum $TQ=5$
Ready Queue

P1, P2, P3, P4, P5, P6, P1, P3, P4, P5, P6, P3, P4, P5

Round Robin Scheduling

P1	P2	P3	P4	P5	P6	P1	P3	P4	P5	P6	P3	P4	P5
0	5	9	14	19	24	29	31	36	41	46	50	55	66

Process ID	Arrival Time	Burst Time	Completion Time	Turn Around Time	Waiting Time
P1	0	7	31	31	24
P2	1	4	9	8	4
P3	2	15	55	53	38
P4	3	11	56	53	42
P5	4	20	66	62	42
P6	4	9	50	46	37

Round Robin Scheduling

- Average Completion Time = (31 + 9 + 55 + 56 + 66 + 50) / 6
- Average Waiting Time = (5 + 26 + 5 + 42 + 42 + 37) / 6
- Average Turn Around Time = (31 + 8 + 53 + 53 + 62 + 46) / 6

Thank You!

Synchronization

ICT1308

Process Synchronization in OS

- When two or more process cooperates with each other, their order of execution must be preserved otherwise there can be conflicts in their execution and inappropriate outputs can be produced.
- A cooperative process is the one which can affect the execution of other process or can be affected by the execution of other process.
- Such processes need to be synchronized so that their order of execution can be guaranteed.
- “Preserving the appropriate order of execution of cooperative processes is known as Process Synchronization.”

Race Condition and Critical Section

- A **Race Condition** typically occurs when two or more threads try to read, write, and possibly make the decisions based on the memory they are accessing concurrently.(occur when two computer program processes, or threads, attempt to access the same resource at the same time and cause problems in the system)
- The region of a program that tries to access shared resources and may cause race condition is called a **critical section**.
- To avoid race conditions among the processes, we must ensure that only one process can be executed within the critical section at a time.

Critical Section Problem in OS

- Critical Section is the part of a program which tries to access shared resources. That resource may be any resource in a computer like a memory location, Data structure, CPU or any IO device.
- The critical section cannot be executed by more than one process at the same time; operating system faces the difficulties in allowing and disallowing the processes from entering the critical section.
- The critical section problem is used to design a set of protocols which can ensure that the Race condition among the processes will never arise.

Requirements of synchronization mechanisms

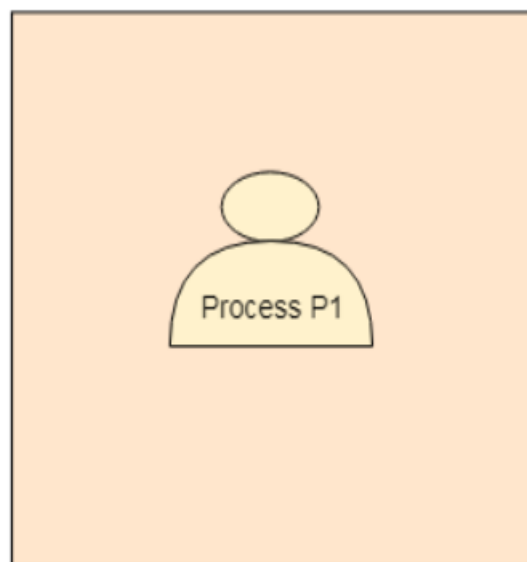
- **Primary**

1. Mutual Exclusion

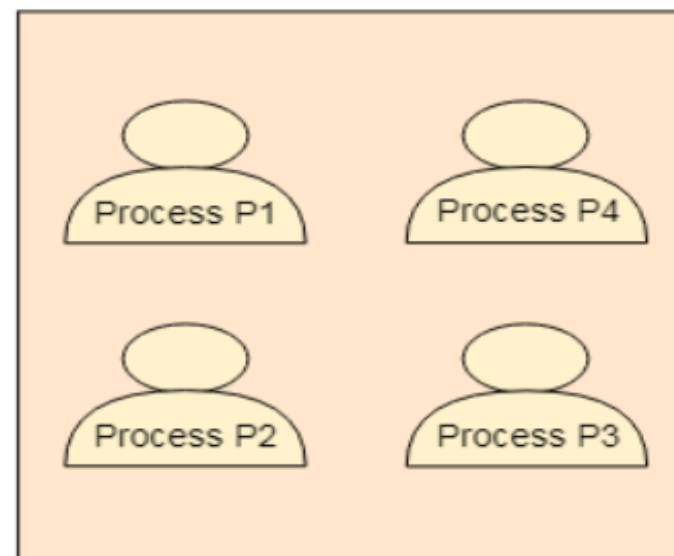
Our solution must provide mutual exclusion. If one process is executing inside a critical section then the other process must not enter the critical section. This is called Mutual Exclusion.



Critical Section



Critical Section



Requirements of synchronization mechanisms

2. Progress

Progress means that if one process doesn't need to execute into a critical section then it should not stop other processes from getting into the critical section.

Secondary

1. Bounded waiting

We should be able to predict the waiting time for every process to get into the critical section. The process must not be endlessly waiting for getting into the critical section.

Requirements of synchronization mechanisms

2. Architectural Neutrality

Our mechanism must be architectural natural. It means that if our solution is working fine on one architecture then it should also run on the other ones as well.

Lock Variable

- Simplest synchronization mechanism. This is a busy waiting solution that can be used for more than two processes.
- In this mechanism, a Lock variable **lock** is used. Two values of lock can be possible, either 0 or 1.
- Lock value 0 means that the critical section is vacant while the lock value 1 means that it is occupied.
- A process which wants to get into the critical section first checks the value of the lock variable.
- If it is 0 then it sets the value of lock as 1 and enters into the critical section, otherwise it waits.

Lock Variable

Entry Section →

While (lock! = 0);

Lock = 1;

//Critical Section

Exit Section →

Lock = 0;

- Pseudocode consist of 3 sections as entry section, critical section and exit section.

Lock Variable

- The problem with the lock variable mechanism is that, at the same time, more than one process can see the vacant tag and more than one process can enter in the critical section. Hence, the lock variable doesn't provide mutual exclusion that's why it cannot be used in general.
- Since the lock variable method failed at the basic step; hence, there is no need to talk about the other conditions to be fulfilled.

Test Set Lock Mechanisms

- In lock variable mechanism, Sometimes Process reads the old value of lock variable and enters the critical section.
- Due to this reason, more than one process might get into critical section.
- In TSL mechanism they try to replace the code in section 1 with the code in section 2 and they manage to provide mutual exclusion to a certain extent but not completely.
- In the updated version of code, the value of Lock is loaded into the local register R0 and then value of lock is set to 1.

Test Set Lock mechanism

- However, in step 3, the previous value of lock (that is now stored into R0) is compared with 0. if this is 0 then the process will simply enter into the critical section otherwise will wait by executing continuously in the loop.

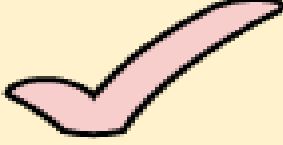
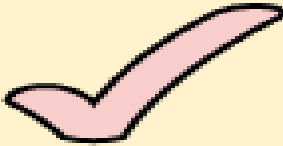
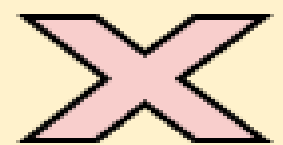
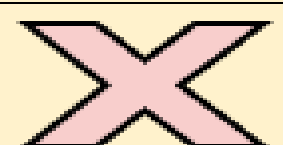
Section 1

```
1. Load Lock, R0  
2. CMP R0, #0  
3. JNZ step1  
4. store #1, Lock
```

Section 2

```
1. Load Lock, R0  
2. Store #1, Lock  
3. CMP R0, #0  
4. JNZ step 1
```

Test Set Lock Mechanism

Mutual Exclusion	
Progress	
Bounded Waiting	
Portability	

Turn Variable or Strict Alteration Approach

- Turn Variable or Strict Alternation Approach is the software mechanism implemented at user mode.
- It is a busy waiting solution which can be implemented only for two processes.
- In general, let the two processes be P_i and P_j . They share a variable called turn variable.

For process P_i

Non - CS

while (turn $\neq$ i);

Critical Section

turn = j;

Non - CS

Turn Variable or Strict Alteration Approach

For process P_j

Non - CS

while (turn $\neq j$);

Critical Section

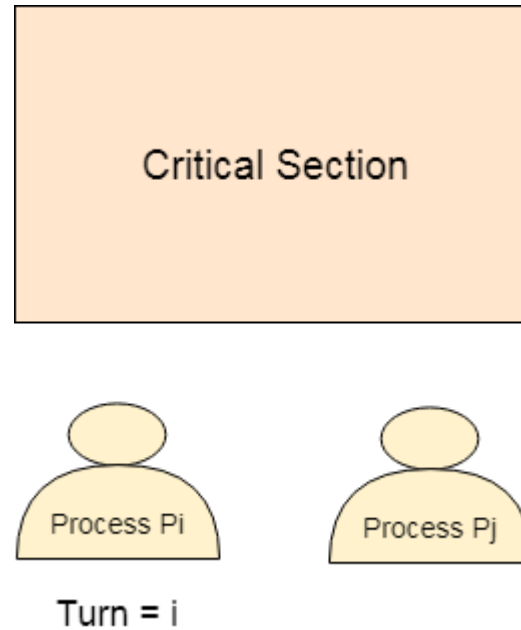
turn = i ;

Non - CS

- In turn variable approach process can enter in the critical section only in the case when the value of the turn variable is equal to the PID of the process.
- There are only two values possible for turn variable, i or j. if its value is not i then it will definitely be j or vice versa.
- In the entry section, in general, the process P_i will not enter in the critical section until its value is j or the process P_j will not enter in the critical section until its value is i.

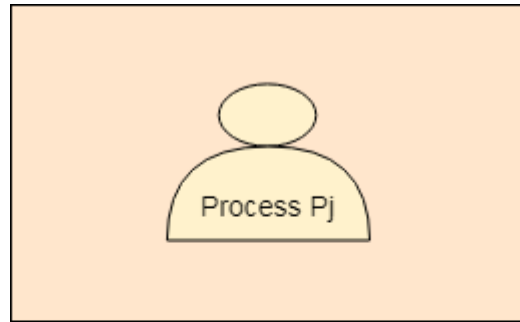
Turn Variable or Strict Alteration Approach

- Initially, two processes P_i and P_j are available and want to execute into critical section

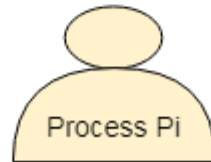


Turn Variable or Strict Alteration Approach

- P_i finishes its critical section and assigns j to turn variable. P_j will get the chance to enter into the critical section. The value of turn remains j until P_j finishes its critical section



Turn = j

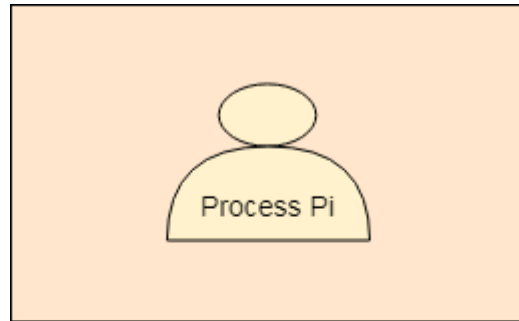


Turn Variable or Strict Alteration Approach

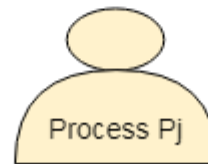
Mutual Exclusion	✓
Progress	✗
Bounded Waiting	✓
Portability	✓

Turn Variable or Strict Alteration Approach

- The turn variable is equal to i hence P_i will get the chance to enter into the critical section. The value of P_i remains 1 until P_i finishes critical section.



Turn = i



Paterson solution

- This is a software mechanism implemented at user mode. It is a busy waiting solution can be implemented for only two processes. It uses two variables that are turn variable and interested variable.
- Peterson solution provides you all the necessary requirements such as Mutual Exclusion, Progress, Bounded Waiting and Portability.

Paterson solution

```
voidEntry_Section (int process)
{
    int other; #1
    other = 1-process; #2
    interested[process] = TRUE; #3
    turn = process; #4
    while (interested [other] =True && TURN=process); #5
}
```

Critical Section

```
voidExit_Section (int process)
{
    interested [process] = FALSE; #6
}
```

Paterson solution

- This is a two process solution.
- Let us consider two cooperative processes P1 and P2. The entry section and exit section are shown above. Initially, the value of interested variables and turn variable is 0.
- Initially process P1 arrives and wants to enter into the critical section. It sets its interested variable to True (instruction line 3) and also sets turn to 1 (line number 4).
- Since the condition given in line number 5 is completely satisfied by P1 therefore it will enter in the critical section.

P1 → 1 2 3 4 5 CS

Paterson solution

- Meanwhile, Process P1 got preempted and process P2 got scheduled. P2 also wants to enter in the critical section and executes instructions 1, 2, 3 and 4 of entry section.
- On instruction 5, it got stuck since it doesn't satisfy the condition (value of other interested variable is still true). Therefore it gets into the **busy waiting**.

P2 → 1 2 3 4 5

- P1 again got scheduled and finish the critical section by executing the instruction no. 6 (setting interested variable to false).
- Now if P2 checks then it are going to satisfy the condition since other process's interested variable becomes false. P2 will also get enter the critical section.

Paterson solution

P1 $\rightarrow$ 6

P2 $\rightarrow$ 5 CS

- Any of the process may enter in the critical section for multiple numbers of times. Hence the procedure occurs in the cyclic order.

Mutual Exclusion

The method provides mutual exclusion for sure. In entry section, the while condition involves the criteria for two variables therefore a process cannot enter in the critical section until the other process is interested and the process is the last one to update turn variable.

Paterson solution

Progress

An uninterested process will never stop the other interested process from entering in the critical section. If the other process is also interested then the process will wait.

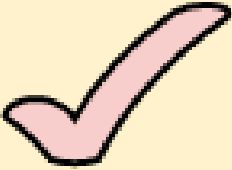
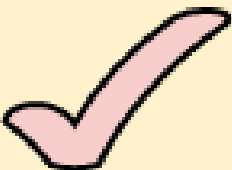
Bounded Waiting

A deadlock can never happen because the process which first sets the turn variable will enter in the critical section for sure. Therefore, if a process is preempted after executing line number 4 of the entry section then it will definitely get into the critical section in its next chance.

Portability

This is the complete software solution and therefore it is portable on every hardware.

Paterson solution

Mutual Exclusion	
Progress	
Bounded Waiting	
Portability	

Sleep and Wake-Producer consumer problem

- The concept of sleep and wake is if the critical section is not empty then the process will go and sleep. It will be waked up by the other process which is currently executing inside the critical section so that the process can get inside the critical section.
- In producer consumer problem, let us say there are two processes, one process writes something while the other process reads that. The process which is writing something is called **producer** while the process which is reading is called **consumer**.
- In order to read and write, both of them are using a buffer. The code that simulates the sleep and wake mechanism in terms of providing the solution to producer consumer problem is shown below

Sleep and Wake-Producer consumer problem

```
#define N 100 //maximum slots in buffer
#define count=0 //items in the buffer
void producer (void)
{
    int item;
    while(True)
    {
        item = produce_item(); //producer produces an item
        if(count == N) //if the buffer is full then the producer will sleep
            Sleep();
        insert_item (item); //the item is inserted into buffer
        count=count+1;
        if(count==1) //The producer will wake up the
            //consumer if there is at least 1 item in the buffer
            wake-up(consumer);
    }
}
```

Sleep and Wake-Producer consumer problem

```
void consumer (void)
{ int item;
  while(True)
  {
    {
      if(count == 0) //The consumer will sleep if the buffer is empty.
        sleep();
      item = remove_item();
      countcount = count - 1;
      if(count == N-1) //if there is at least one slot available in the buffer
        //then the consumer will wake up producer
        wake-up(producer);
      consume_item(item); //the item is read by consumer.
    }
  }
}
```

Sleep and Wake-Producer consumer problem

- The producer produces the item and inserts it into the buffer. The value of the global variable count got increased at each insertion.
- If the buffer is filled completely and no slot is available then the producer will sleep, otherwise it keep inserting.
- On the consumer's end, the value of count got decreased by 1 at each consumption.
- If the buffer is empty at any point of time then the consumer will sleep otherwise, it keeps consuming the items and decreasing the value of count by 1.
- The consumer will be waked up by the producer if there is at least 1 item available in the buffer which is to be consumed. The producer will be waked up by the consumer if there is at least one slot available in the buffer so that the producer can write that.

Sleep and Wake-Producer consumer problem

- The producer produces the item and inserts it into the buffer. The value of the global variable count got increased at each insertion.
- If the buffer is filled completely and no slot is available then the producer will sleep, otherwise it keep inserting.
- On the consumer's end, the value of count got decreased by 1 at each consumption.
- If the buffer is empty at any point of time then the consumer will sleep otherwise, it keeps consuming the items and decreasing the value of count by 1.
- The consumer will be waked up by the producer if there is at least 1 item available in the buffer which is to be consumed. The producer will be waked up by the consumer if there is at least one slot available in the buffer so that the producer can write that.

Sleep and Wake-Producer consumer problem

- Well, the problem arises in the case when the consumer got preempted just before it was about to sleep. Now the consumer is neither sleeping nor consuming. Since the producer is not aware of the fact that consumer is not actually sleeping therefore it keep waking the consumer while the consumer is not responding since it is not sleeping.

Sleep and Wake-Producer consumer problem

Using a flag bit to get rid of this problem

- A flag bit can be used in order to get rid of this problem.
- The producer can set the bit when it calls wake-up on the first time. When the consumer got scheduled, it checks the bit.
- The consumer will now get to know that the producer tried to wake him and therefore it will not sleep and get into the ready state to consume whatever produced by the producer.
- This solution works for only one pair of producer and consumer.

Sleep and Wake-Producer consumer problem

Thank You!

Deadlock

ICT1308

What is Deadlock in Operating System

- Every process needs some resources to complete its execution. However, the resource is granted in a sequential order.
 - The process request for some resources.
 - OS grants the resource if it is available otherwise let the process wait.
 - The process uses it and releases it on the completion.
- “A Deadlock is a situation where each of the computer process waits for a resource which is being assigned to some another process. In this situation, none of the process gets executed since the resource it needs, is held by some other process which is also waiting for some other resource to be released”.

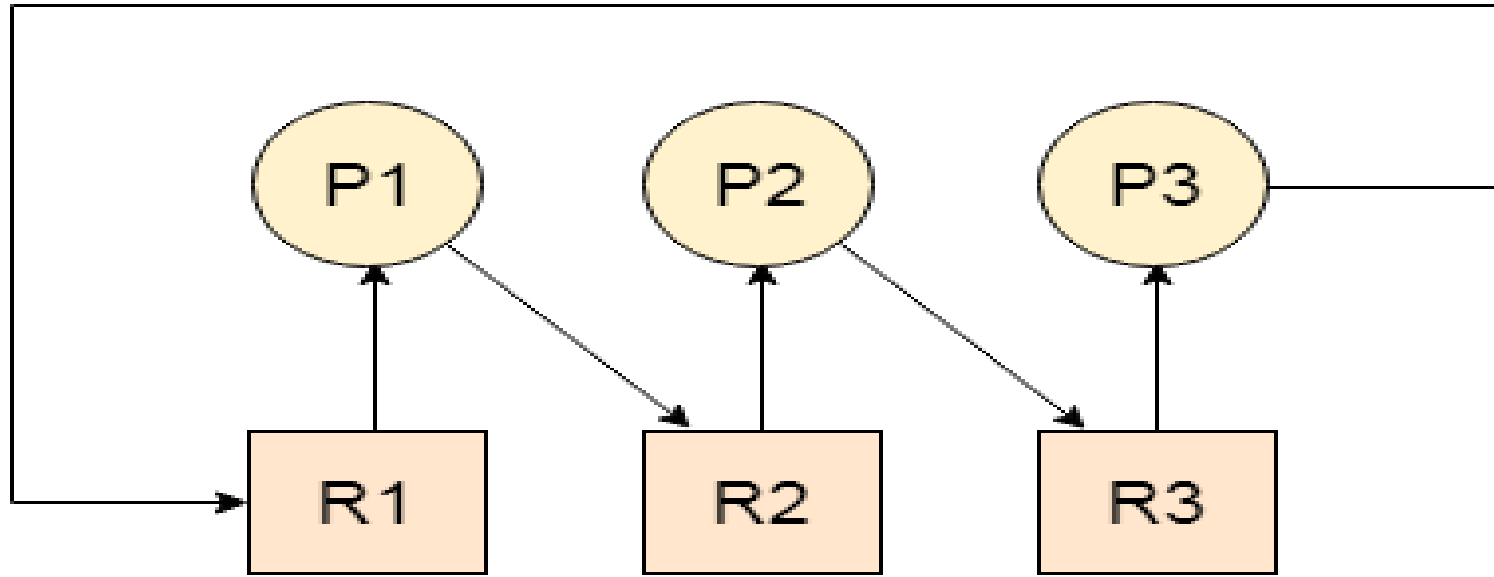
What is Deadlock in Operating System

- Lets assume that there are three processes P1,P2 and P3 and 3 resources R1,R2 and R3.
 - R1 is assigned to P1
 - R2 is assigned to P2
 - R3 is assigned to P3
- After some time P1 demands for R2 which is being used by P2.P1 halts its execution since it can't complete without R2.
- P2 also demands for R3 which is being used by P3.P2 also stops its execution because it can't continue without R3.

What is Deadlock in Operating System

- P3 also demands for R1 which is being used by P1 therefore P3 also stops its execution.
- In this scenario, a cycle is being formed by the three processes.
- None of the process is progressing and they are all waiting.
- The computer became unresponsive since all the processes got blocked.

What is Deadlock in Operating System



Difference between starvation and deadlock

Deadlock	Starvation
Deadlock is a situation where no process got blocked and no process proceed.	Low-priority processes got blocked and high-priority processes proceeded.
Deadlock is and infinite waiting	Starvation is a long waiting, not infinite
Every Deadlock is always a starvation.	Every starvation need not be deadlock.
The requested resource is blocked by the other process.	The requested resource is continuously be used by the higher priority processes.
Deadlock happens when Mutual exclusion, hold and wait, No preemption and circular wait occurs simultaneously.	It occurs due to uncontrolled priority and resource management.

Necessary conditions for deadlocks

Deadlock occurs when all the 4 conditions happen simultaneously.

1. Mutual Exclusion

- Two processes cannot use the same resource at the same time.

2. Hold and wait

- Process waits for some resources while holding another resource at the same time.

3. No preemption

- The process which once scheduled will be executed till the completion. No other process can be scheduled by the scheduler meanwhile

4. Circular wait

- All the processes must be waiting for the resources in a cyclic manner so that the last process is waiting for the resource which is being held by the first process.

Strategies for handling deadlock

1. Deadlock ignorance
2. Deadlock prevention
3. Deadlock avoidance
4. Deadlock detection and recovery

Deadlock Ignorance

- In this approach OS assumes deadlock never occurs and it simply ignores deadlock.
- This approach is best suitable for a single end-user system where user uses the system only for browsing and all other normal stuff.
- In these types of systems, the user has to simply restart the computer in the case of deadlock.
- Windows and Linux are mainly using this approach.

Deadlock prevention

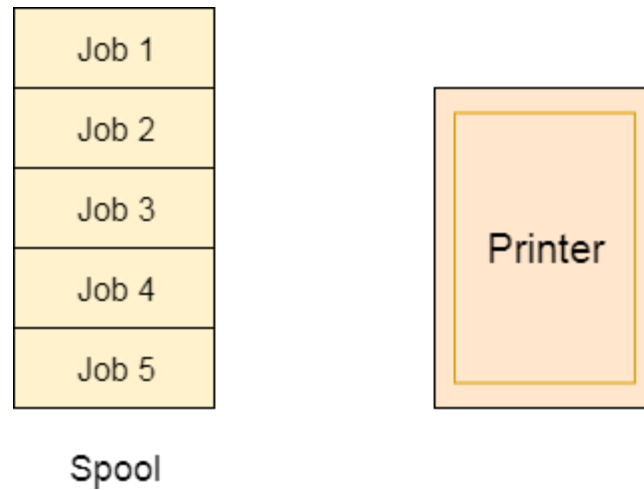
- Mutual exclusion, hold and wait, no preemption, and circular wait happening simultaneously cause the deadlock.
- If one condition is violated we can prevent happening of the deadlock.

Violating mutual exclusion

- Spooling is one suggested method to overcome mutual exclusion. Spooling can apply to a resource like a printer.
- There is a memory associated with the printer which stores jobs from each of the process into it.
- Later, Printer collects all the jobs and print each one of them according to FCFS.

Deadlock prevention

- By using this mechanism, the process doesn't have to wait for the printer and it can continue whatever it was doing.
- Later, it collects the output when it is produced.



Deadlock prevention

- Spooling suffers from two kinds of problems.
 - This cannot be applied to every resource.
 - After some time, race conditions arise between processes to get space in that spool.
- We cannot force a resource to be used by more than one process at the same time since it will not be fair enough and some serious problems may arise in the performance.
- Therefore, we cannot violate mutual exclusion for a process practically.

Deadlock prevention

Hold and wait

- Hold and wait condition lies when a process holds a resource and waits for some other resource to complete its task.
- This can be implemented practically if a process declares all the resources initially.
- But the computer system can't determine the necessary resources initially.
- The problem with the approach is:
 1. Practically not possible.
 2. Possibility of getting starved will increase due to the fact that some processes may hold a resource for a very long time.

Deadlock prevention

No preemption

- Deadlock arises due to the fact that a process can't be stopped once it starts.
- However, if we take the resource away from the process which is causing deadlock then we can prevent deadlock.
- This is not a good approach since if we take a resource away which is being used by the process then all the work which it has done till now can become inconsistent.

Deadlock prevention

Circular wait

- To violate circular wait, we can assign a priority number to each of the resources.
- A process can't request a lesser priority resource.
- This ensures that not a single process can request a resource that is being utilized by some other process and no cycle will be formed.
- Among all the methods, violating Circular wait is the only approach that can be implemented practically.

Deadlock avoidance

- In deadlock avoidance, the request for any resource will be granted if the resulting state of the system doesn't cause deadlock in the system.
- The state of the system will continuously be checked for safe and unsafe states.
- In order to avoid deadlocks, the process must tell OS, the maximum number of resources a process can request to complete its execution.
- Process should declare the maximum number of resources of each type it may ever need.

Deadlock avoidance

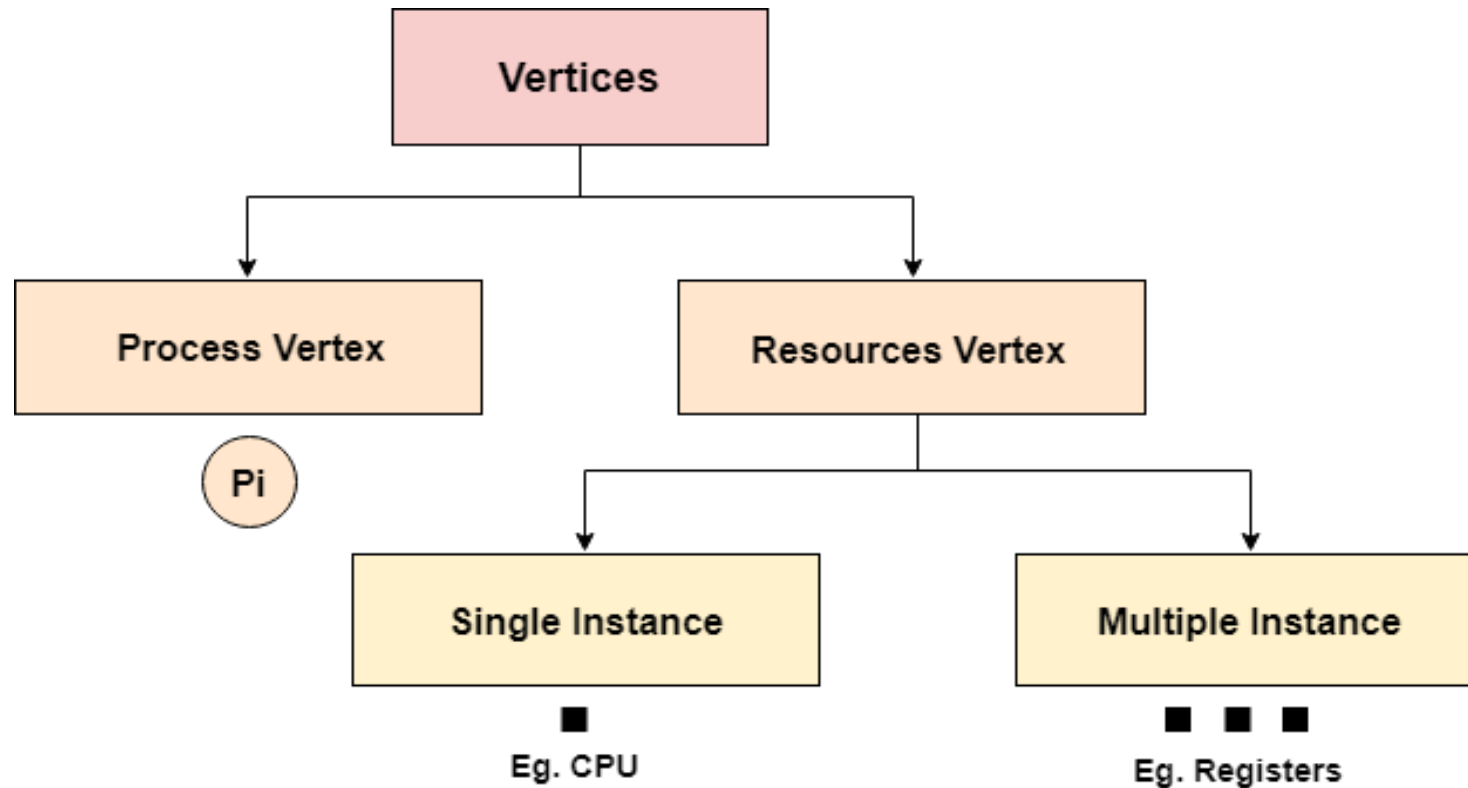
Safe and Unsafe states

- Safe state-> No deadlock
- Unsafe state-> Possibility of deadlock
- Avoidance->Assuring that system never enters in unsafe state.

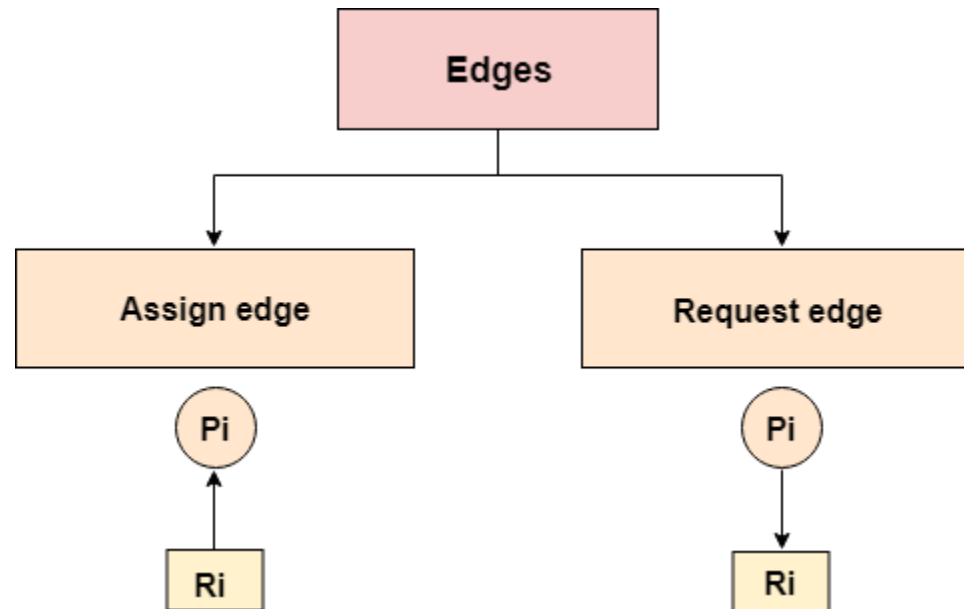
Resource Allocation Graph(RAG)

- Resource allocation graph is the complete information about all the processes that are holding some resources or waiting for some resources.
- In the Resource allocation graph, the process is represented by a Circle while the Resource is represented by a rectangle.
- Vertices are mainly of two types, Resource and process. Each of them will be represented by a different shape.
- The circle represents process while rectangle represents resource.

Resource Allocation Graph(RAG)



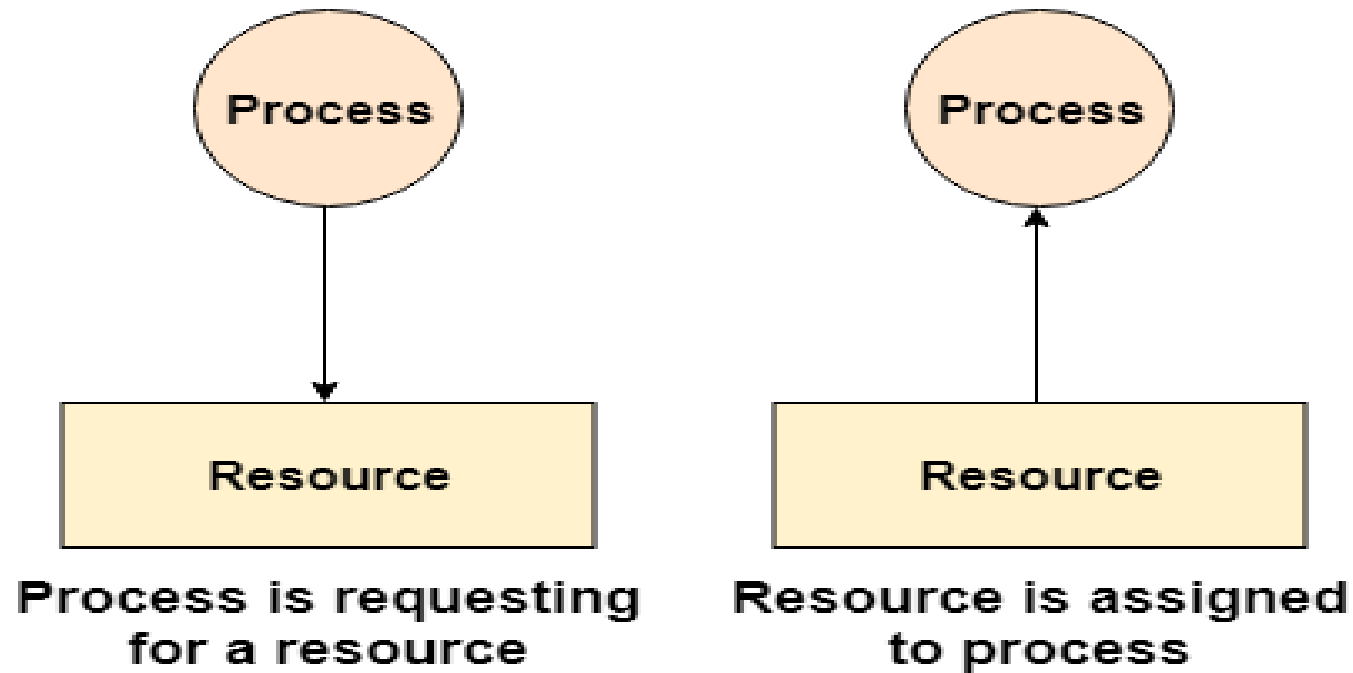
Resource Allocation Graph(RAG)



Resource Allocation Graph(RAG)

- A resource can have more than one instance. Each instance will be represented by a dot inside the rectangle.
- Edges in RAG are also of two types, one represents assignment and other represents the wait of a process for a resource.
- A resource is shown as assigned to a process if the tail of the arrow is attached to an instance to the resource and the head is attached to a process.
- A process is shown as waiting for a resource if the tail of an arrow is attached to the process while the head is pointing towards the resource.

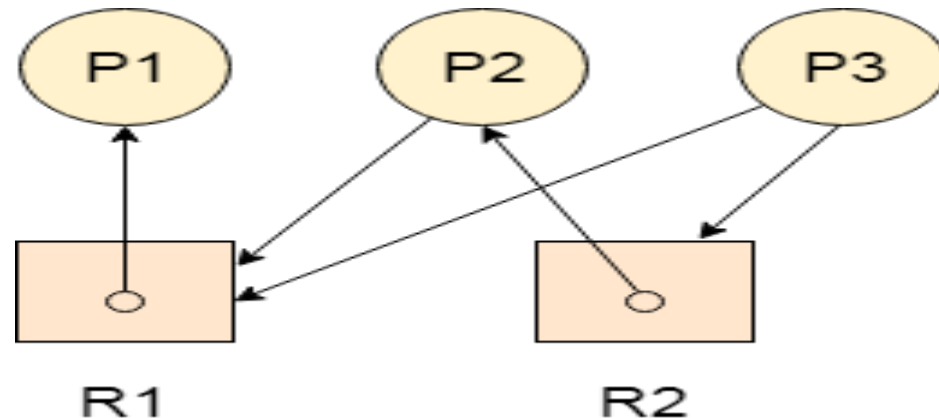
Resource Allocation Graph(RAG)



Resource Allocation Graph(RAG)

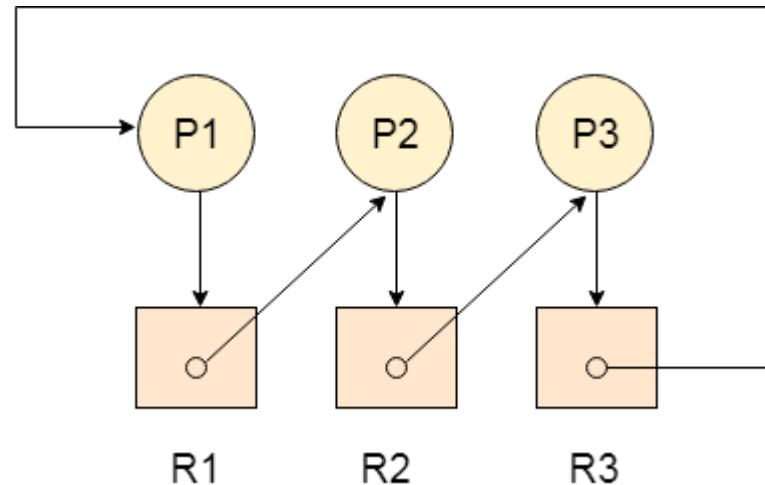
Example

- Let's consider 3 processes P1, P2 and P3, and two types of resources R1 and R2. The resources are having 1 instance each.
- According to the graph, R1 is being used by P1, P2 is holding R2 and waiting for R1, P3 is waiting for R1 as well as R2.
- The graph is deadlock free since no cycle is being formed in the graph.



Deadlock detection using RAG

- If a cycle is being formed in a Resource allocation graph where all the resources have the single instance then the system is deadlocked.
- In Case of Resource allocation graph with multi-instanced resource types, Cycle is a necessary condition of deadlock but not the sufficient condition.



Deadlock detection using RAG

- The before example contains three processes P1, P2, P3 and three resources R2, R2, R3. All the resources are having single instances each.
- If we analyze the graph then we can find out that there is a cycle formed in the graph since the system is satisfying all the four conditions of deadlock.

Deadlock detection using RAG

Allocation Matrix

- Allocation matrix can be formed by using the Resource allocation graph of a system. In Allocation matrix, an entry will be made for each of the resource assigned. For Example, in the following matrix, an entry is being made in front of P1 and below R3 since R3 is assigned to P1.

Process	R1	R2	R3
P1	0	0	1
P2	1	0	0
P3	0	1	0

Deadlock detection using RAG

Request Matrix

- In request matrix, an entry will be made for each of the resource requested. As in the following example, P1 needs R1 therefore an entry is being made in front of P1 and below R1.

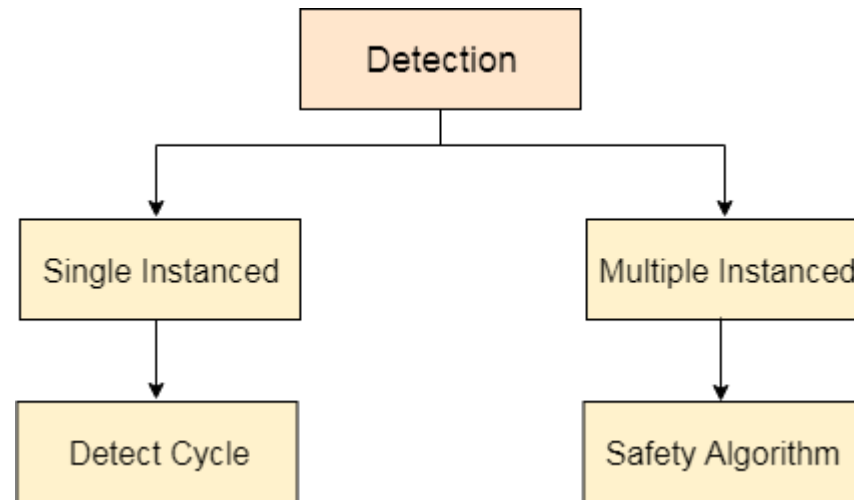
Process	R1	R2	R3
P1	1	0	0
P2	0	1	0
P3	0	0	1

Deadlock detection using RAG

- Neither we are having any resource available in the system nor a process going to release.
- Each of the process needs at least single resource to complete therefore they will continuously be holding each one of them.
- We cannot fulfill the demand of at least one process using the available resources therefore the system is deadlocked

Deadlock detection and Recovery

- The main task of the OS is detecting the deadlocks.
- The OS can detect the deadlocks with the help of Resource allocation graph.

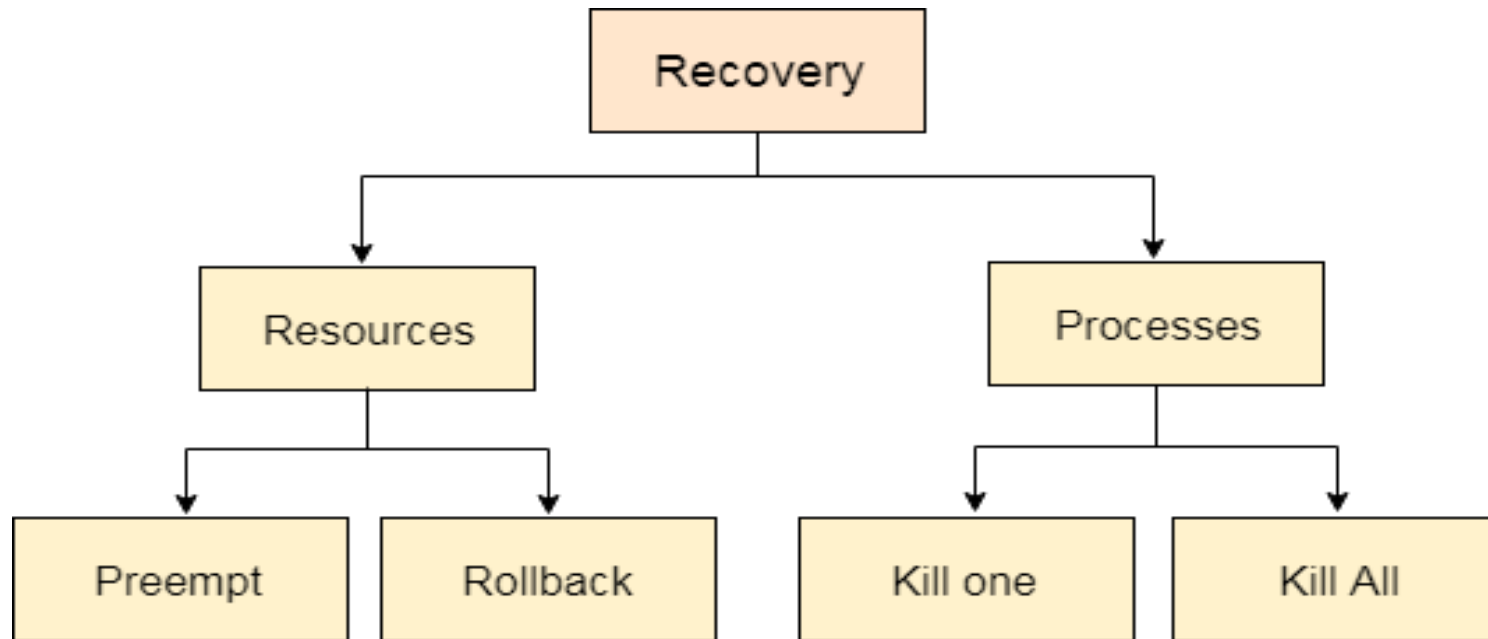


Deadlock detection and Recovery

- In single instanced resource types, if a cycle is being formed in the system then there will definitely be a deadlock.
- On the other hand, in multiple instanced resource type graph, detecting a cycle is not just enough.
- We have to apply the safety algorithm on the system by converting the resource allocation graph into the allocation matrix and request matrix.

Deadlock detection and Recovery

- In order to recover the system from deadlocks, either OS considers resources or processes.



Deadlock detection and Recovery

- Preempt the resource-We can snatch one of the resources from the owner of the resource (process) and give it to the other process with the expectation that it will complete the execution and will release this resource sooner.
- Rollback to a safety state-System passes through various states to get into the deadlock state. The operating system can rollback the system to the previous safe state. For this purpose, OS needs to implement check pointing at every state.
- Kill a process-Generally, Operating system kills a process which has done least amount of work until now.
- Kill all processes-Killing all process will lead to inefficiency in the system because all the processes will execute again from starting.

Thank You!

Memory Management

ICT1308

Introduction

What is Main Memory?

- The main memory is central to the operation of a Modern Computer.
- Main Memory is a large array of words or bytes, ranging in size from hundreds of thousands to billions.
- Main memory is a repository of rapidly available information shared by the CPU and I/O devices.
- Main memory is the place where programs and information are kept when the processor is effectively utilizing them.
- Main memory is associated with the processor, so moving instructions and information into and out of the processor is extremely fast.

Introduction

- Main memory is also known as RAM (Random Access Memory). This memory is volatile. RAM loses its data when a power interruption occurs.

What is memory management?

- In a multiprogramming computer, the Operating System resides in a part of memory, and the rest is used by multiple processes.
- The task of subdividing the memory among different processes is called Memory Management.
- Memory management is a method in the operating system to manage operations between main memory and disk during process execution.
- The main aim of memory management is to achieve efficient utilization of memory.

Introduction

Why memory management is required?

- Allocate and de-allocate memory before and after process execution.
- To keep track of used memory space by processes.
- To minimize fragmentation issues.
- To proper utilization of main memory.
- To maintain data integrity while executing of process.

Logical and Physical address space

- **Logical Address space-** An address generated by the CPU is known as a “Logical Address”. It is also known as a Virtual address. Logical address space can be defined as the size of the process. A logical address can be changed.
- **Physical Address space** –Address seen by memory unit is called ‘physical address or Real address’. The set of all physical addresses corresponding to these logical addresses is known as Physical address space. The run-time mapping from virtual to physical addresses is done by a hardware device Memory Management Unit(MMU). The physical address always remains constant.

Static and Dynamic Loading

- Loading a process into the main memory is done by a loader. There are two different types of loading :
 1. Static Loading-Loading the entire program into a fixed address. It requires more memory space.
 2. Dynamic Loading-The entire program and all data of a process must be in physical memory for the process to execute. So, the size of a process is limited to the size of physical memory. To gain proper memory utilization, dynamic loading is used. In dynamic loading, a routine is not loaded until it is called. All routines are residing on disk in a relocatable load format.

Fixed Partitioning

- The earliest and one of the simplest technique which can be used to load more than one processes into the main memory is Fixed partitioning or Contiguous memory allocation.
- In this technique, the main memory is divided into partitions of equal or different sizes.
- The operating system always resides in the first partition while the other partitions can be used to store user processes.
- The memory is assigned to the processes in contiguous way.

Fixed partitioning

1. The partition cannot overlap
2. A process must be contiguously present in a partition for the execution.

- There are various cons of using fixed fragmentation.

1. Internal fragmentation-

- If the size of the process is lesser than the total size of the partition then some size of the partition get wasted and remain unused.
- This is wastage of the memory and called internal fragmentation.
- For example the 4 MB partition is used to load only 3 MB process and the remaining 1 MB got wasted.

Fixed partitioning

2.External Fragmentation

- The total unused space of various partitions cannot be used to load the processes even though there is space available but not in the contiguous form.
- For example remaining 1MB space of each partition cannot be used as a unit to store a 4MB process.

3.Limitation of the size of the process

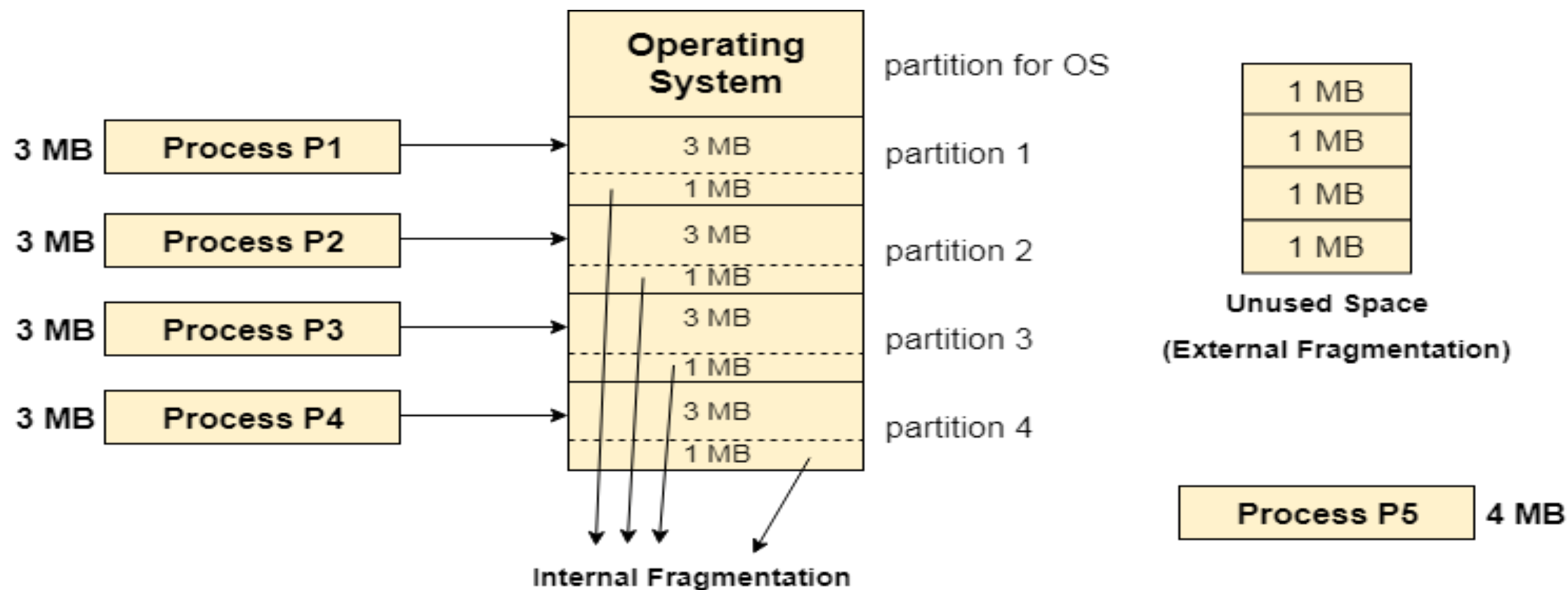
- If the process size is larger than the size of maximum sized partition then that process cannot be loaded into the memory.

Fixed partitioning

- Therefore, a limitation can be imposed on the process size that is it cannot be larger than the size of the largest partition.

4.Degree of multiprogramming is less

- By Degree of multi programming, we simply mean the maximum number of processes that can be loaded into the memory at the same time.
- In fixed partitioning, the degree of multiprogramming is fixed and very less due to the fact that the size of the partition cannot be varied according to the size of processes.



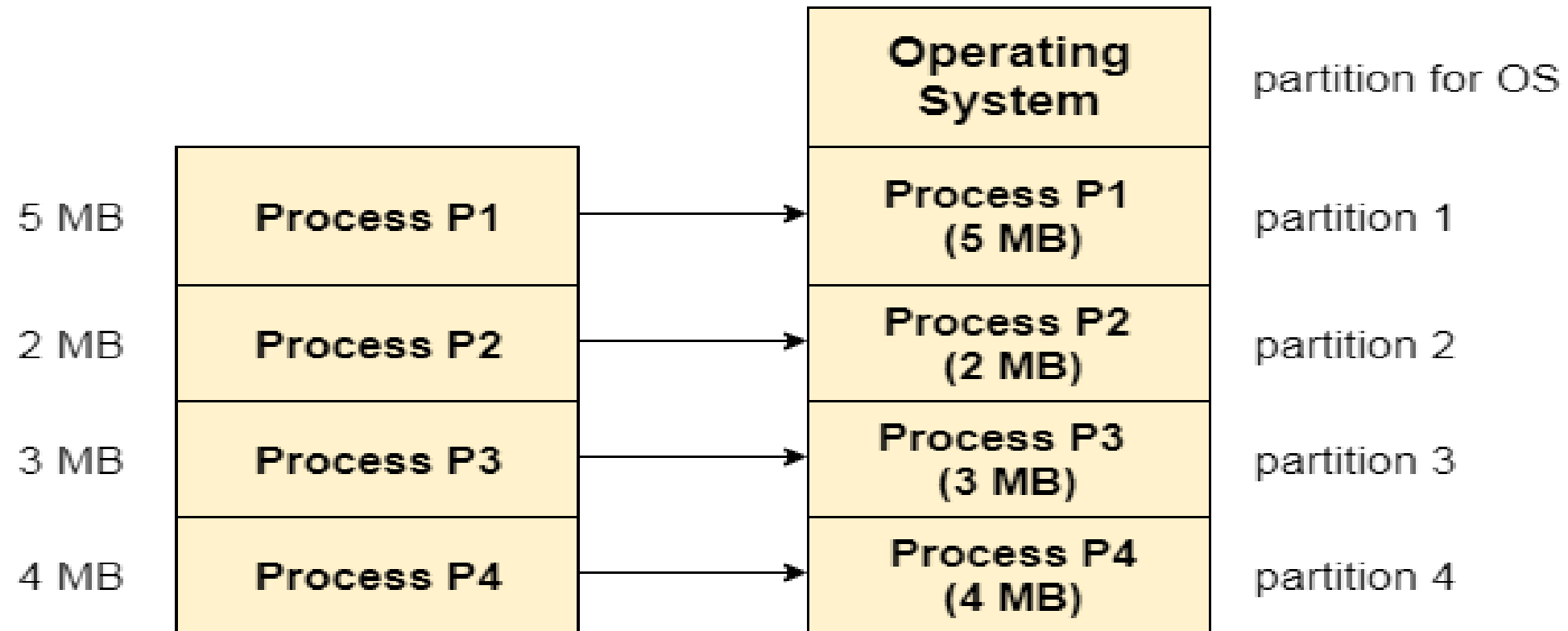
Fixed Partitioning

(Contiguous memory allocation)

Dynamic Partitioning

- Dynamic partitioning tries to overcome the problems caused by fixed partitioning.
- In this technique, the partition size is not declared initially. It is declared at the time of process loading.
- The first partition is reserved for the operating system.
- The remaining space is divided into parts. The size of each partition will be equal to the size of the process.
- The partition size varies according to the need of the process so that the internal fragmentation can be avoided.

Dynamic Partitioning



Dynamic Partitioning

(Process Size = Partition Size)

Advantages of dynamic partitioning

1. No internal Fragmentation

- As the dynamic partitioning is created according to the need of the process there is no internal fragmentation as there will not be any unused remaining space in the partition.

2. No limitation on the size of the process

- The process size can't be restricted since the partition size is decided according to the process size.

3. Degree of multiprogramming is dynamic

- Due to the absence of internal fragmentation, there will not be any unused space in the partition hence more processes can be loaded in the memory at the same time.

Disadvantages of Dynamic Partitioning

1. External fragmentation

- Absence of internal fragmentation doesn't mean that there will not be external fragmentation.
- If three processes P1 (1 MB) and P2 (3 MB) and P3 (1 MB) are being loaded in the respective partitions of the main memory. And after some time P1 and P3 completed and their assigned space is freed. Now there are two unused partitions (1 MB and 1 MB) available in the main memory but they cannot be used to load a 2 MB process in the memory since they are not contiguously located.
- The rule says that the process must be contiguously present in the main memory to get executed. We need to change this rule to avoid external fragmentation.

Disadvantages of Dynamic Partitioning

2. Complex Memory allocation

- In Fixed partitioning, the list of partitions is made once and will never change but in dynamic partitioning, the allocation and deallocation is very complex since the partition size will be varied every time when it is assigned to a new process.
- OS has to keep track of all the partitions.
- Due to the fact that the allocation and deallocation are done very frequently in dynamic memory allocation and the partition size will be changed at each time, it is going to be very difficult for OS to manage everything.

Bit Map for Dynamic Partitioning

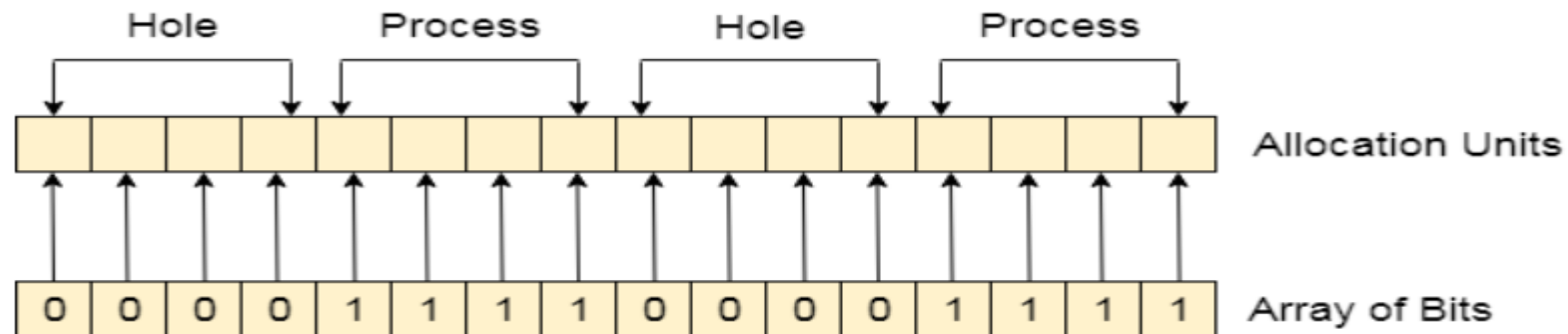
- The Main concern for dynamic partitioning is keeping track of all the free and allocated partitions.
- However, the Operating system uses following data structures for this task.
 - Bit Map
 - Linked List

Bit Map for Dynamic Partitioning

- In bit map allocation main memory is divided into collection of allocation units.
- One or more allocation units may be allocated to a process according to the need of that process.
- However, the size of the allocation unit is fixed that is defined by the Operating System and never changed.
- Although the partition size may vary but the allocation size is fixed.
- The main task of the operating system is to keep track of whether the partition is free or filled.
- For this purpose, the operating system also manages another data structure that is called bitmap.

Bit Map for Dynamic Partitioning

- The process or the hole in Allocation units is represented by a flag bit of bitmap.
- The flag bit is set to 1 if there is a contiguously present process at the adjacent bit in allocation unit otherwise it is set to 0.
- A string of 0s in the bitmap shows that there is a hole in the relative Allocation unit while the string of 1s represents the process in the relative allocation unit.



Linked list for Dynamic Partitioning

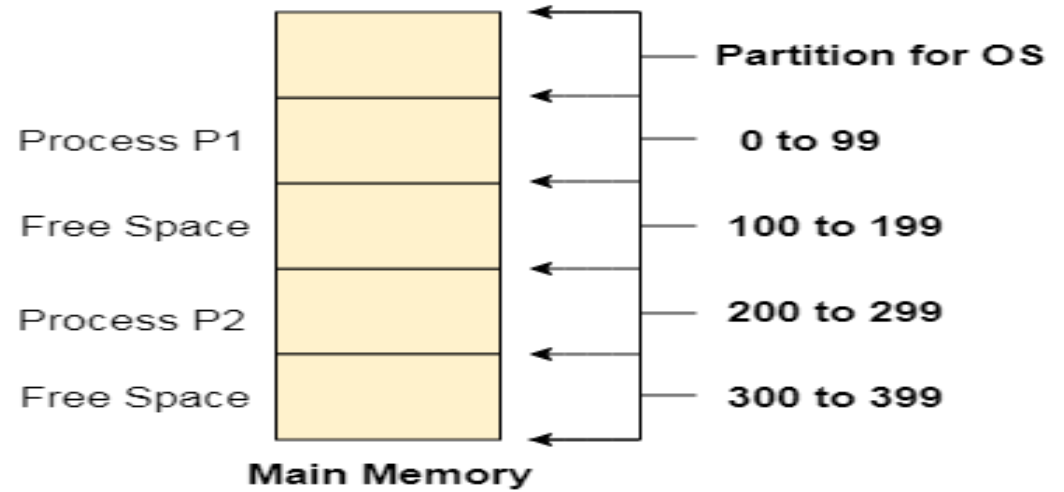
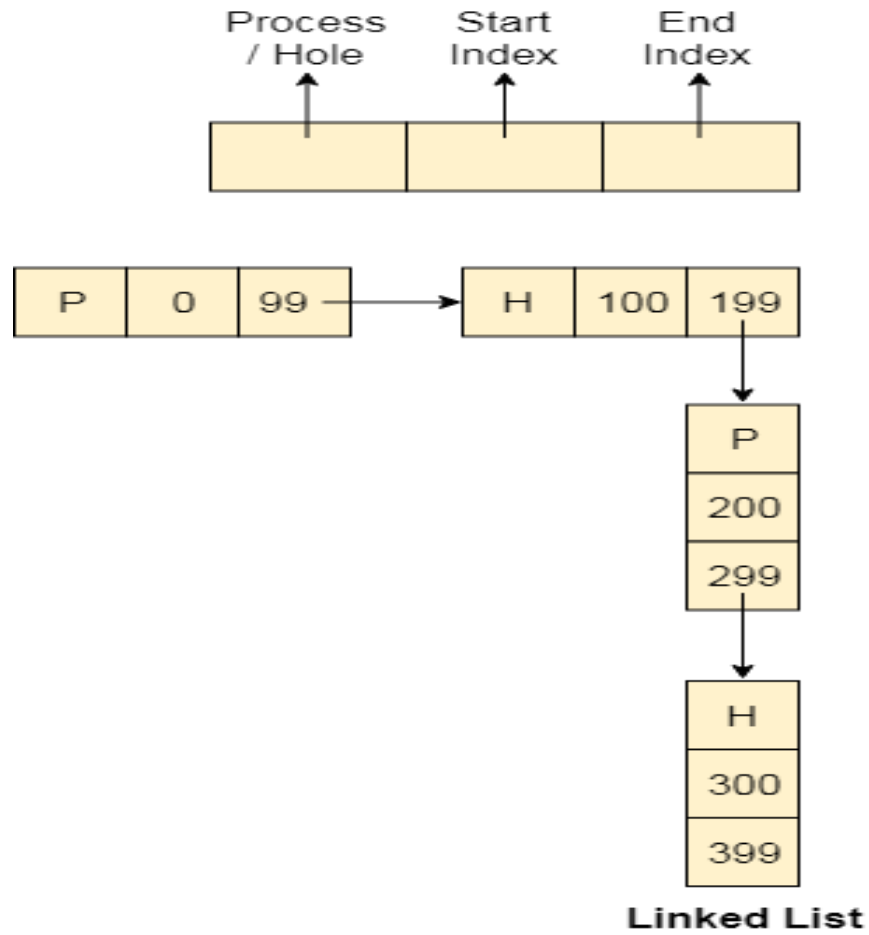
- Most popular approach to keep track the free or filled partitions is using Linked List.
- In this approach, the Operating system maintains a linked list where each node represents each partition. Every node has three fields.
 - 1.First field of the node stores a flag bit which shows whether the partition is a hole or some process is inside.
 - 2.Second field stores the starting index of the partition.
 - 3.Third field stores the end index of the partition.
- If a partition is freed at some point of time then that partition will be merged with its adjacent free partition without doing any extra effort.

Linked list for Dynamic Partitioning

- While using this approach
 - 1.The OS must be very clear about the location of the new node which is to be added in the linked list. However, adding the node according to the increasing order of starting index is suggestible.
 - 2.Using a doubly linked list will make some positive effects on the performance due to the fact that a node in the doubly link list can also keep track of its previous node.

Linked list for Dynamic Partitioning

Linked List for Dynamic Partitioning



Partitioning algorithms

- There are various algorithms which are implemented by OS in order to find out the holes in the linked list and allocate the to processes.
 1. First Fit algorithm
 2. Next Fi algorithm
 3. Best Fit algorithm
 4. Worst Fit algorithm
 5. Quick Fit algorithm

Partitioning algorithms

1. First Fit algorithm

- First Fit algorithm scans the linked list and whenever it finds the first big enough hole to store a process, it stops scanning and load the process into that hole.
- This procedure produces two partitions. Out of them, one partition will be a hole while the other partition will store the process.
- First Fit algorithm maintains the linked list according to the increasing order of starting index. This is the simplest to implement among all the algorithms and produces bigger holes as compare to the other algorithms.

Partitioning algorithms

2. Next Fit algorithm

- Next fit doesn't scan the whole list, it starts scanning the list from the next node.
- The idea behind the next fit is the fact that the list has been scanned once therefore the probability of finding the hole is larger in the remaining part of the list.
- Experiments over the algorithm have shown that the next fit is not better than the first fit.
- So it is not being used these days in most of the cases.

Partitioning algorithms

3. Best Fit algorithm

- The Best Fit algorithm tries to find out the smallest hole possible in the list that can accommodate the size requirement of the process.
- It is slower because it scans the entire list every time and tries to find out the smallest hole.
- Due to the fact that the difference between the whole size and the process size is very small, the holes produced will be as small as it cannot be used to load any process and therefore it remains useless.
- Despite of the fact that the name of the algorithm is best fit, It is not the best algorithm among all.

Partitioning algorithms

4. Worst Fit algorithm

- Scans the entire list every time and tries to find out the biggest hole in the list which can fulfill the requirement of the process.

5.Quick fit algorithm

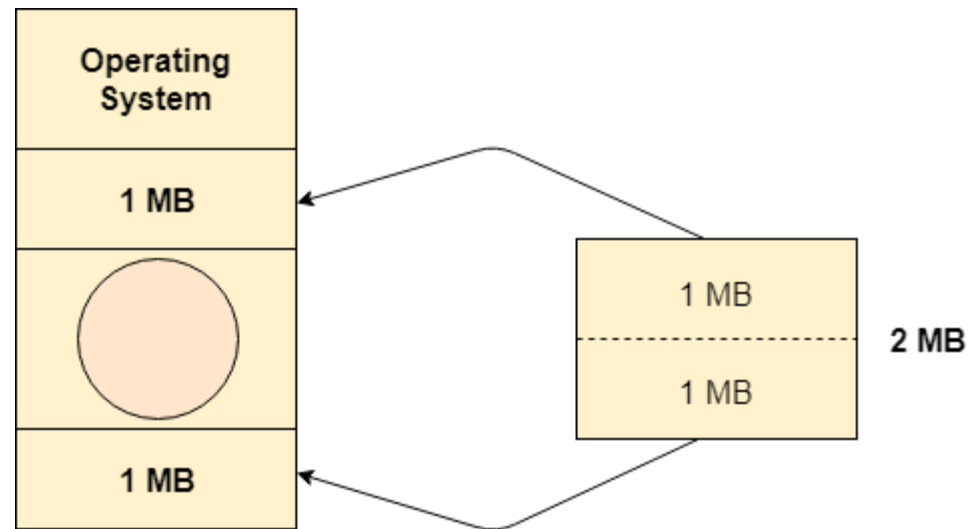
- The quick fit algorithm suggests maintaining the different lists of frequently used sizes.
- Although, it is not practically suggestible because the procedure takes so much time to create the different lists and then expending the holes to load a process.

Partitioning algorithms

- The first fit algorithm is **the best algorithm** among all because
 - 1.It takes lesser time compare to the other algorithms.
 - 2.It produces bigger holes that can be used to load other processes later on.
 - 3.It is easiest to implement.

Paging in OS

- The Idea behind paging is to divide the process in pages so that, we can store them in the memory at different holes.

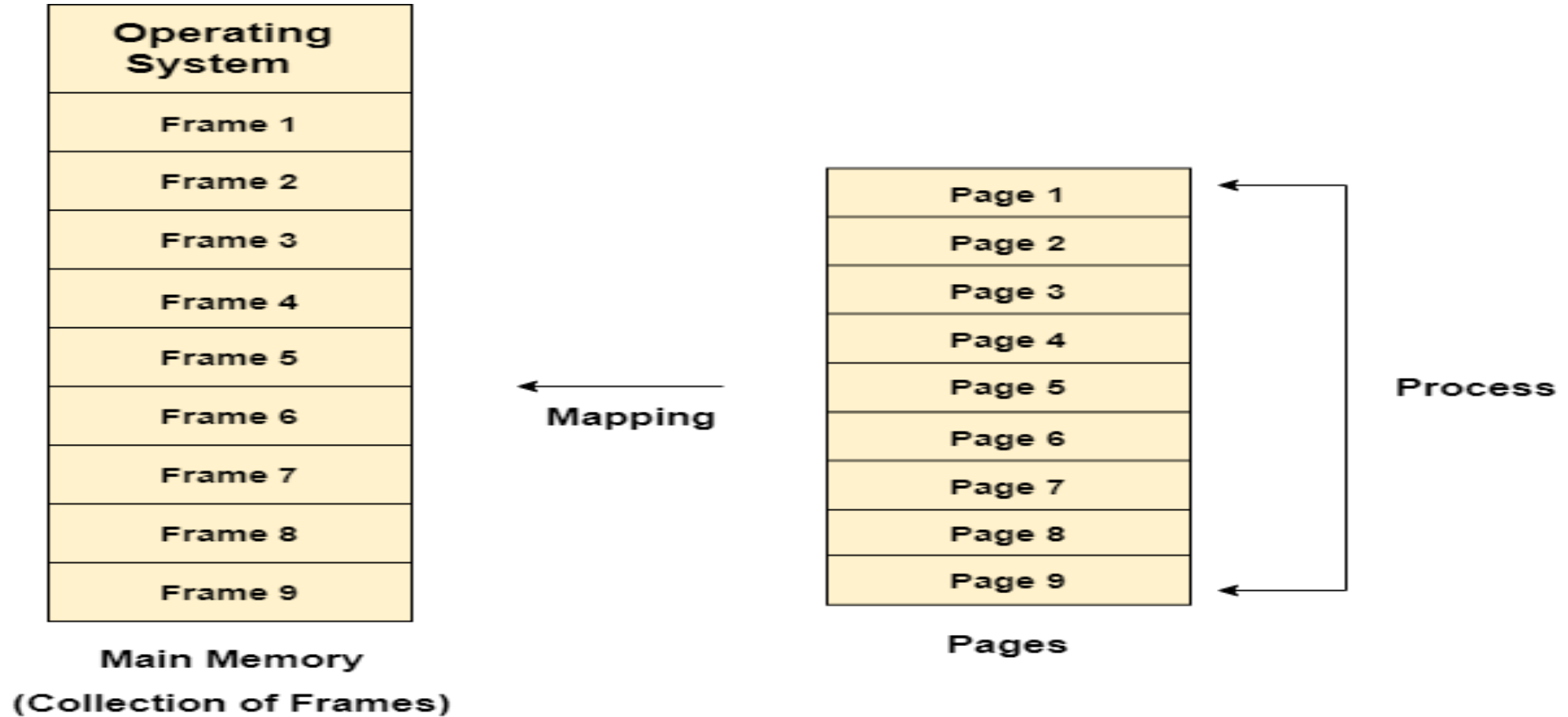


The process needs to be divided into two parts to get stored at two different places.

Paging in OS

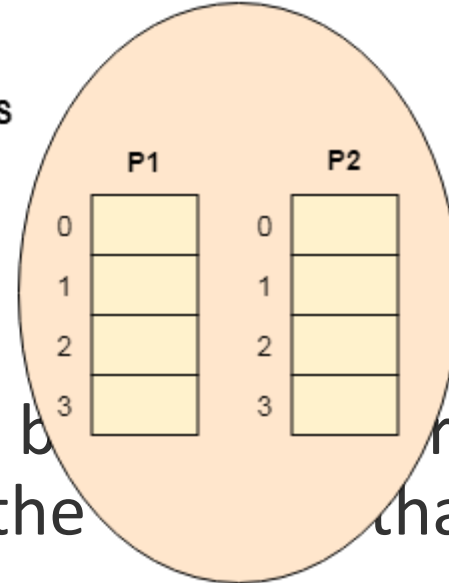
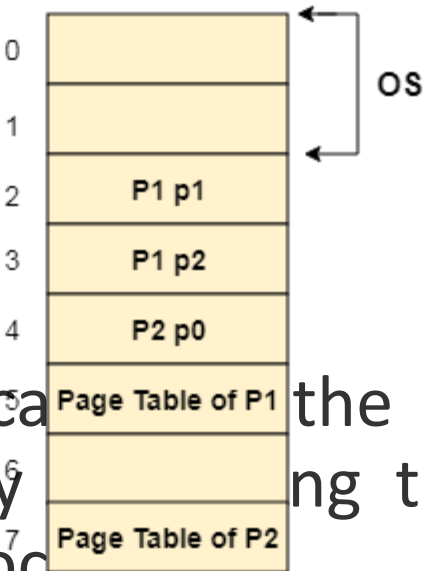
- In Operating Systems, Paging is a storage mechanism used to retrieve processes from the secondary storage into the main memory in the form of pages.
- The main idea behind the paging is to divide each process in the form of pages. The main memory will also be divided in the form of frames.
- One page of the process is to be stored in one of the frames of the memory. The pages can be stored at the different locations of the memory but the priority is always to find the contiguous frames or holes.
- Different operating system defines different frame sizes. The sizes of each frame must be equal. Considering the fact that the pages are mapped to the frames in Paging, page size needs to be as same as frame size.

Paging in OS



	Frame	Present / Absent	D Bit	Reference bit	Protection
0		0	0	0	
1	2	1	0	1	
2	3	1	1	0	
3		0	0	0	

Page Table of P1



- In this scheme, User can load more processes than the available main memory by using the fact that the memory is available to load the process.

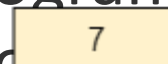
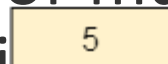
	Frame	Present / Absent	D Bit	Reference bit	Protection
0	4	1	0	1	
1		0	0	0	
2		0	0	0	
3		0	0	0	

Page Table of P2

- Instead of loading one big process in the main memory, the Operating System can load different parts of more than one process in the main memory.



- The degree of multiprogramming will be increased and the utilization of the system will also be increased.



Page Table Base (P1) Page Table Base (P2)

How virtual memory works?

- In this scheme, whenever some pages need to be loaded in the main memory for the execution and the memory is not available for those many pages, then in that case, instead of stopping the pages from entering in the main memory, the OS searches for the RAM area that are least used in the recent times or that are not referenced and copy that into the secondary memory to make the space for the new pages in the main memory.

Virtual Memory

Advantages of virtual memory

- 1.The degree of Multiprogramming will be increased.
- 2.User can run large application with less real RAM.
- 3.There is no need to buy more memory RAMs.

Disadvantages of virtual memory

- 1.The system becomes slower since swapping takes time.
- 2.It takes more time in switching between applications.
- 3.The user will have the lesser hard disk space for its use.

Demand Paging

- In demand paging, the pages of a process which are least used, get stored in the secondary memory.
- A page is copied to the main memory when its demand is made or page fault occurs.
- Whenever any page is referred for the first time in the main memory, then that page will be found in the secondary memory.
- If the referred page is not present in the main memory then there will be a miss and the concept is called Page miss or page fault.
- The CPU has to access the missed page from the secondary memory. If the number of page fault is very high then the effective access time of the system will become very high.

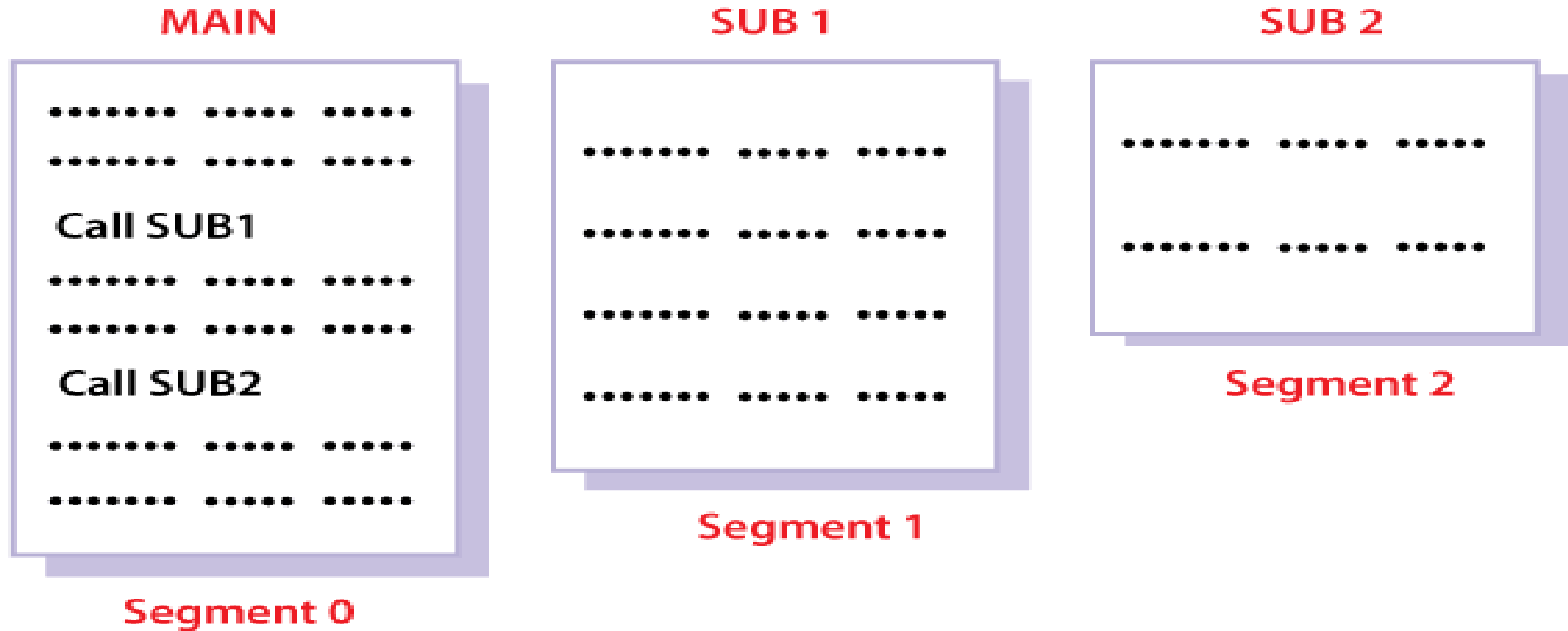
Segmentation

- Segmentation is a memory management technique in which the memory is divided into variable size parts. Each part is known as a segment which can be allocated to a process.
- The details about each segment are stored in a table called a segment table. Segment table is stored in one (or many) of the segments.
- Segment table contains mainly two information about segment:
 - 1.Base: It is the base address of the segment
 - 2.Limit: It is the length of the segment.
- Previously used paging technique for main memory management was more close to OS than user.

Segmentation

- It divides all the processes into the form of pages regardless of the fact that a process can have some relative parts of functions which need to be loaded in the same page.
- It is better to have segmentation which divides the process into the segments.
- Each segment contains the same type of functions such as the main function can be included in one segment and the library functions can be included in the other segment.

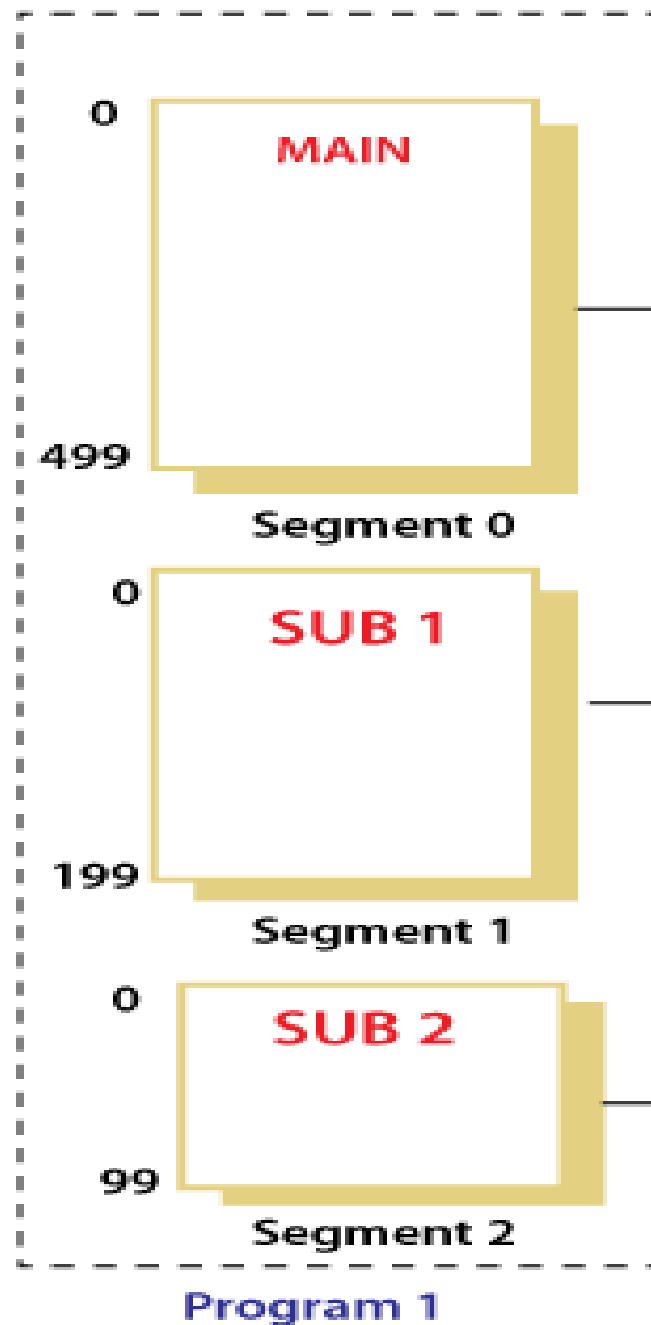
Segmentation



Segmentation

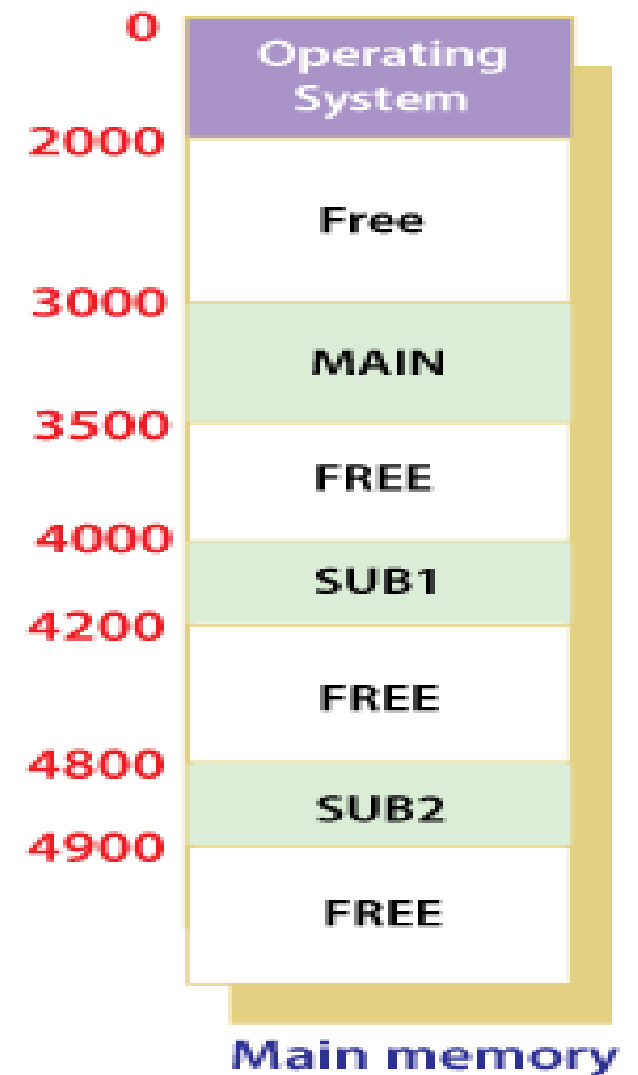
Example

- When a program is loaded into memory, the segmentation system tries to locate space that is large enough to hold the first segment of the process, space information is obtained from the free list maintained by memory manager.
- Then it tries to locate space for other segments. Once adequate space is located for all the segments, it loads them into their respective areas.



Limit	Base Address	Access
500	3000	Executable
200	4000	Executable
100	4800	Executable

Segment Map Table(SMT)
for process 1



Paging	Segmentation
Non-Contiguous memory allocation	Non-contiguous memory allocation
Paging divides program into fixed size pages.	Segmentation divides program into variable size segments.
OS is responsible	Compiler is responsible.
Paging is faster than segmentation	Segmentation is slower than paging
Paging is closer to Operating System	Segmentation is closer to User
It suffers from internal fragmentation	It suffers from external fragmentation
There is no external fragmentation	There is no internal fragmentation
Logical address is divided into page number and page offset	Logical address is divided into segment number and segment offset
Page table is used to maintain the page information.	Segment Table maintains the segment information
Page table entry has the frame number and some flag bits to represent details about pages.	Segment table entry has the base address of the segment and some protection bits for the segments.

Thank you!

File Management

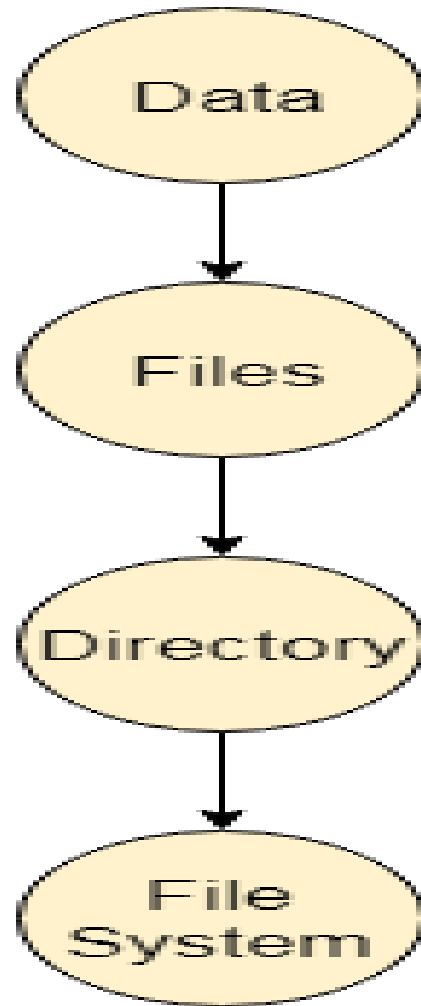
ICT1308

M.M.A.Nisansala

Introduction

What is a File?

- A file can be defined as a data structure which stores the sequence of records.
- Files are stored in a file system, which may exist on a disk or in the main memory.
- Files can be simple (plain text) or complex (specially-formatted).
- The collection of files is known as Directory.
- The collection of directories at the different levels, is known as File System.



Attributes of the File

1. Name

Every file carries a name by which the file is recognized in the file system. One directory cannot have two files with the same name.

2. Identifier

Along with the name, Each File has its own extension which identifies the type of the file. For example, a text file has the extension **.txt**, A video file can have the extension **.mp4**.

3. Location

In the File System, there are several locations on which, the files can be stored. Each file carries its location as its attribute.

Attributes of the File

4. Type

In a File System, the Files are classified in different types such as video files, audio files, text files, executable files, etc.

5. Size

The Size of the File is one of its most important attribute. By size of the file, we mean the number of bytes acquired by the file in the memory.

6. Protection

The Admin of the computer may want the different protections for the different files. Therefore each file carries its own set of permissions to the different group of Users.

7. Time and Date

Every file carries a time stamp which contains the time and date on which the file is last modified.

Operations on the File

- The various operations which can be implemented on a file such as read, write, open and close etc. are called file operations.
- These operations are performed by the user by using the commands provided by the operating system.

1.Create Operation

This operation is used to create a file in the file system. To create a new file of a particular type the associated application program calls the file system. This file system allocates space to the file. As the file system knows the format of directory structure, so entry of this new file is made into the appropriate directory.

Operations on the File

2. Open Operation

When the user wants to open a file, it provides a file name to open the particular file in the file system. It tells the operating system to invoke the open system call and passes the file name to the file system.

3. Write operation

This operation is used to write the information into a file. A system call write is issued that specifies the name of the file and the length of the data has to be written to the file. Whenever the file length is increased by specified value and the file pointer is repositioned after the last byte written.

Operations on the File

4. Read Operation

This operation reads the contents from a file. A Read pointer is maintained by the OS, pointing to the position up to which the data has been read.

5. Re-operation or Seek operation

The seek system call re-positions the file pointers from the current position to a specific place in the file i.e. forward or backward depending upon the user's requirement.

Operations on the File

6. Delete Operation

Deleting the file will not only delete all the data stored inside the file it is also used so that disk space occupied by it is freed. In order to delete the specified file the directory is searched. When the directory entry is located, all the associated file space and the directory entry is released.

7. Truncate operation

Truncating is simply deleting the file except deleting attributes. The file is not completely deleted although the information stored inside the file gets replaced.

Operations on the File

8. Close operation

When the processing of the file is complete, it should be closed so that all the changes made permanent and all the resources occupied should be released.

9. Append operation

This operation adds data to the end of the file.

10. Rename operation

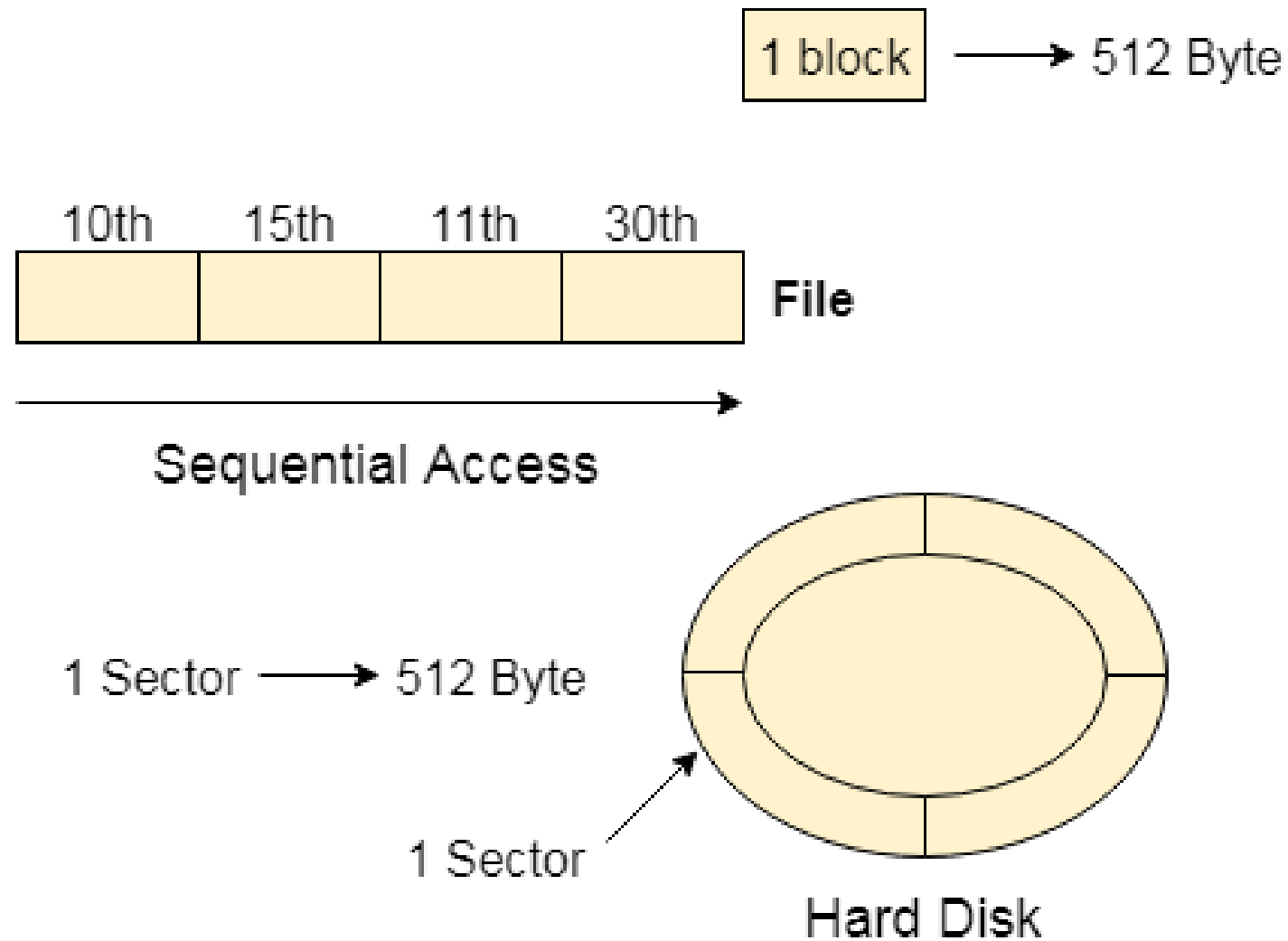
This operation used to rename the existing file.

File access methods

1.Sequential Access

- In sequential access, the OS read the file word by word. A pointer is maintained which initially points to the base address of the file. If the user wants to read first word of the file then the pointer provides that word to the user and increases its value by 1 word. This process continues till the end of the file.
- Sequential access is mostly used due to the fact that most of the files such as text files, audio files, video files, etc need to be sequentially accessed.

File access methods

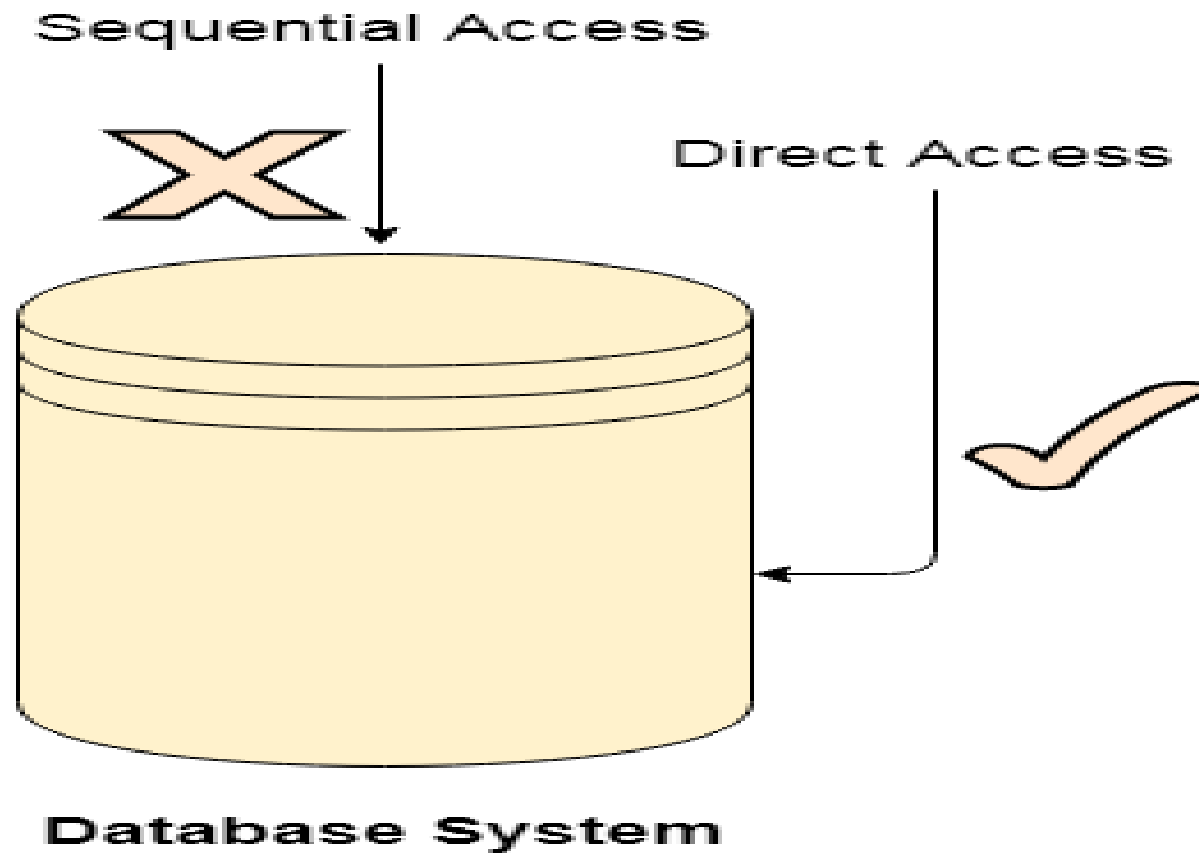


File access methods

2. Direct access

- The Direct Access is mostly required in the case of database systems.
- In most of the cases, we need filtered information from the database. The sequential access can be very slow and inefficient in such cases.
- Suppose every block of the storage stores 4 records and we know that the record we needed is stored in 10th block.
- In that case, the sequential access will not be implemented because it will traverse all the blocks in order to access the needed record.

File access methods



File access methods

3. Indexed Access

- If a file can be sorted on any of the filed then an index can be assigned to a group of certain records.
- However, A particular record can be accessed by its index. The index is nothing but the address of a record in the file.
- In index accessing, searching in a large database became very quick and easy but we need to have some extra space in the memory to store the index value.

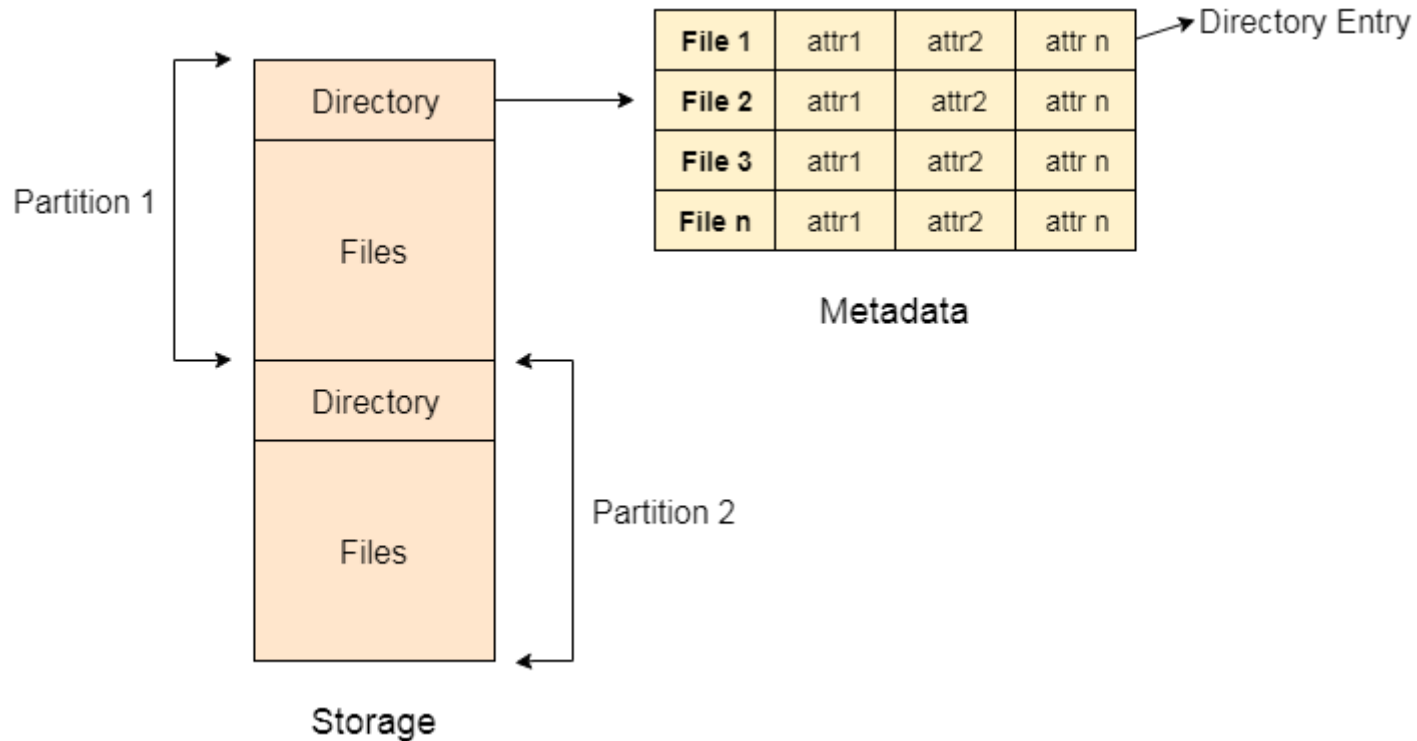
Directory Structure in OS

What is a directory?

- Directory can be defined as the listing of the related files on the disk.
- The directory may store some or the entire file attributes.
- To get the benefit of different file systems on the different operating systems, A hard disk can be divided into the number of partitions of different sizes.
- The partitions are also called volumes or mini disks.
- Each partition must have at least one directory in which, all the files of the partition can be listed.

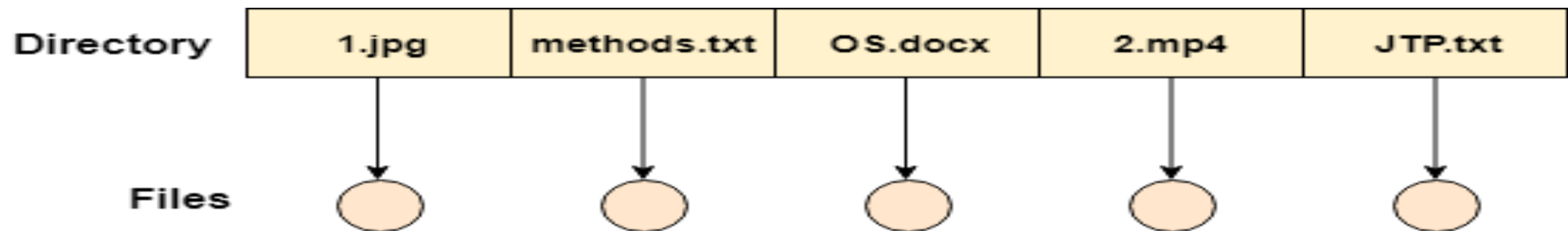
Directory structures in OS

- A directory entry is maintained for each file in the directory which stores all the information related to that file.



Single level directory

- The simplest method is to have one big list of all the files on the disk. The entire system will contain only one directory which is supposed to mention all the files present in the file system.
- The directory contains one entry per each file present on the file system.

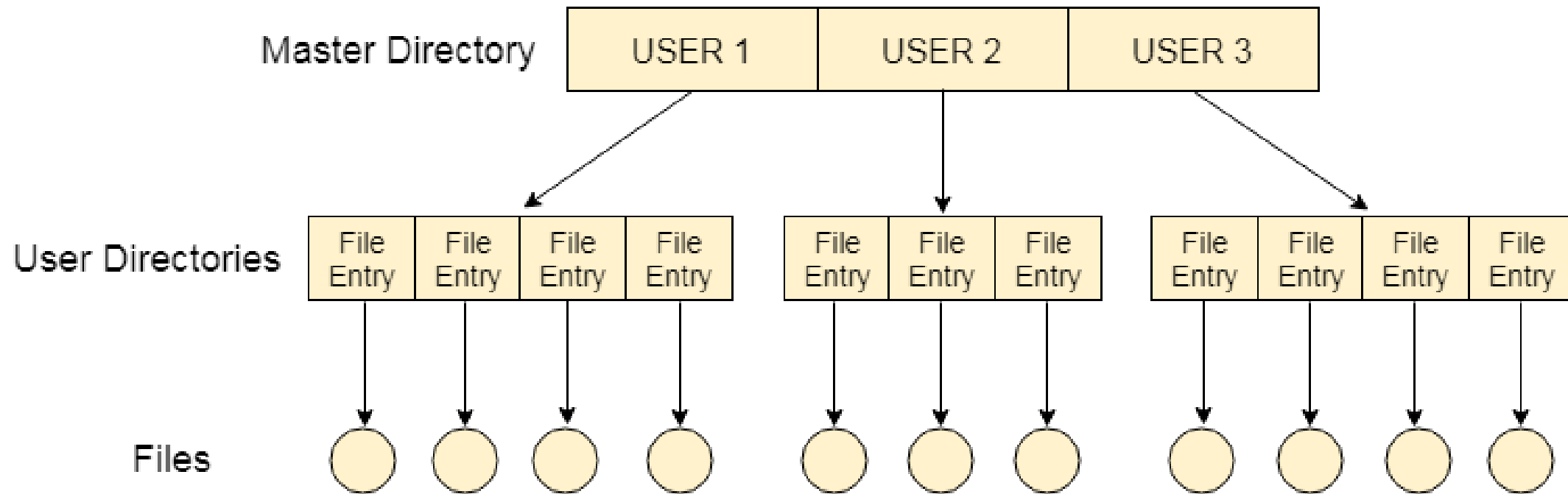


Single Level Directory

Two level directory

- In two level directory systems, we can create a separate directory for each user.
- There is one master directory which contains separate directories dedicated to each user.
- For each user, there is a different directory present at the second level, containing group of user's file.
- The system doesn't let a user to enter in the other user's directory without permission.

Two level directory

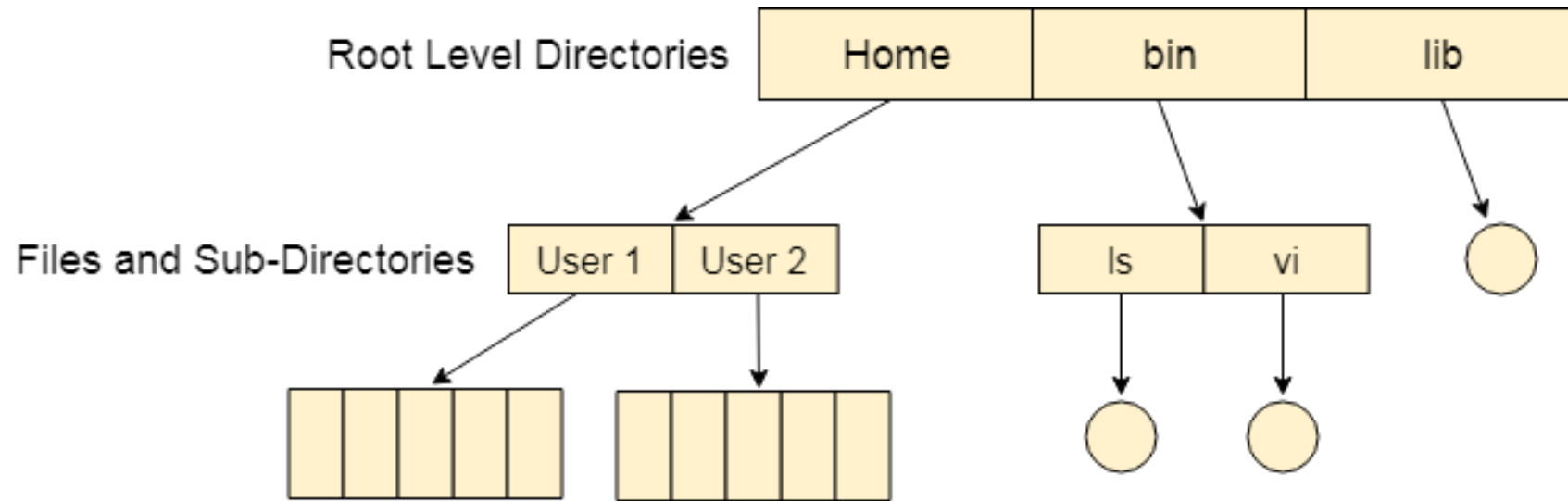


Two Level Directory

Tree-structured directory

- In Tree structured directory system, any directory entry can either be a file or sub directory.
- Tree structured directory system overcomes the drawbacks of two level directory system.
- Each user has its own directory and it cannot enter in the other user's directory.
- However, the user has the permission to read the root's data but he cannot write or modify this.
- Only administrator of the system has the complete access of root directory.

Tree structured directory

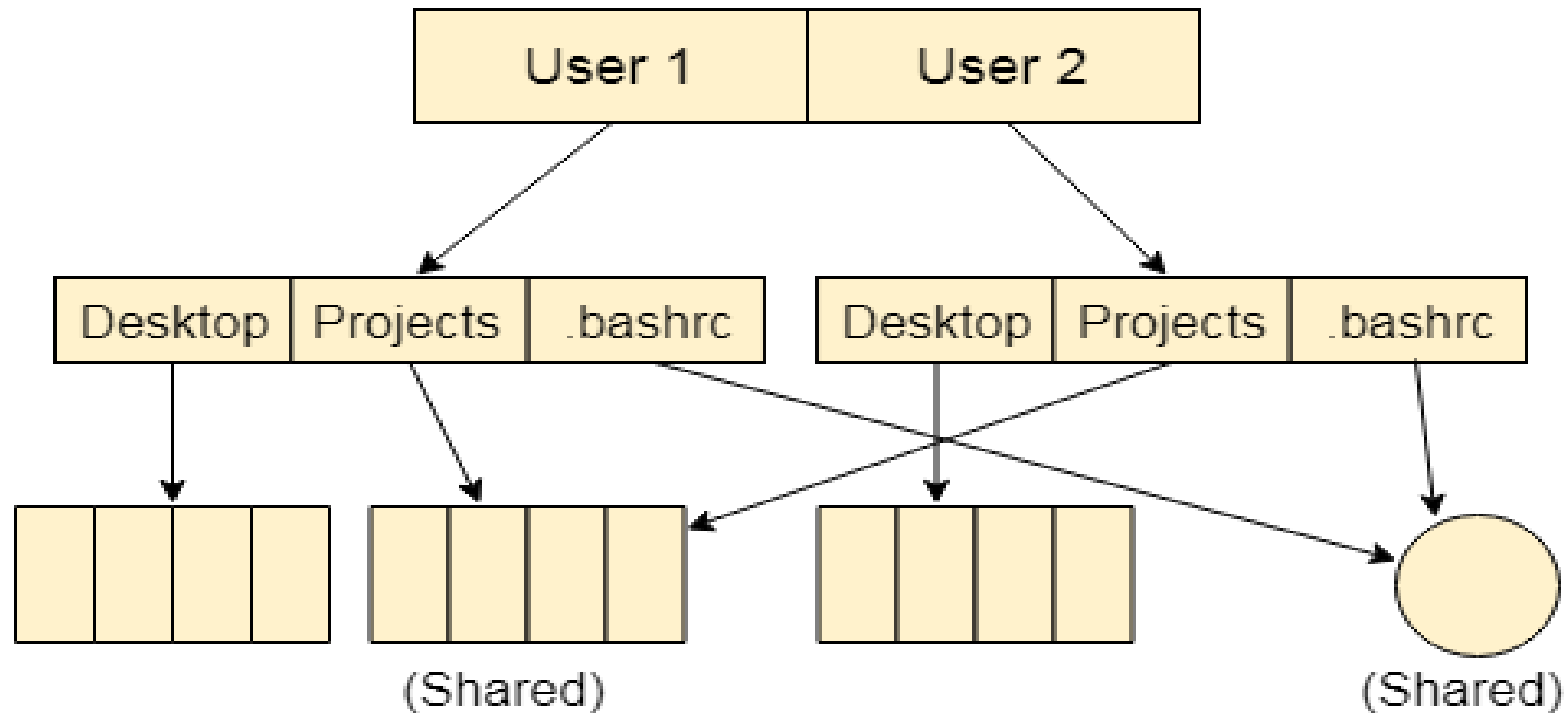


The Structured Directory System

Acyclic graph-structured directory

- The tree structured directory system doesn't allow the same file to exist in multiple directories therefore sharing is major concern in tree structured directory system.
- We can provide sharing by making the directory an acyclic graph. In this system, two or more directory entry can point to the same file or sub directory.
- That file or sub directory is shared between the two directory entries.
- These kinds of directory graphs can be made using links or aliases. We can have multiple paths for a same file.
- Links can either be symbolic (logical) or hard link (physical).

Acyclic graph-structured directory



Acyclic-Graph Structured Directory System

File systems

File system take care of following issues.

- **File structure**-The task of the file system is to maintain an optimal file structure from different data structures where file can be stored.
- **Recovering free space**-Whenever a file gets deleted from the hard disk, there is a free space created in the disk. There can be many such spaces which need to be recovered in order to reallocate them to other files.

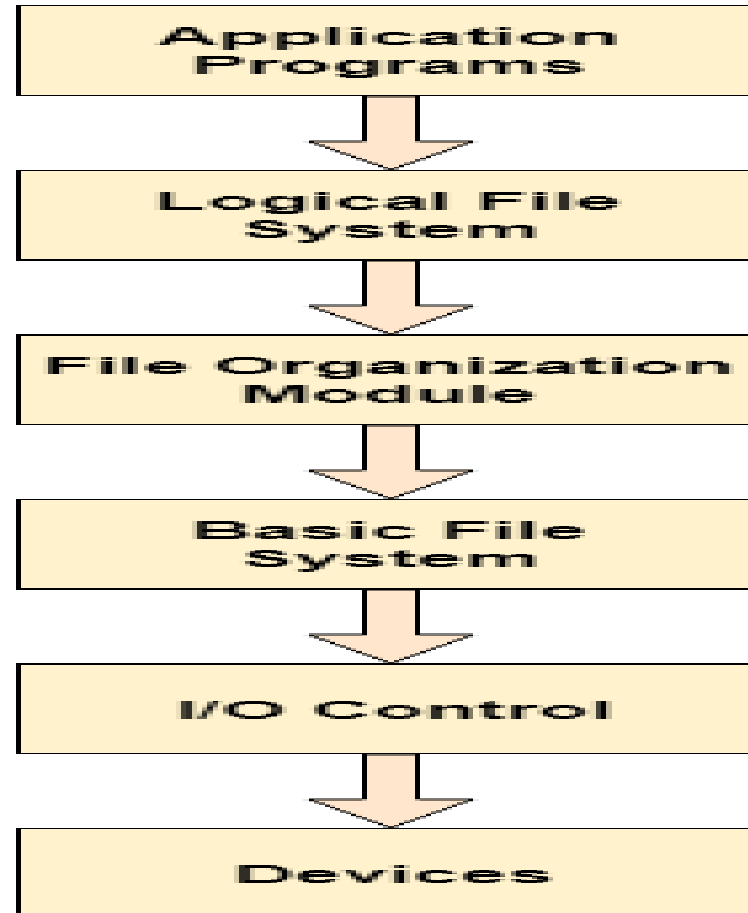
File systems

- **Disk space assignment to the files**-Using various disk scheduling algorithms can decide where to store files on the hard disk.
- **Tracking data location**-A File may or may not be stored within only one block. It can be stored in the non contiguous blocks on the disk. We need to keep track of all the blocks on which the part of the files reside.

File system structure

- File System provide efficient access to the disk by allowing data to be stored, located and retrieved in a convenient way.
- A file System must be able to store the file, locate the file and retrieve the file.
- Most of the Operating Systems use layering approach for every task including file systems. Every layer of the file system is responsible for some activities.
- The diagram elaborates how the file system is divided in different layers, and also the functionality of each layer.

File system structure



File system structure

- When an application program asks for a file, the first request is directed to the logical file system. The logical file system contains the Meta data of the file and directory structure. If the application program doesn't have the required permissions of the file then this layer will throw an error.
- Generally, files are divided into various logical blocks. Files are to be stored in the hard disk and to be retrieved from the hard disk. Hard disk is divided into various tracks and sectors. Therefore, in order to store and retrieve the files, the logical blocks need to be mapped to physical blocks. This mapping is done by File organization module. It is also responsible for free space management.

File system structure

- Once File organization module decided which physical block the application program needs, it passes this information to basic file system. The basic file system is responsible for issuing the commands to I/O control in order to fetch those blocks.
- I/O controls contain the codes by using which it can access hard disk. These codes are known as device drivers. I/O controls are also responsible for handling interrupts.

Input/Output Management

ICT1308

M.M.A.Nisansala

Input/Output

- OS controls all the I/O devices.
 - Issue commands to the devices
 - Catch interrupts
 - Handle errors
- It should also provide an interface between the devices and the rest of the system that is simple and easy to use.
 - Interface is device independent
- This section is related to how the H/W has programmed not how they work.

I/O Devices

- Two types:
 1. Block devices
 - Stores information in fixed-size blocks ranging from 512-65,536 bytes
 - Each block has its address
 - Possible to read or write each block independently
 - Ex-Hard disks, Blu-ray discs, and USB sticks
 2. Character devices
 - Delivers or accepts a stream of characters
 - Not addressable and no seek operations
 - Ex-Printers, network interface, mice(for pointing)

I/O Devices

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Scanner at 300 dpi	1 MB/sec
Digital camcorder	3.5 MB/sec
4x Blu-ray disc	18 MB/sec
802.11n Wireless	37.5 MB/sec
USB 2.0	60 MB/sec
FireWire 800	100 MB/sec
Gigabit Ethernet	125 MB/sec
SATA 3 disk drive	600 MB/sec
USB 3.0	625 MB/sec
SCSI Ultra 5 bus	640 MB/sec
Single-lane PCIe 3.0 bus	985 MB/sec
Thunderbolt 2 bus	2.5 GB/sec
SONET OC-768 network	5 GB/sec

Figure 5-1. Some typical device, network, and bus data rates.

Device Controllers

- I/O units consist of two parts-
 1. Mechanical components-Device itself
 2. Electronic components-Device controller
- The controller card usually has a connector on it, into which a cable leading to the device itself can be plugged.
- Interface between controller and device can be a standard interface like ANSI,IEEE or ISO standard.
- Controller can handle multiple devices.

Device controllers

- Interface between device and controller is often very low level one.
 - Disk drive
 - Controller converts serial bit stream in to error-free blocks to be copied into memory
 - Controller for LCD display
 - Take bytes of characters from memory and signal the backlight for the corresponding pixel for the characters to be displayed.
 - OS involves only by writing parameters to controller.

Memory Mapped I/O

- How CPU communicate with control registers and device data buffers.

1. First Approach

- Control register is assigned to I/O port number in I/O port space
 - 8-16 bit integer
 - Resides in RAM where user programs cant access
 - Use special instructions »

Ex: IN REG,PORT

CPU can read in control register PORT and store the result in CPU register REG

OUT PORT,REG

CPU can write the contents of REG to a control register

Memory Mapped I/O

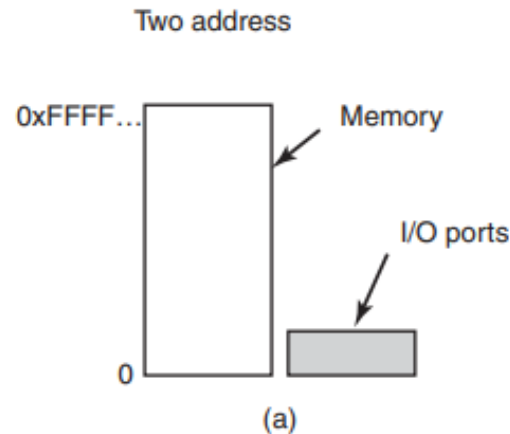
- In this scheme, the address spaces for memory and I/O are different.

IN R0,4

- reads the contents of I/O port 4 and puts it in R0

MOV R0,4

-reads the contents of memory word 4 and puts it in R0

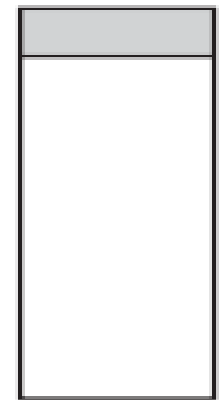


Memory Mapped I/O

2. Second approach

- Introduced with the PDP-11
- Map all the control registers into the memory space
- Each control register is assigned a unique memory address to which no memory is assigned.
- In most systems, the assigned addresses are at or near the top of the address space. Fig.(b))

One address space

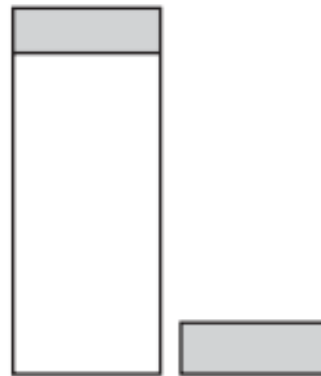


(b)

Memory Mapped I/O

- Also it can be a hybrid scheme, with memory-mapped I/O data buffers and separate I/O ports for the control registers(Fig.(C)) .

Two address spaces



(c)

Memory Mapped I/O

- How do these schemes actually work?
 - When the CPU wants to read a word, either from memory or from an I/O port, it puts the address it needs on the bus address lines and then asserts a READ signal on a bus' control line.
 - Second signal line is used to tell whether I/O space or memory space is needed.
 - If it is memory space, the memory responds to the request.
 - If it is I/O space, the I/O device responds to the request.

Memory Mapped I/O

- Advantages
 - No need of assembly code to access IO registers
 - no special protection mechanism is needed to keep user processes from performing I/O.
 - every instruction that can reference memory can also reference control registers
 - Otherwise 2 instructions are needed

Direct Memory Access

- CPU needs to address Device controllers to exchange data.
- Can ask one byte at a time.
- Waste CPU time
- With DMA, it is easy
- Memory, devices and CPU is communicating via a single System Bus that connects the CPU, the memory, and the I/O devices.
- With one DMA controller can regulate transfer to multiple devices concurrently.
- DMA controller has access to the system bus independent of the CPU

Direct Memory Access

- DMA controller has registers
 - Memory address register, a byte count register, and one or more control registers.
- Without DMA, the CPU has to read data from the controller's buffer.
- With DMA
 - CPU writes to DMA registers what to transfer where
 - Also commands to device register to write to its buffer.
 - Then DMA starts

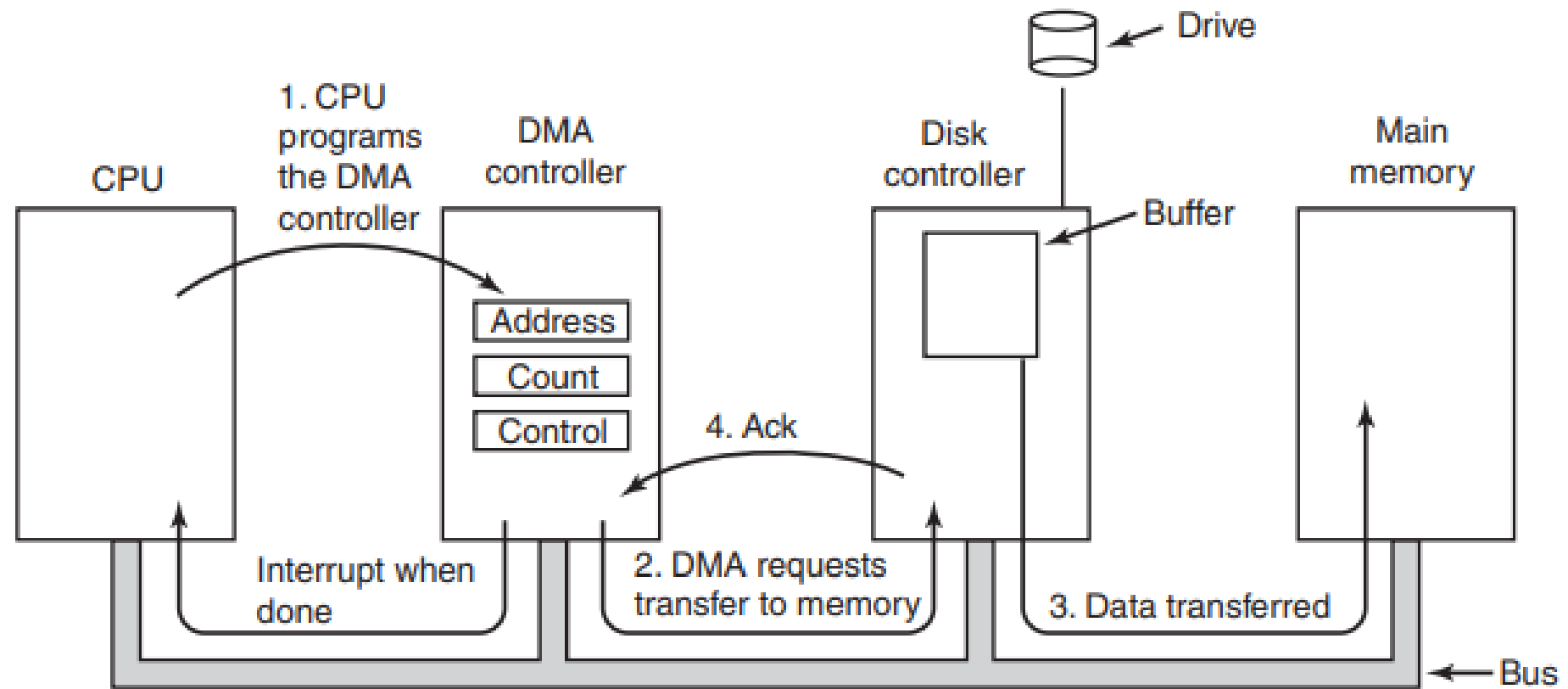


Figure 5-4. Operation of a DMA transfer.

Direct Memory Access

- While having one transfer at time, some DMA controllers can be programmed
 - To have multiple transfer at time
 - Sets of registers
 - After one transfer controller decides which to next
 - Use round robin algorithm

Direct Memory Access

- Many buses can operate in two modes:
 1. word-at-a-time mode
 - DMA controller employs cycle stealing mechanism – Transfers on word and releases the bus
 2. Block mode
 - DMA controller makes the device acquire the bus and do the transferring
 - This is called burst mode
 - Efficient since acquiring bus is expensive
 - Can block the CPU for a substantial time

Interrupt Handling

- When the I/O device finishes its work, it causes an interrupt
 - It is done by asserting a signal on a bus line that has been assigned
- This is detected by the interrupt controller
 - The interrupt controller interrupts the CPU specifying the device to be serviced
 - This is done through the interrupt-request line
- CPU stops what it is doing, saves current state, and starts doing something else
 - It looks for a new program counter

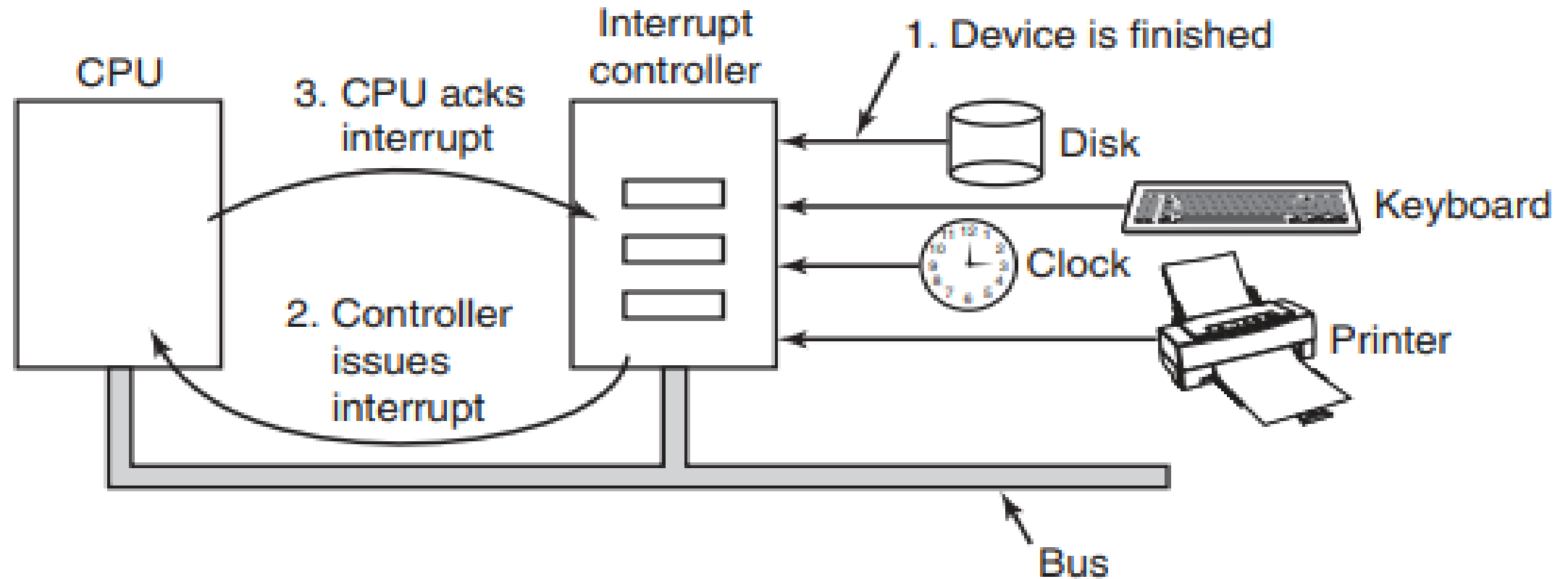
Interrupt Handling

- The number which defines the device is used as an index to the interrupt vector
 - This will fetch the new program counter
- This program counter points to the corresponding interrupt service procedure
 - The interrupt service procedure acknowledges the interrupt controller
 - The acknowledgment tells the controller that it is free to issue another interrupt
- Before leaving the processing, the CPU saves its current information in the kernel stack

Interrupt Handling

- An interrupt that leaves the machine in a well-defined state is called a precise interrupt
 - The PC is saved in a known place
 - All instructions before the one pointed to by the PC have fully executed
 - No instruction beyond the one pointed to by the PC has been executed
 - The execution state of the instruction pointed to by the PC is known
- An interrupt that does not meet these requirements is called an imprecise interrupt

Interrupt Handling



Input Output Software

- Goal of IO software is achieving **device independence**.
- Which means is that we should be able to write programs that can access any I/O device without having to specify the device in advance
- OS is the one worries about device characteristics.

Issues for IO Software

- A key concept is device independence
 - Uniform naming is a closely related goal -The name of a file or a device should simply be a string or an integer and not depend on the device in any way. All devices integrated in a single hierarchy
- Error handling
 - Errors should be handled as close to the hardware as possible.. Only if lower layers are unable to deal should upper layers be told about it
- Synchronous vs asynchronous transfers
 - User programs are much easier to write if the I/O operations are blocking. After a read system call the program is automatically suspended until the data are available in the buffer.

Issues for IO Software

- Buffering

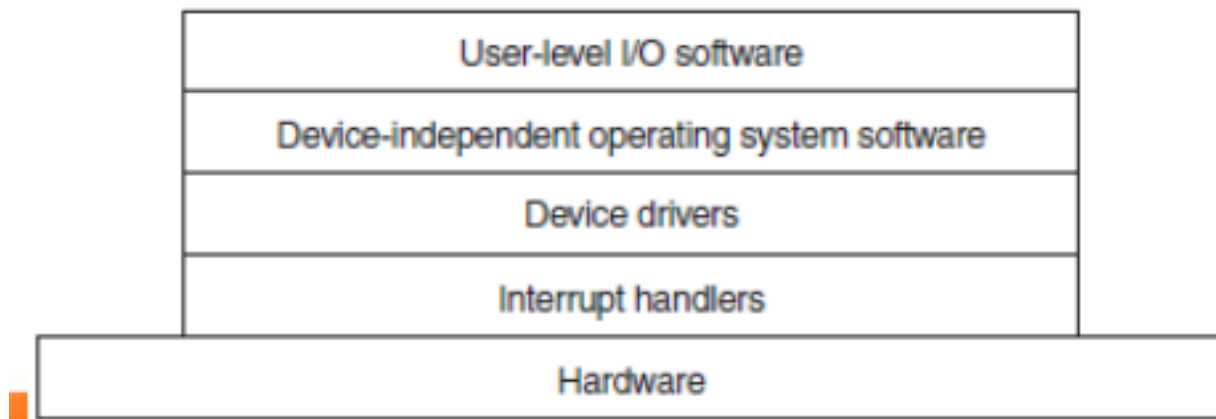
- Often data that come off a device cannot be stored directly in their final destination.
- Buffering involves considerable copying and often has a major impact on I/O performance. Has to store data until it is read by the OS.

- Sharable vs Dedicated devices

- Keeping track of multiple requests in sharable
- Avoiding synchronizing problems in dedicated

IO Software Layers

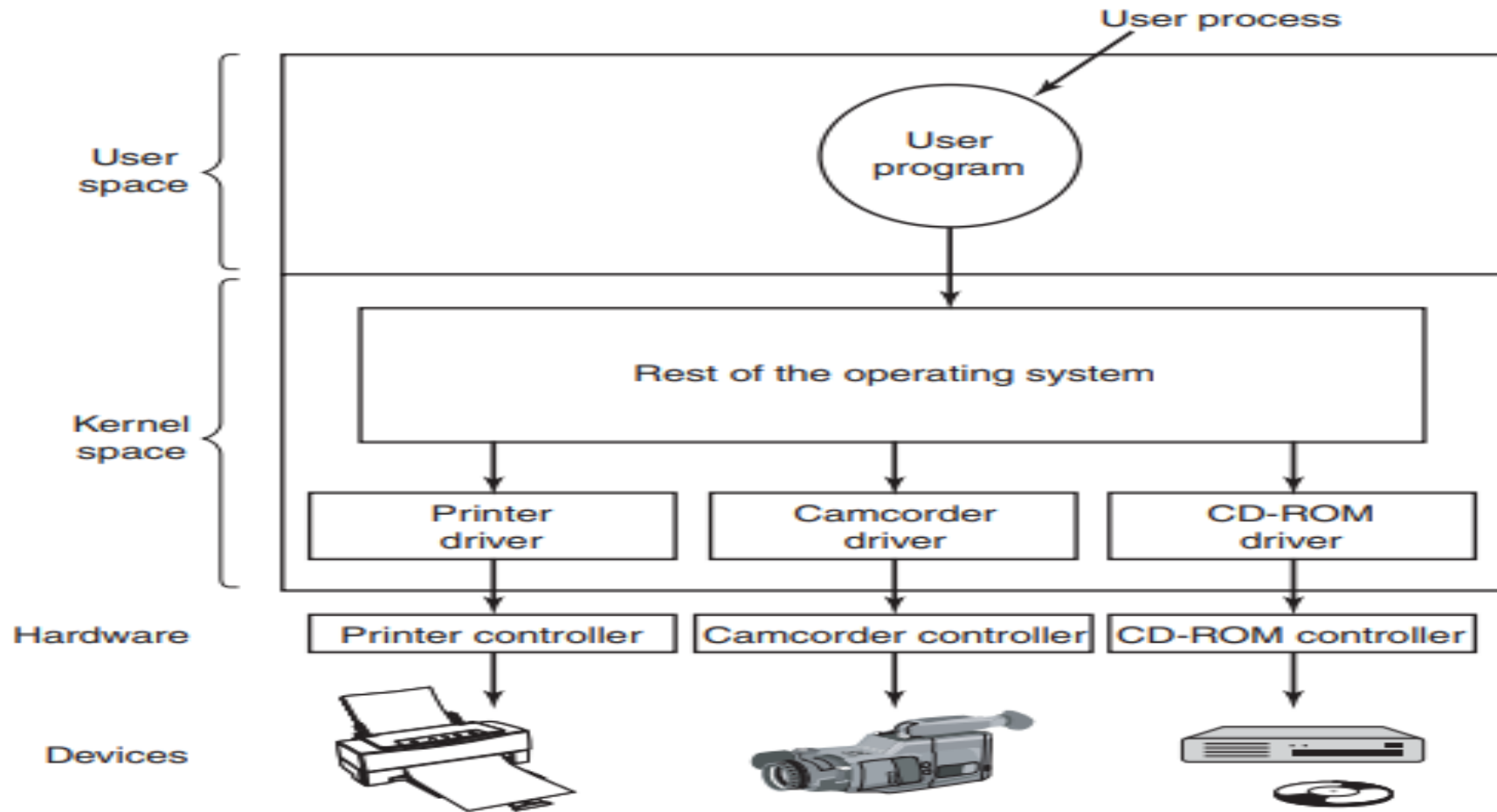
- IO software is organized in to layers
- Each layer handles a specific function and become an interface to the adjacent layer.
 - These may be differ from one system to another



Device drivers

- Each I/O device attached to a computer needs some device-specific code for controlling it.
- This code, called the **device driver**, is generally written by the device's manufacturer and delivered along with the device.
- Since each operating system needs its own drivers, device manufacturers commonly supply drivers for several popular operating systems.
- Driver can handle one or same class devices .

Device drivers



Device drivers

- Two types of drivers are:
 1. Block devices - Disks, which contain multiple data blocks that can be addressed independently.
 2. Character devices - Keyboards and printers, which generate or accept a stream of characters
- Most OSs define a standard interface that all block drivers must support and a second standard interface that all character drivers must support.

Device Drivers

- Device driver has several functions:
 1. Accept abstract read and write requests from the device-independent software.
 2. Translation between abstract to concrete terms of the parameters.
 3. Check device status

Device Drivers

- When an interrupt occurs, device driver blocks
 - Interrupt handler will execute the interrupt service procedure
 - Then unblock device driver
- Device Drivers
 - Control peculiarities of devices
 - Usually run in kernel space
 - OS define different interfaces for block drivers and character drivers
 - Dynamically loaded into the system

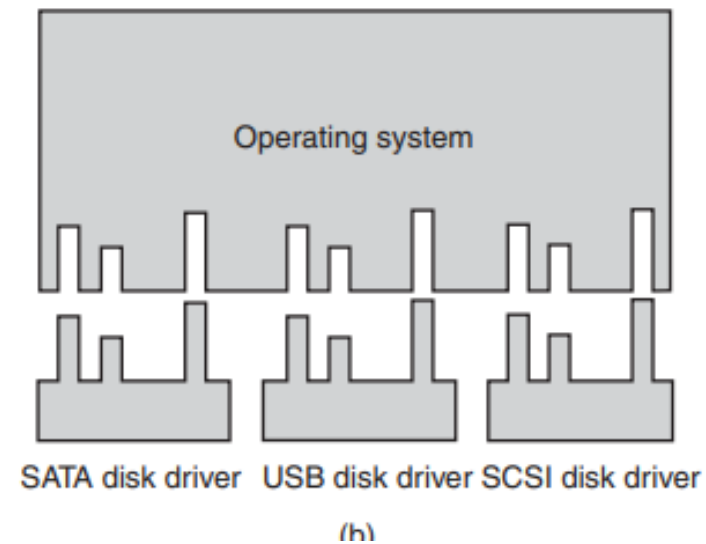
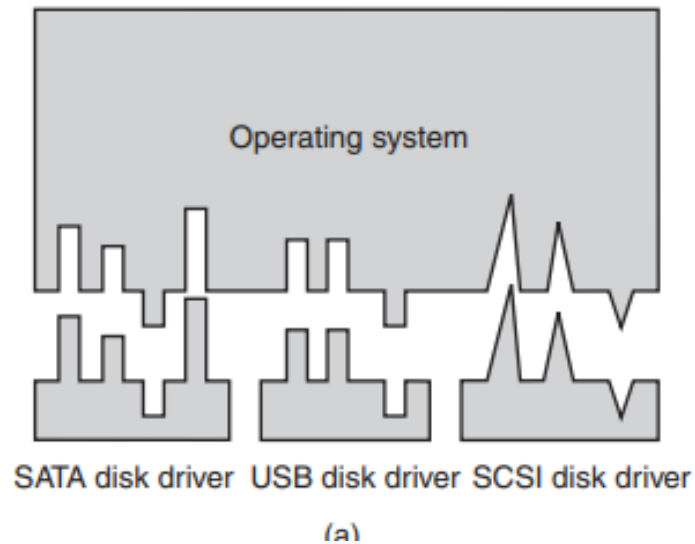
Device independent I/O Software

- Although some of the I/O software is device specific, other parts of it are device independent.
- The basic function of the device-independent software is to perform the I/O functions that are common to all devices and to provide a uniform interface to the user-level software.

Uniform interfacing for device drivers
Buffering
Error reporting
Allocating and releasing dedicated devices
Providing a device-independent block size

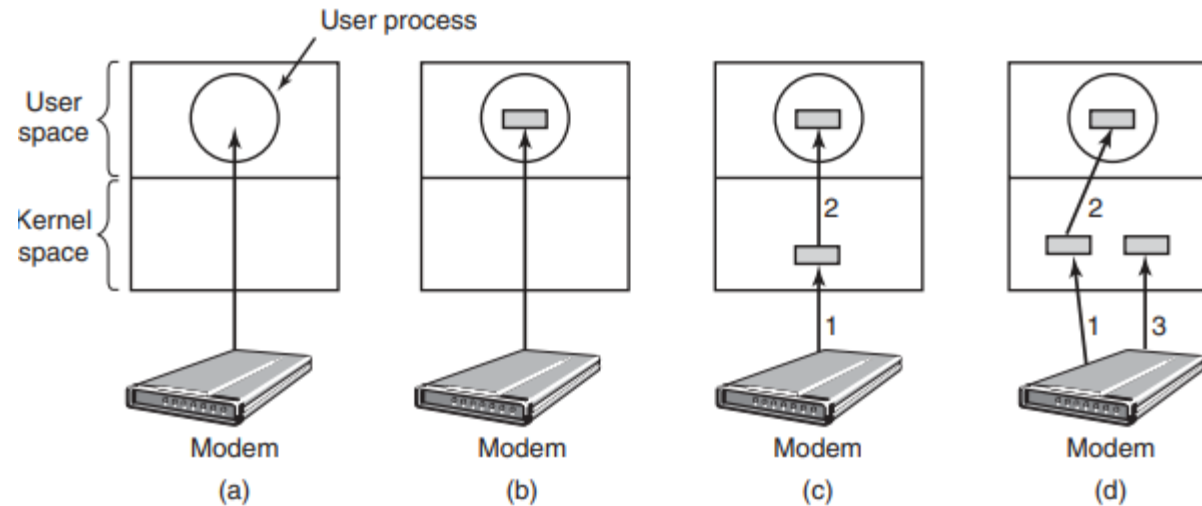
Device drivers

- Device-independent software has many functions
 - Uniform interface for device drivers
 - Protection is achieved



Device drivers

- In figure (a) driver function differs from driver to driver. It means interfacing each new driver requires a lot of new programming effort.
- In figure(b) different designs in which all drivers have same interface.
- Buffering



Device drivers

- Fig(a)-User process do a read system call and block waiting for one character. Each arriving character causes an interrupt. The interrupt-service procedure hands the character to the user process and unblocks it. After putting the character somewhere, the process reads another character and blocks again. The trouble with this way of doing business is that the user process has to be started up for every incoming character. Allowing a process to run many times for short runs is inefficient, so this design is not a good one.
- Fig(b)- User process provides an n-character buffer in user space and does a read of n characters until buffer is full. Then only wakes up the user process. Drawback is if buffer is paged out when character arrives it will be a issue.
- Fig(c)- Create a buffer inside the kernel and have the interrupt handler put the characters there. When this buffer is full, the page with the user buffer is brought in. The issue is what happens to characters that arrive while the page with the user buffer is being brought in from the disk?

Device drivers

- Fig(d)-Having a kernel buffer will solve the above mentioned issue. After the first buffer fills up, but before it has been emptied, the second one is used. When the second buffer fills up, it is available to be copied to the user (assuming the user has asked for it). While the second buffer is being copied to user space, the first one can be used for new characters.
- Error reporting
 - Programming errors- These occur when a process asks for something impossible, such as writing to an input device (keyboard, scanner, mouse, etc.) or reading from an output device (printer, plotter, etc. Other errors are providing an invalid buffer address or other parameter, and specifying an invalid device (e.g., disk 3 when the system has only two disks), and so on

Device driver

- I/O errors- trying to write a disk block that has been damaged or trying to read from a camcorder that has been switched off.
- Allocating and releasing dedicated devices
 - Devices like printers can be used only by a single process at a time. It is up to OS to examine requests for device usage and accept or reject them depending on whether requested device is available or not.
- Device independent block size
 - Different disks have different sector sizes. It is up to device independent software to hide this fact and provide uniform block size to higher layers.

Thank You!

